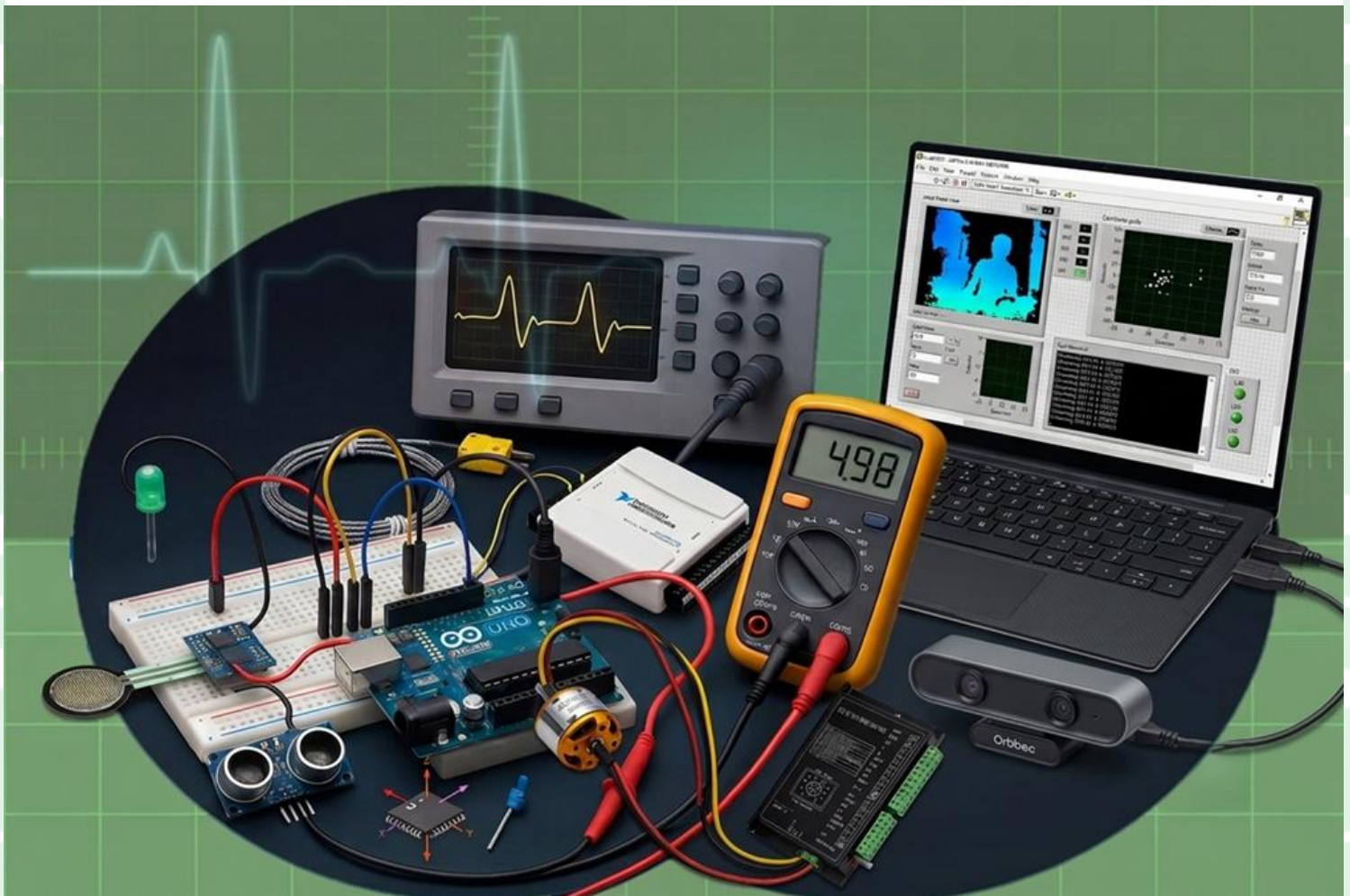


SENSORES E INSTRUMENTACIÓN VIRTUAL PARA INGENIERÍA APLICADA

DEL ACONDICIONAMIENTO A LA VISUALIZACIÓN INTERACTIVA



Primera Edición. 2026

PhD, MSc. Frank Alexander Ruiz Holguín
PhD. Andrés Felipe López García
PhD. MSc. Diana María Gómez Jaramillo



**UNIVERSIDAD
DE ANTIOQUIA**

SENSORES E INSTRUMENTACIÓN VIRTUAL PARA INGENIERÍA APLICADA

DEL ACONDICIONAMIENTO A LA VISUALIZACIÓN INTERACTIVA

PhD, MSc. Frank Alexander Ruiz Holguín
PhD. Andrés Felipe López García
PhD. MSc. Diana María Gómez Jaramillo



**UNIVERSIDAD
DE ANTIOQUIA**

SENSORES E INSTRUMENTACIÓN VIRTUAL PARA INGENIERÍA APLICADA: DEL ACONDICIONAMIENTO A LA VISUALIZACIÓN INTERACTIVA

PhD, MSc. Frank Alexander Ruiz Holguín^{1,2}

PhD. Andrés Felipe López García²

PhD. MSc. Diana María Gómez Jaramillo²

¹Institución Universitaria Pascual Bravo

²Universidad de Antioquia



**UNIVERSIDAD
DE ANTIOQUIA**

Luquegi Gil Neira

Rector de la Universidad de Antioquia

Maribel Barreto Mesa

Vicerrectora de Docencia

Elkin Libardo Ríos Ortiz

Decano de la Facultad de Ingeniería

Ricardo Emilio Aristizábal Sierra

Vicedecano de la Facultad de Ingeniería

© Frank Alexander Ruiz Holguín

© Andrés Felipe López García

© Diana María Gómez Jaramillo

Autores y expertos temáticos

Primera edición: Medellín, agosto de 2026

ISBN: ####

e-ISBN: ####

Esta publicación es un producto de Ude@ Educación Virtual de la Universidad de Antioquia. No se permite la distribución, copia, duplicación, reproducción, o venta (total o parcial) del contenido sin la previa autorización escrita por parte del titular. Todos los derechos reservados.

Medellín, Colombia

Tabla de contenido

Prólogo	6
Capítulo 1	7
Instalación de LabVIEW	8
Librerías LINX-MakerHub (Hobbyist), VISA y DAQmx	9
Ubicación de las librerías LINX-MakerHub o Hobbyist	9
Descarga de librerías LINX (Hobbyist toolkit)	11
Bloques de las librerías Hobbyist	13
Instalación de las librerías NI-VISA y NI-DAQmx	15
.....	21
Capítulo 2	21
Prácticas de Laboratorio	21
Práctica No.1	22
Sensado de temperatura con LM35, termistor y termopar tipo K	22
Práctica No.2	33
Sensado de niveles de presión usando sensor piezoeléctrico	33
Práctica No.3	40
Sensado de peso con celdas de carga y transmisión serial	40
Práctica No.4	48
Medición de posición usando acelerómetros tri-axiales	48
Práctica No.5	67
Medición de distancia por ultrasonido e imágenes de profundidad	67
Práctica No.6	88
Sensado y control de velocidad de un motor DC y control de marcha y sentido de giro de un motor Brushless	88
Práctica No.7	107
Transmisión inalámbrica de datos con módulos XBEE en modo API	107
Práctica No.8	124
Transmisión de datos usando protocolo HTTP y LabVIEW: aplicaciones de Internet of Thing- IoT	124

Prólogo

En el vertiginoso avance de la ingeniería moderna, la medición y adquisición de datos en tiempo real se han convertido en pilares fundamentales para el análisis, el control y la automatización de procesos. La integración de sensores, el procesamiento de señales y la visualización de información en entornos gráficos han permitido optimizar sistemas en una amplia gama de aplicaciones, desde la industria automotriz hasta la automatización industrial y el Internet de las Cosas (IoT). Este libro, "Sensores e instrumentación virtual para ingeniería aplicada: del acondicionamiento a la visualización interactiva", nace como una herramienta pedagógica diseñada para fortalecer la formación práctica de estudiantes, docentes e investigadores en el campo de la instrumentación y el acondicionamiento de señales.

Este libro ha sido estructurado cuidadosamente para abordar, de manera progresiva, el funcionamiento y aplicación de diversos sensores utilizados en la medición de variables físicas fundamentales. A lo largo del texto, el lector encontrará prácticas detalladas sobre sensores de temperatura, como termopares, LM35 y termistores, sensores de presión piezoeléctricos, acelerómetros y giroscopios para la medición de posición y orientación, así como encoders ópticos para la estimación de velocidad en motores eléctricos. Además, se incluyen ejercicios enfocados en la implementación de sistemas de adquisición de datos mediante plataformas como Arduino y tarjetas de mayor robustez, proporcionando un enfoque integral para la captura, procesamiento y análisis de señales en entornos reales.

Uno de los aspectos distintivos de este libro es la incorporación de entornos gráficos de monitoreo y control, desarrollados en la plataforma LabVIEW, lo que permite a los usuarios visualizar, interpretar y manipular los datos de manera intuitiva y eficiente. Asimismo, se incluyen prácticas innovadoras relacionadas con el IoT y la comunicación inalámbrica ZigBee, que son tecnologías clave en el desarrollo de sistemas inteligentes y conectados.

Más que un simple compendio de ejercicios de prácticas de laboratorio, este libro representa una guía integral para la comprensión de los principios que rigen la adquisición y el procesamiento de señales en ingeniería. Se ha procurado que cada práctica siga una metodología clara, proporcionando los fundamentos teóricos necesarios, una descripción detallada del procedimiento experimental y un análisis de los resultados esperados, fomentando así el aprendizaje basado en la experimentación y la resolución de problemas.

Esperamos que este libro sirva como un recurso valioso para quienes buscan consolidar sus conocimientos en instrumentación virtual y acondicionamiento de señales, promoviendo el desarrollo de competencias técnicas esenciales para enfrentar los desafíos tecnológicos del presente y del futuro.



Capítulo 1

En esta sección se describe la forma de realizar la búsqueda e instalación de las librerías de software necesarias para la preparación de los equipos de cómputo en los cuales se realizarán las prácticas de laboratorio. Antes de proceder con el alistamiento del equipo, asegúrese de tener instalado en su ordenador el software LabVIEW.

Instalación de LabVIEW

Para conocer qué es LabVIEW, cómo funciona y qué se puede hacer con esta plataforma, visite el siguiente enlace, allí podrá encontrar mayor información al respecto:

<https://www.ni.com/es/shop/LabVIEW.html>

Para proceder con la instalación de la plataforma de programación gráfica LabVIEW, visite el enlace dispuesto a continuación y siga los pasos:

<https://www.ni.com/es/support/downloads/software-products/download.LabVIEW.html#559067>

LabVIEW

NI LabVIEW es un entorno de programación gráfica que proporciona aceleradores de productividad únicos para el desarrollo de sistemas de pruebas, como un enfoque intuitivo para la programación, conectividad con cualquier instrumento e interfaces de usuario completamente integradas.

[+ Leer más](#)

DESCARGAS

SO Soportado: Windows [Ver notas de la versión](#)

Versión: 2026 Q1

Ediciones Incluidas:

- ☐ Base, Completo, Profesional
- ☒ **Comunidad**
- ☐ Runtime

Comunidad

Esta edición es solo para uso no comercial. Aprenda más sobre el [uso de la edición de la comunidad](#) y las [políticas de licencia de software de NI](#).

Valor de Bits de la Aplicación: 32 bits

Idioma: Inglés

Incluir Controladores de Software: No

LabVIEW 2026 Q1 Community

Fecha de Lanzamiento: 29/1/26

- > SO Soportado
- > Idioma
- > Suma de Verificación

Descargar

[Instalar fuera de línea >](#)

Tamaño del Archivo: 3.86 GB

Figura 1. Pantalla de descarga del software

Seleccione el sistema operativo y la versión del software. Para descargar la plataforma asegúrese de seleccionar la **última versión** disponible (versiones anteriores no están disponibles para la descarga gratuita) y escoja la edición de **comunidad**¹, esta edición es para uso no comercial. Oprima el botón “*Descargar*”, le llegará un mensaje al correo electrónico registrado para confirmar la descarga.

¹ Estudiantes inscritos en cualquier nivel de casos de uso de la academia

Los estudiantes inscritos en cualquier nivel de la academia pueden usar la edición de la comunidad de *LabVIEW* y la edición de la comunidad del *G Web Development software* en cualquier PC que sea propiedad del estudiante para cualquier propósito no comercial ni industrial.

Librerías LINX-MakerHub (Hobbyist), VISA y DAQmx

Ubicación de las librerías LINX-MakerHub o Hobbyist

Para establecer comunicación y programar una placa Arduino® (entre otros sistemas embebidos) a través de LabVIEW, se puede contar con las librerías *LINX de MakerHub*, las cuales vienen pre-instaladas en versiones posteriores a la versión 2020 inclusive. Se podrá verificar que estas librerías están correctamente instaladas siguiendo la siguiente ruta desde el Diagrama de Bloques de LabVIEW (Ver figura 2 y 3): **Functions/MakerHub/LINX**.

Para versiones más recientes de LabVIEW la ruta es: Functions/Hobbyist.

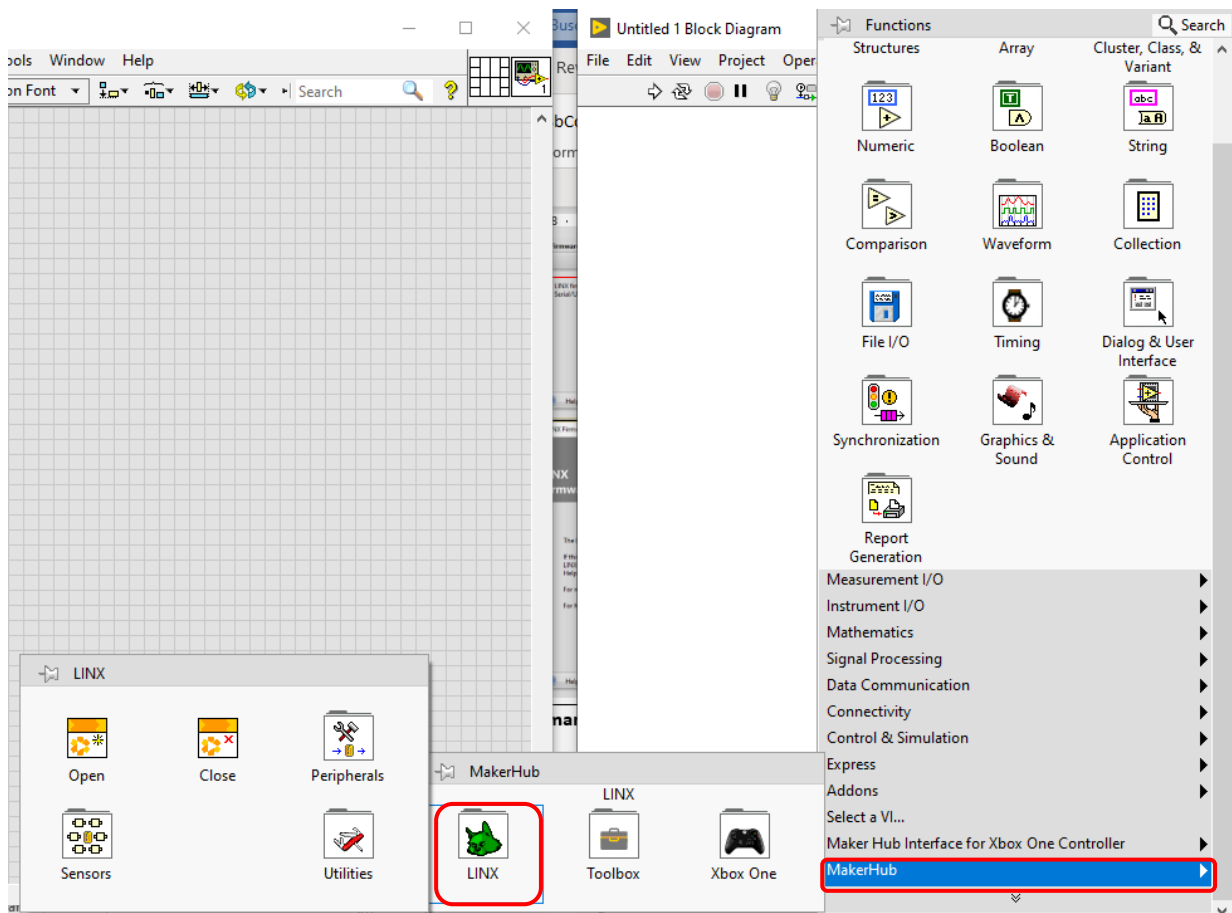


Figura 2. Ruta de acceso a las librerías LINX de Makerhub

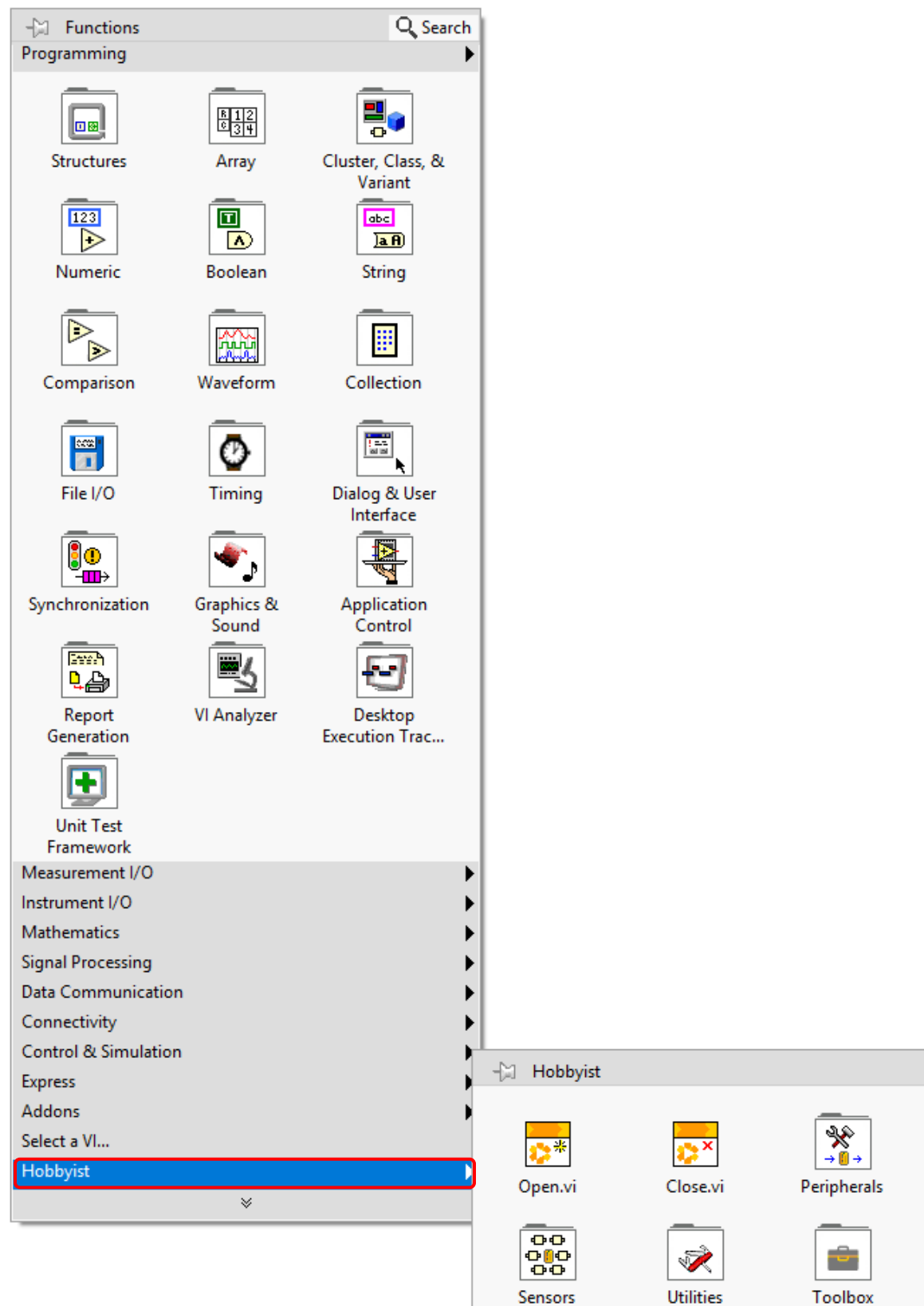


Figura 3. Ruta de acceso a las librerías Hobbyist

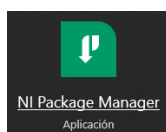
Si las librerías LINX (o Hobbyist) no aparecen instaladas, deberá seguir los siguientes pasos para su instalación:

Descarga de librerías LINX (Hobbyist toolkit)

Descargue las librerías del enlace proporcionado a continuación e instálelas en su equipo. Será necesario tener instalado previamente una versión de LabVIEW igual o superior a la versión de 2020:

<http://sine.ni.com/nips/cds/view/p/lang/es/nid/212478>

También podrá descargar estas librerías directamente desde el administrador de paquetes de LabVIEW (previamente instalado) **NI Package Manager (recomendado)**.



Para ello abra el administrador de paquetes y busque la palabra **LINX o Hobbyist**. Aparecerán las librerías que debe instalar (*LabVIEW Hobbyist Toolkit (32-bit)*). Luego de *click* en el ícono de descarga e instale las librerías.

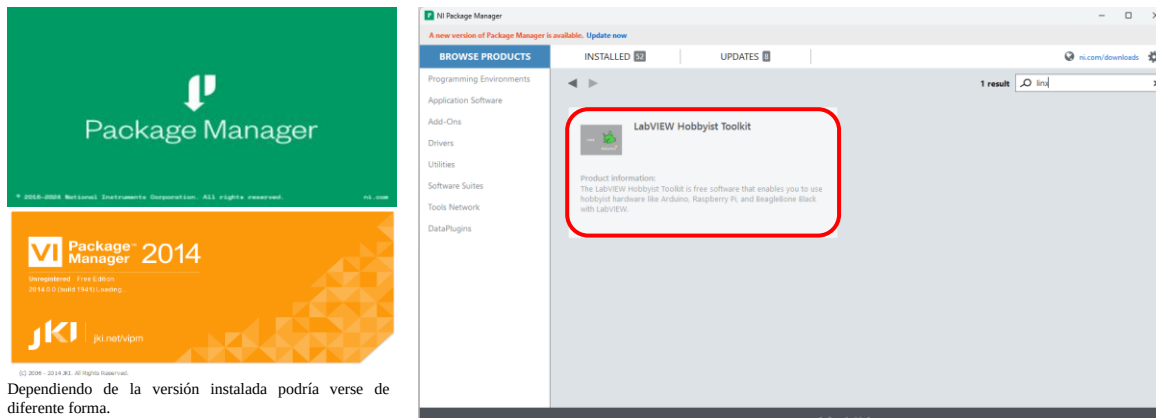


Figura 4. NI Package Manager. Descarga de las librerías *LINX o Hobbyist toolkit*

Algunos pasos más:

1. Vea el siguiente video tutorial donde se explica cómo instalar y usar la interfaz de LINX en LabVIEW

Nueva Interfaz LINX que reemplaza al LIFA (LabVIEW Interface For Arduino)

<https://www.youtube.com/watch?v=oiAA61eZHKY&nohtml5=False>

Estas librerías por su buen desempeño, incorporación de nuevos sensores y módulos y por su compatibilidad con LabVIEW y otros sistemas embebidos adicionales a Arduino, resulta ser una excelente interfaz para la programación de aplicaciones orientadas a la visualización, monitoreo y control de sistemas de bajo coste, sugiriendo el reemplazo directo de su interfaz predecesora **LIFA**.

Para Programar el ARDUINO desde LabVIEW, siga las instrucciones mostradas en la Figura 5 y 6.

En el Panel Frontal, haga *click* sobre la opción “*Tools*”, luego busque la opción **Hobbyist/ Firmware Wizard**

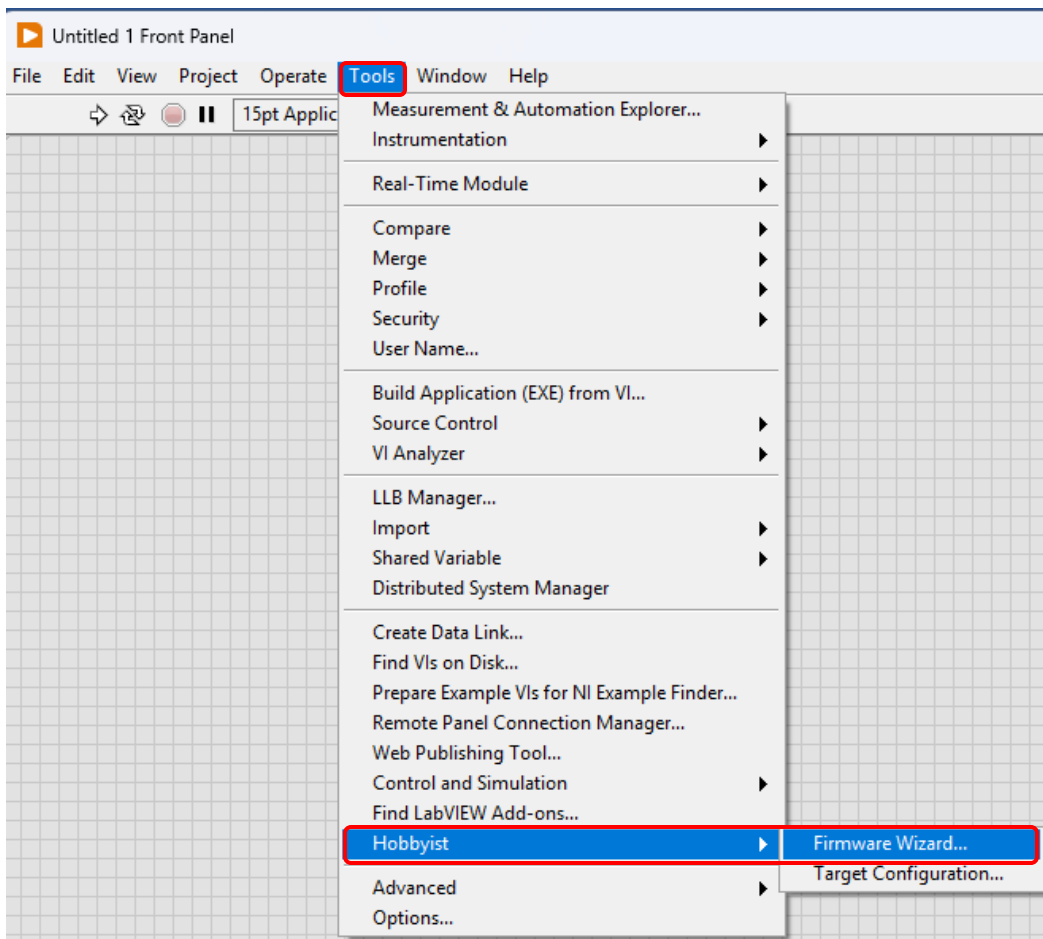


Figura 5. Ruta para programar Arduino desde LabVIEW usando Hobbyist

Siga los pasos del asistente de programación y termine con el botón “Finish”.

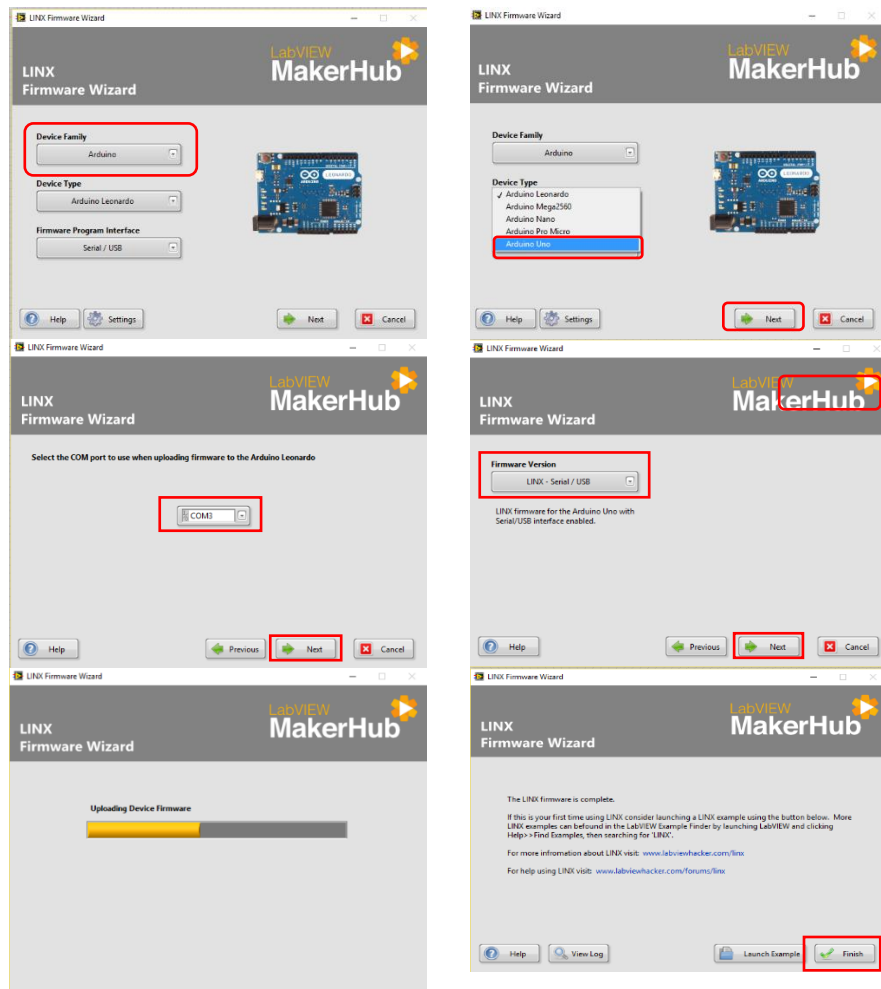


Figura 6. Pasos para programar la placa Arduino usando LINX

Bloques de las librerías Hobbyist

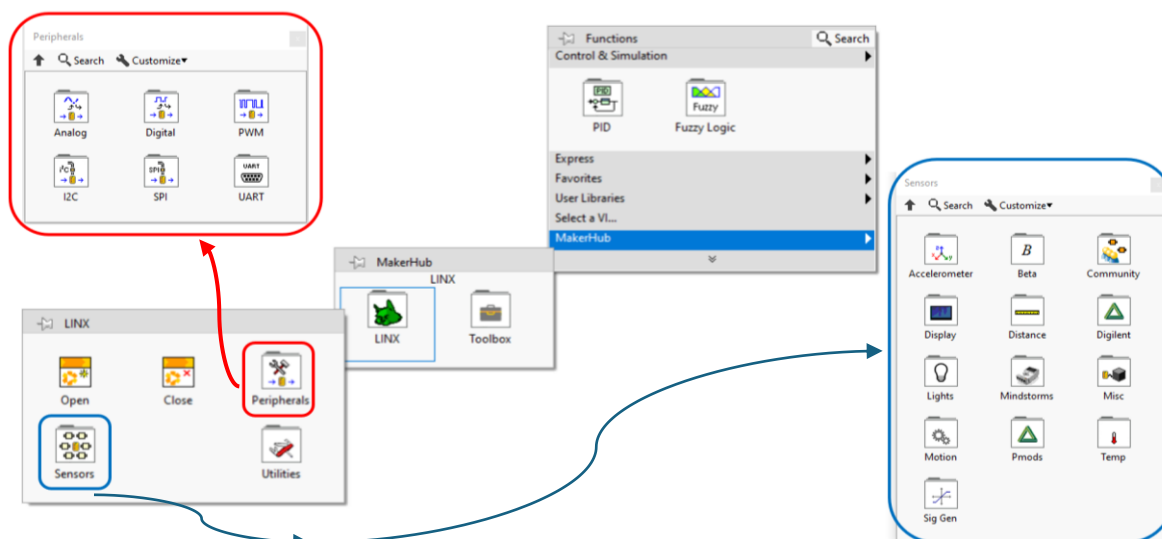


Figura 7. Bloques de las Librerías Hobbyist

Verificando el funcionamiento de una aplicación:

A continuación, se presentan algunos ejemplos de sistemas de adquisición de datos basados en Arduino y las librerías *Hobbyist de LabVIEW*. Usted podrá implementar estas aplicaciones y verificar su funcionamiento.

Lectura de 1 canal Analógico (A0)

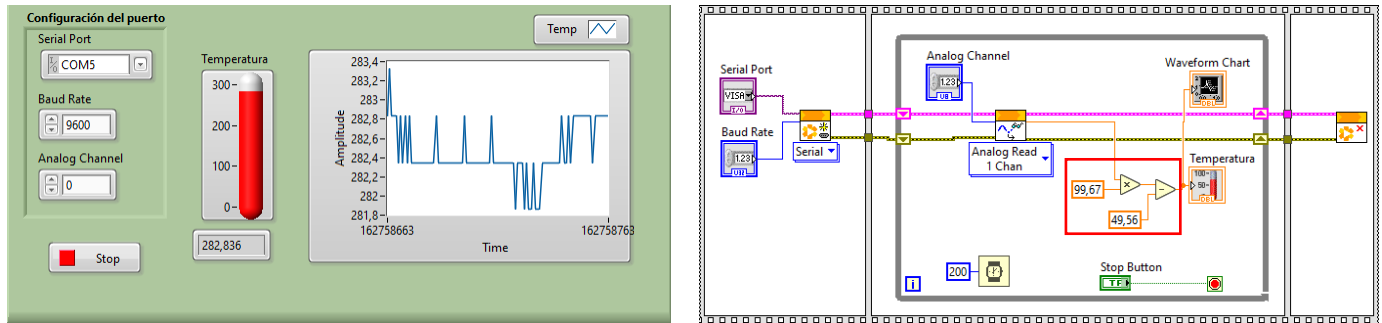


Figura 8. Lectura del canal Analógico A0 del Arduino

Tenga en cuenta que la ecuación usada en el diagrama de bloques de la figura 8 implementada para convertir las señales de voltaje [v] en magnitudes de temperatura [°C] (como en este caso), presión [bar], velocidad [rpm], posición [%], etc, dependerá de las características propias de cada sensor.

Lectura de 2 canales Analógicos (A0 y A5)

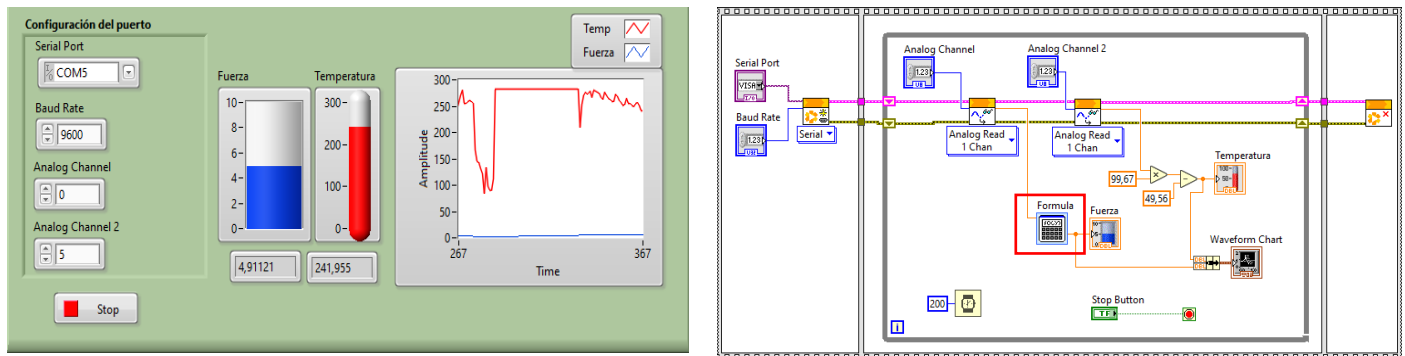


Figura 9. Lectura de los canales Analógicos A0 y A5 del Arduino

En la Figura 9, se muestra el bloque “**Fórmula**” usado para editar la ecuación característica del sensor que mide la variable Fuerza, en lugar de los bloques básicos de las operaciones elementales (suma, resta, multiplicación, división).

Lectura de 2 canales Analógicos (A0 y A1)

En la Figura 10, se ilustra el código para sensar dos variables directamente en la interfaz de LabVIEW, nótese que en esta oportunidad, las variables no han sido escaladas y se presentan en niveles de voltaje [v].

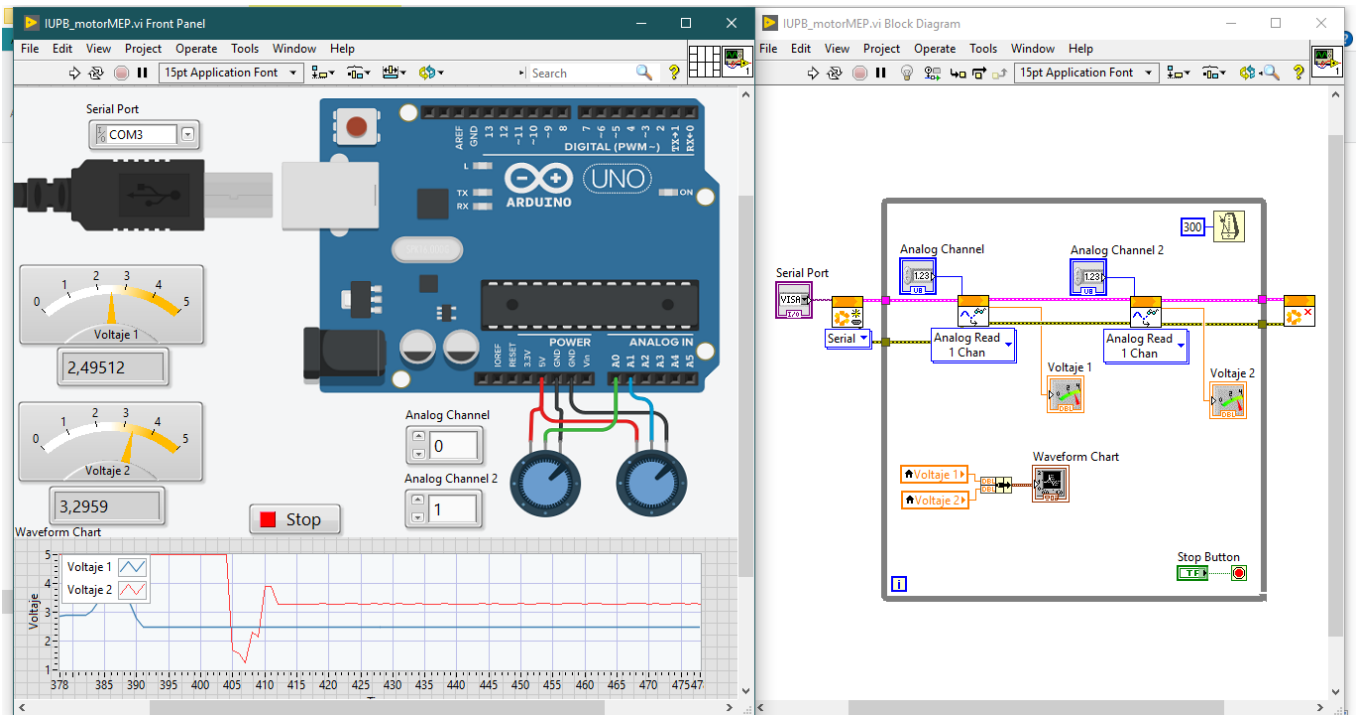


Figura 10. Lectura de los canales Analógicos A0 y A1 del Arduino

Instalación de las librerías NI-VISA y NI-DAQmx

Las librerías **NI-VISA** deben ser instaladas para establecer comunicación serial con los diferentes dispositivos conectados al PC a través de puertos USB. Los controladores **NI-DAQmx** deben ser instalados para el reconocimiento y la comunicación con tarjetas de adquisición de datos.

Para descargar tanto los NI- VISA como los NI-DAQmx, visite los siguientes links, allí encontrará acceso a la descarga de estos controladores (driver's).

NI-VISA:

<https://www.ni.com/es/support/downloads/drivers/download.ni-visa.html#590234>

DAQmx drivers:

<https://www.ni.com/es/support/downloads/drivers/download.ni-daq-mx.html#590033>

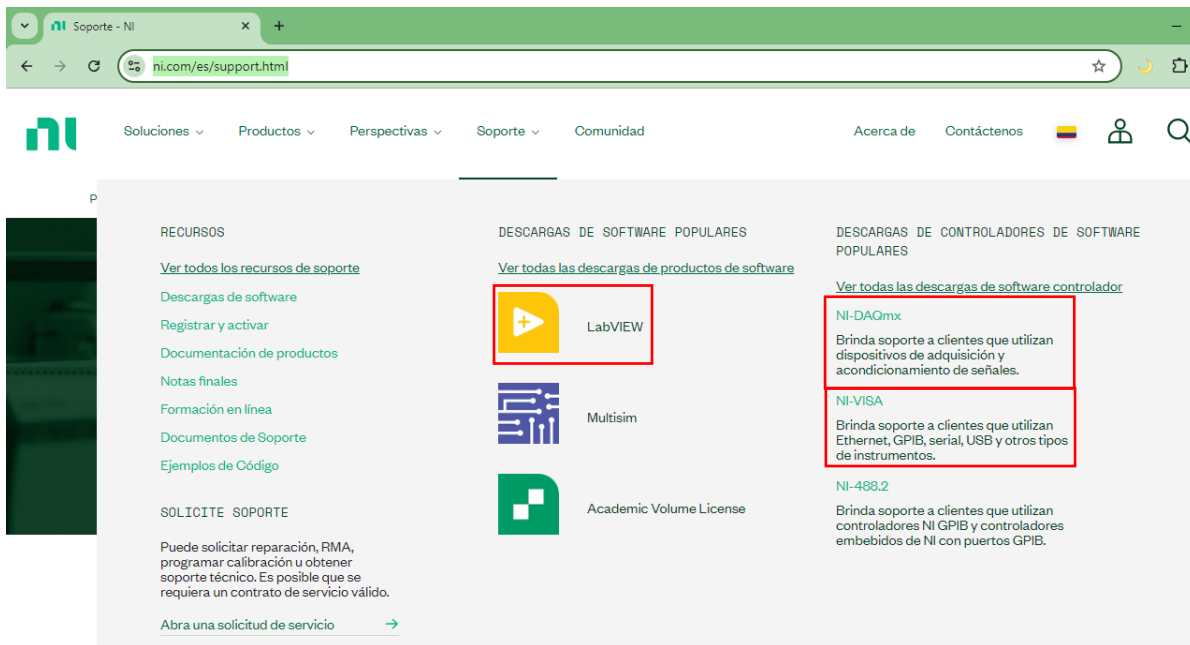


Figura 11. Página de descarga de controladores de comunicación serial VISA y de adquisición de datos DAQmx

Haga *click* en los enlaces respectivos y seleccione el sistema operativo y la versión que desea instalar. En la Figura 12 puede ver la ventana de descarga de los NI-VISA drivers y en la Figura 13, puede ver la ventana de descarga de los NI-DAQmx drivers.

Tanto para la descarga de los NI-VISA como de los NI-DAQmx será necesario iniciar sesión con una cuenta de la *National Instruments (ni)*. Si no posee cuenta, deberá crearla (es gratis), al correo electrónico registrado le llegará una notificación para la activación de la cuenta.

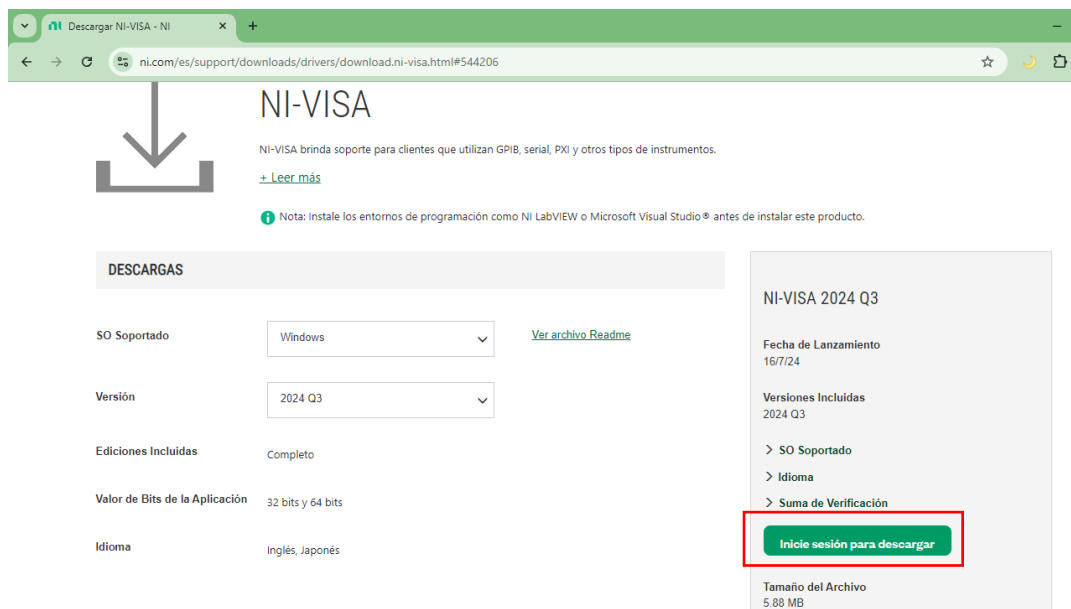


Figura 12. Ventana de descarga de controladores de comunicación serial NI-VISA

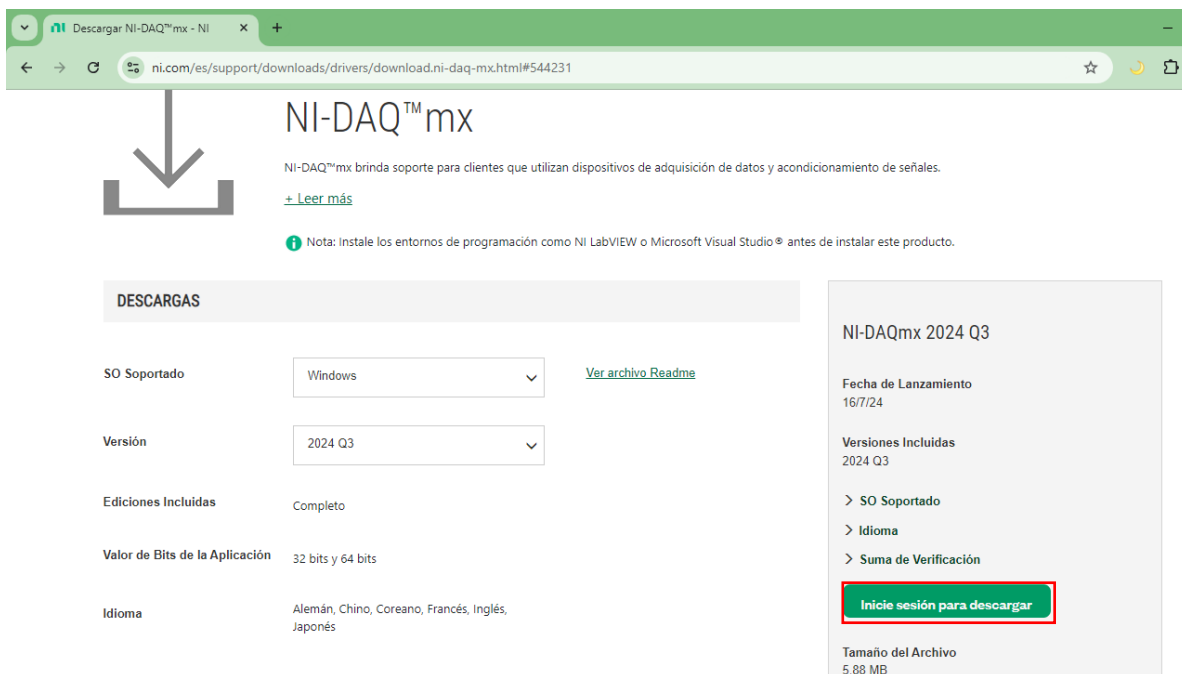
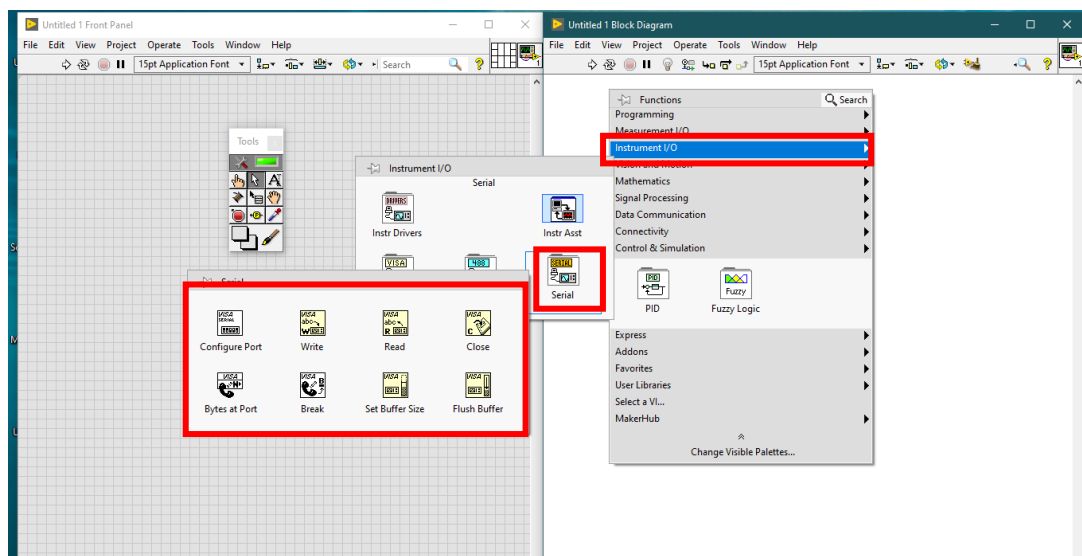


Figura 13. Ventana de descarga de controladores de adquisición de datos NI-DAQmx

A continuación, se presentan los bloques que hacen parte de los NI-VISA:



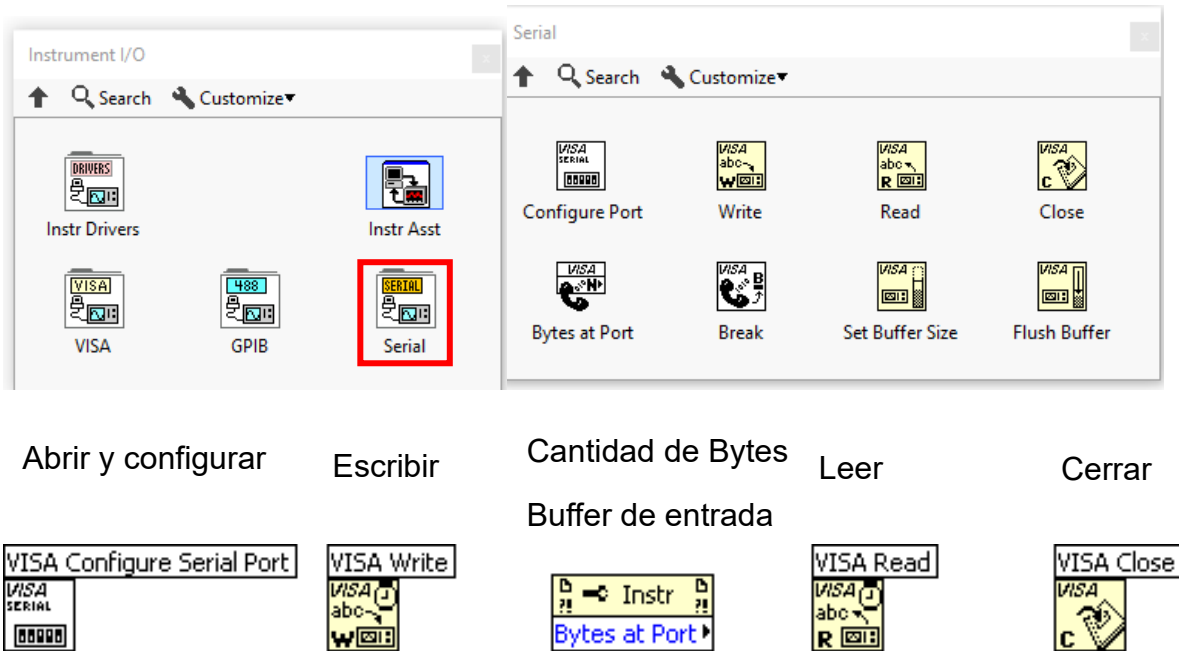


Figura 14. Bloques de comunicación serial

Una forma de verificar que los *drivers* de comunicación serial NI-VISA están correctamente instalados en el PC, es conectando su dispositivo de hardware (ARDUINO por ejemplo) a uno de los puertos seriales USB del computador. Pero antes de conectarlo, cablee al bloque “*Visa configure serial port*” un control como se indica en la Figura 15. Nótese que en el control de la interfaz de usuario no se visualiza **ningún puerto COM**, lo cual indica efectivamente que el dispositivo no se encuentra conectado. Una vez que conecte el dispositivo al PC, podrá validar por cuál puerto **COM** se establecerá la conexión e intercambio de información. En la Figura 16 se muestra la conexión y reconocimiento del puerto COM5, esto comprueba que los NI-VISA están bien instalados.

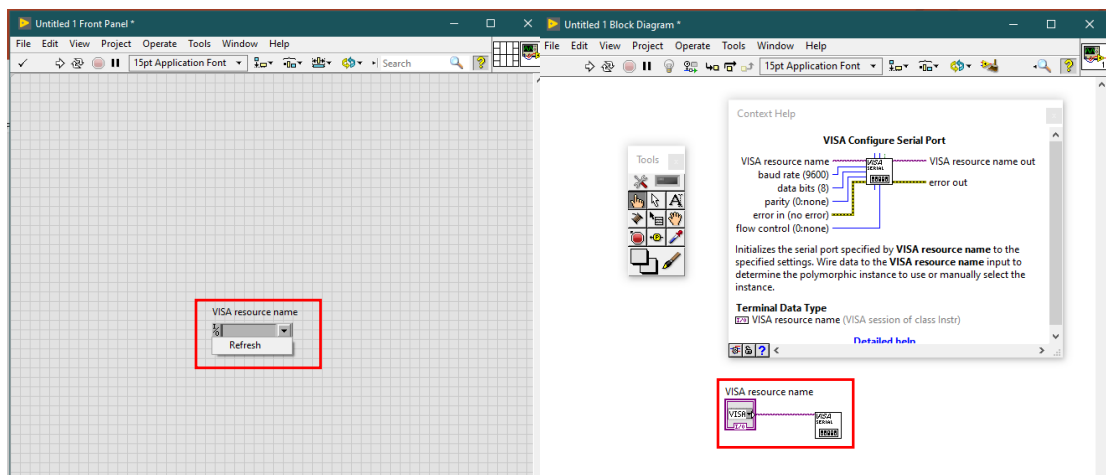


Figura 15. Conexión de un control para gestionar el número del puerto de comunicación desde el panel frontal

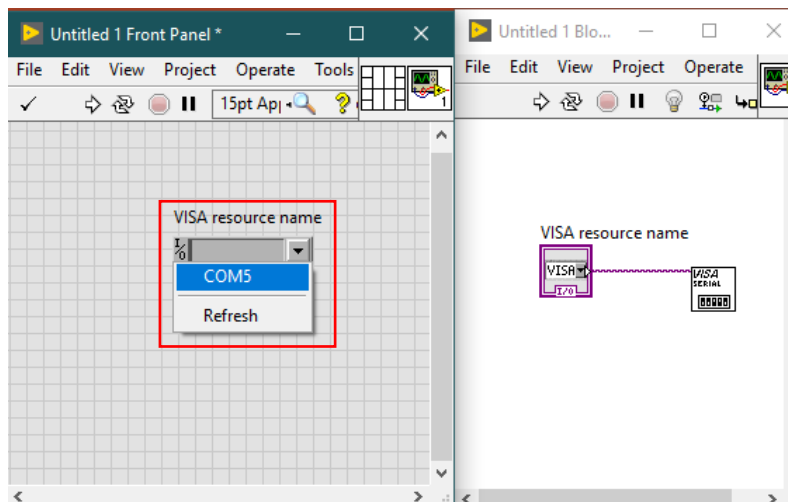


Figura 16. Reconocimiento del puerto COM5

A continuación, se presenta la ruta de acceso a los bloques que hacen parte de los NI-DAQmx:

Functions / Measurement IO/ NI DAQmx

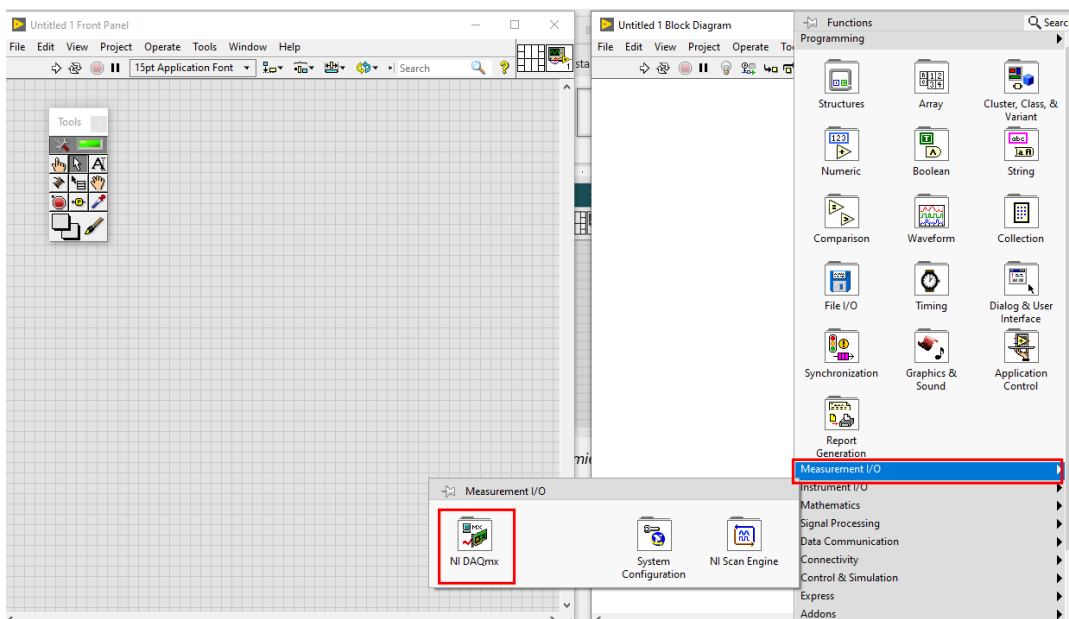


Figura 17. Acceso a los bloques de adquisición de datos NI-DAQmx

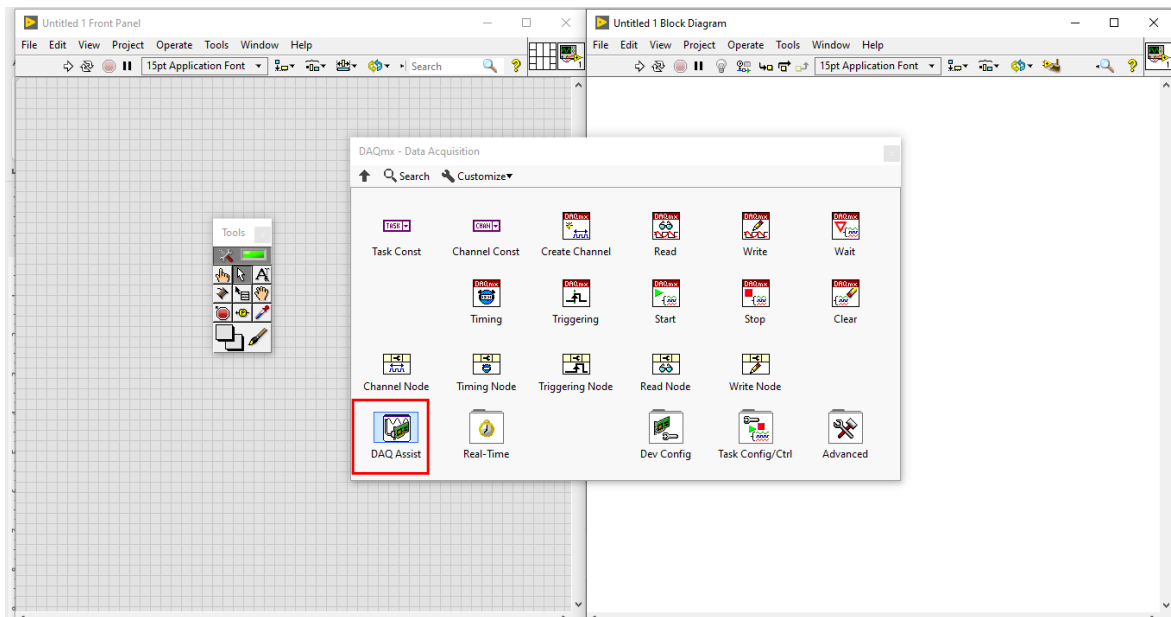


Figura 18. Bloques de adquisición de datos NI-DAQmx

Se trabajará especialmente con el bloque *DAQ assist*, el cual es un asistente de adquisición de datos.



Capítulo 2

Prácticas de Laboratorio

En este capítulo se presentan diferentes guías procedimentales para realizar prácticas de laboratorio relacionadas con los módulos académicos de aprendizaje de acondicionamiento de señales, sensores e instrumentación virtual. Cada guía contiene los objetivos que se desean lograr, materiales y/o equipos necesarios para realizar cada práctica, descripción del procedimiento detallado (incluyendo pre-informe y estudio previo) y los pasos a seguir durante la experimentación.

Práctica No.1

Sensado de temperatura con LM35, termistor y termopar tipo K

Objetivo:

Diseñar un sistema de sensado de temperatura usando diferentes tipos de transductores, donde cada variable física pueda ser acondicionada eléctricamente y visualizada en un entorno gráfico usando LabVIEW.

Materiales:

- PC con software LabVIEW Instalado
- Driver de comunicación **NI-VISA** instalado para el establecimiento de la comunicación serial.
- Driver de comunicación **DAQmx** instalado para el reconocimiento de las tarjetas de adquisición de datos.
- Librerías **LINX-Makerhub (Hobbyist)**, para la programación de las placas Arduino con LabVIEW
- Software Arduino IDE
- Placa **Arduino UNO o MEGA**-cable de comunicación USB
- Tarjeta **NI-USB-6009**
- Sensor LM35
- Termistor tipo NTC de $10K\Omega$
- Ventilador pequeño para computador de 5V
- Relé de 5V
- Diodo D1N4148
- Transistor NPN
- Sensor de temperatura Termopar tipo K
- Amplificador operacional LM358
- Circuito integrado AD595
- Candela de gas para calentar los sensores
- Resistencias, capacitores varios, cables y caimanes
- Board, Fuente DC

Estudio Previo

1. Consultar los *datasheets* del sensor de temperatura **LM35** y del circuito integrado **AD595**. Realice un cuadro o tabla donde resalte las características técnicas más relevantes de estos dispositivos como niveles de polarización, curva característica de salida, rangos de operación, etc.
2. Consulte qué es y cuál es el principio de funcionamiento de un **Termopar** (del Inglés Thermocouple). Especifique cuáles son los tipos de termopares más comunes en el mercado y mencione sus características principales.
3. Consultar qué es y cómo funciona un termistor, responder qué tipo de termistores existen en el mercado y explicar sus diferencias básicas. Adjuntar una imagen con los diferentes tipos de termistores consultados.
4. Adjunte la tabla o gráfica que relaciona la resistencia ($K\Omega$) *vs* Temperatura ($^{\circ}C$) del termistor que usará en la práctica.
5. Consultar el *datasheet* del amplificador operacional (A.O) **LM358** y diseñar un amplificador no inversor con una ganancia $G=V_{out}/V_{in}$ tal que al sensar una temperatura de $150^{\circ}C$ con el LM35, no se exceda 5V.
6. Si se tiene un termopar tipo k conectado a un CI. AD595, qué temperatura estaría sensando el termopar si en la salida del AD595 hubiesen 5V?
7. Describir gráficamente cuáles son los pines de la placa *Arduino UNO o MEGA* para cablear señales de entrada analógicas.
8. Consultar el datasheet de la tarjeta NI-USB-6009 y describir cuáles son sus características técnicas mas significativas. Mediante un diagrama, establecer cuáles son sus pines de entrada/salida analógicos y digitales (*pinout*).
9. Realice el montaje en *board* del (os) circuito(s) propuesto(s) en la práctica. Recuerde que el tiempo en el laboratorio es limitado y lo debe optimizar realizando los montajes en un horario extracurricular.
10. Para presentar su preinforme, use el formato (*template*) de la revista científica IEEE dispuesto en el siguiente enlace, con una extensión máxima de 3-4 páginas:
<https://www.ieee.org/conferences/publishing/templates> .

Experimento

Sensor de temperatura LM35

1. Monte el sensor LM35 en la *board* y polarícelo correctamente, para ello siga el manual del fabricante.

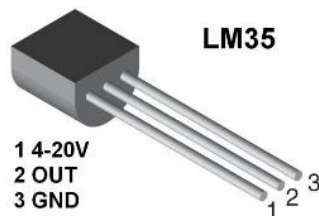


Figura 1.1. Sensor de Temperatura LM35

2. Conecte la señal de salida (*out*) del sensor, a la entrada (V_{in}) del amplificador operacional LM358. Lo que se pretende es realizar un amplificador no inversor de tal modo que cuando el sensor capte una señal de temperatura de 150°C , la salida del operacional no supere 5V max (esto debido a que se podrían quemar los puertos del microcontrolador en la placa Arduino o en la tarjeta de adquisición).

Nota: Para este diseño tenga en cuenta la característica de transferencia del sensor consultada en el preindorme y la topología circuital de un A.O no inversor.

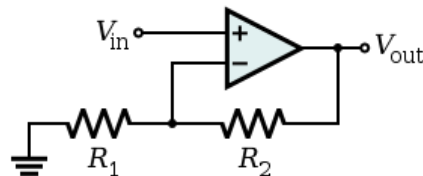


Figura 1.2. Amplificador no inversor

3. Después de haber diseñado el amplificador del numeral anterior, proceda a conectar la salida de éste (V_{out}), a una de las entradas analógicas elegidas y disponibles de la placa *Arduino UNO o MEGA*.

Nota: La figura 1.3 sólo es esquemática, nótese que en ese montaje falta el amplificador, sin embargo, ilustra una conexión directa del sensor con el sistema microcontrolado.

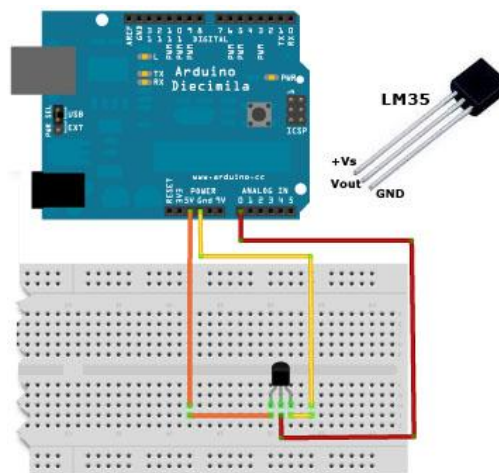


Figura 1.3. Conexión directa entre una placa Arduino y un sensor de temperatura LM35

4. Conecte la Placa *Arduino UNO o MEGA* al PC e identifique el puerto COM donde se ha iniciado la comunicación con el dispositivo (**lo puede verificar en el administrador de dispositivos de su PC**).
5. Diseñe una interfaz en LabVIEW que permita comunicarse con la placa *Arduino* y se pueda visualizar en ella la temperatura captada por el sensor LM35. Asegúrese de

elegir correctamente el puerto de comunicación (COM) y la velocidad de transmisión de datos (*Baud Rate*). Haga uso de las librerías **LINX (Hobbyist)** (ver sección previa) para programar la placa. Tenga en cuenta que el procedimiento de escalamiento de la señal de voltaje se debe realizar de acuerdo con la ganancia del amplificador encontrada en el pre-informe. Para representar gráficamente una señal de temperatura en lugar de una de voltaje, se debe realizar el procedimiento respectivo de escalamiento, es decir, la señal de voltaje deberá ser multiplicada por un factor o escalar para convertir ese nivel a valores de temperatura en °C. En la figura 1.4 se puede ver un ejemplo de la interfaz a desarrollar:

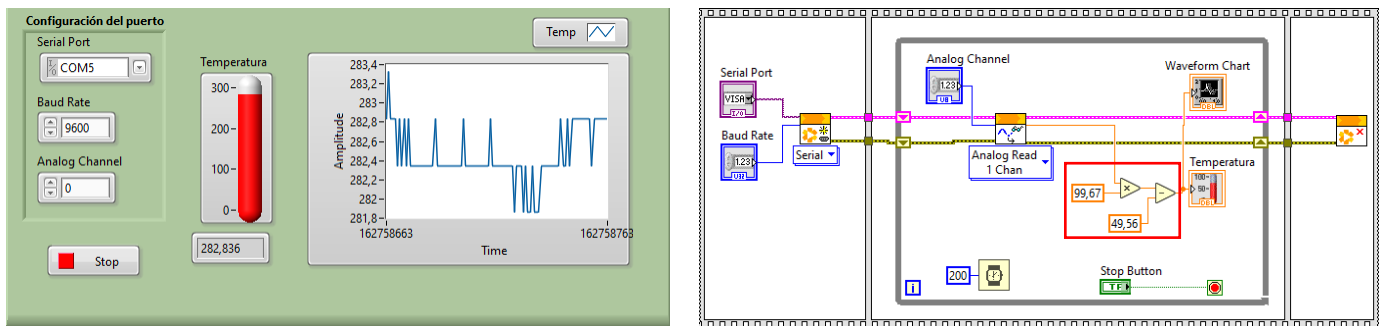


Figura 1.4. Interfaz de lectura de un canal analógico

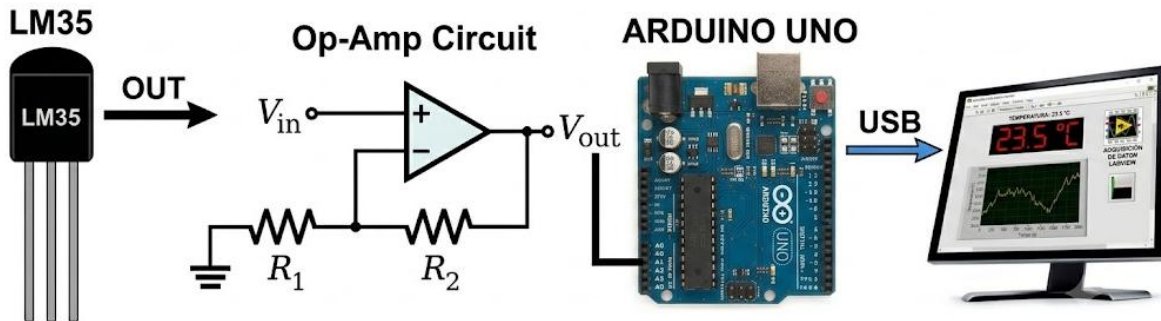


Figura 1.5. Diagrama esquemático del sistema de sensado de temperatura usando LM35, LabVIEW y Arduino

6. Otra forma de realizar el sensado de temperatura, es usando en lugar del Arduino®, una tarjeta de adquisición de datos (DAQ). En esta práctica se cuenta con la disponibilidad de la tarjeta **NI-USB-6009** (ver figura 1.6). Diseñe una interfaz en LabVIEW que permita comunicarse con la tarjeta **NI-USB-6009** y se pueda visualizar en ella la temperatura captada por el sensor LM35.
Para realizar el sensado de temperatura mediante la lectura de una señal analógica de voltaje usando la tarjeta, siga los siguientes pasos:



Figura 1.6. Tarjeta de adquisición de datos NI-USB-6009

- Una vez instalados los controladores DAQmx, conecte su tarjeta a uno de los puertos USB de su PC. Asegúrese que encienda el LED indicador.
- Use el bloque **DAQ Assistant**, el cual lo puede obtener de la siguiente ruta:

Programming/Express/Input/DAQAssist

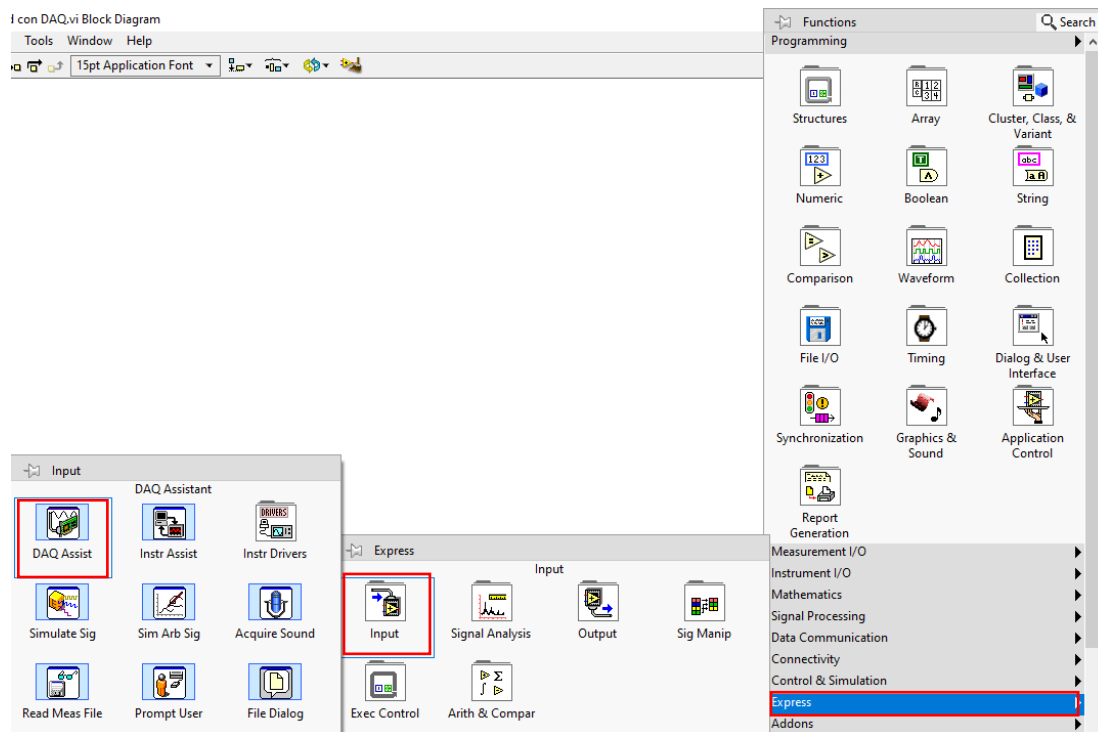


Figura 1.7. Ruta para acceder al bloque DAQ Assistant

- Haga doble *click* sobre el asistente de Adquisición de datos y configure la tarjeta así:
 - Adquirir señales/Entradas Analógicas/voltaje/Seleccione Dispositivo/Seleccione canal/Fin

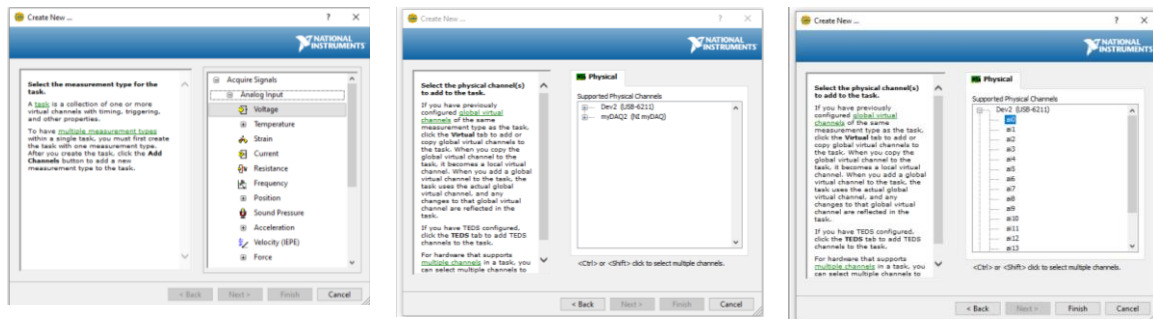


Figura 1.8. Selección de la tarjeta y del canal de adquisición de datos

- Seleccione el modo de Adquisición de datos: *1 sample (On Demand)* o *Continuous samples*

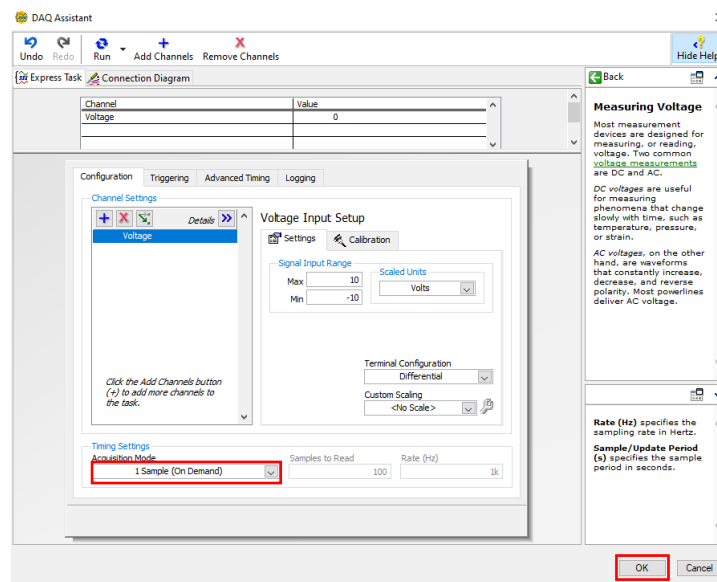


Figura 1.9. Selección del modo de adquisición de datos

- Programe en LabVIEW el código mostrado en la figura 1.10.

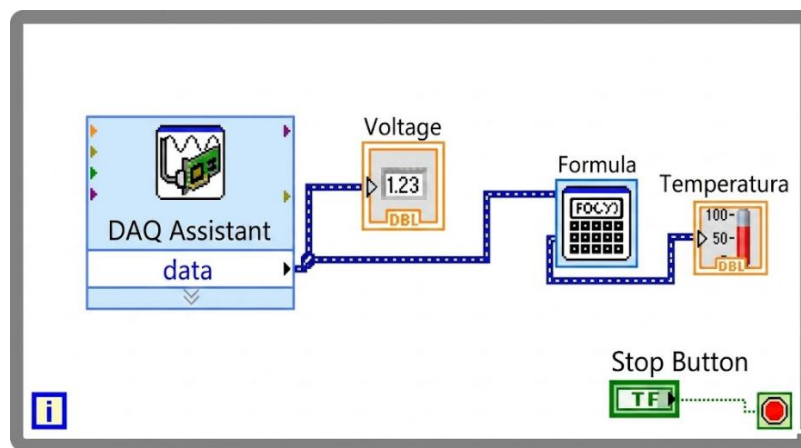


Figura 1.10. Programa para sensar una señal analógica de voltaje con la NI-DAQ

Sensor de temperatura Termopar tipo K

- Realice el montaje sugerido en la figura 1.11, para ello conecte el termopar al CI. AD595 y polarice éste últimamente adecuadamente siguiendo el manual del fabricante.

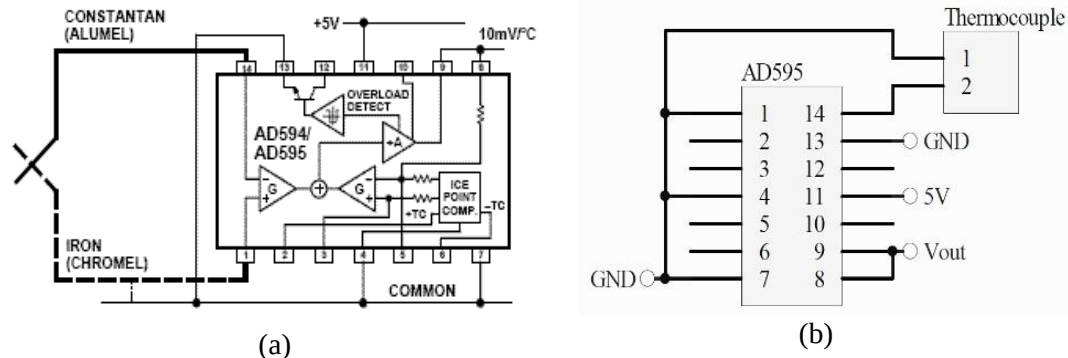


Figura 1.11. (a)Acondicionamiento para Termopar usando CI. AD595. (b)Diagrama de conexión simplificada

- Conecte físicamente la salida de voltaje del CI. AD595 a otra entrada analógica disponible de la tarjeta de adquisición de datos NI-USB-6009 o de la placa Arduino. En la interfaz desarrollada previamente en LabVIEW, configure y adicione un canal más de lectura de voltaje (ver figura 1.8), la idea es poder visualizar en el mismo entorno gráfico, las dos señales de temperatura (LM35 y Termopar).

En la Figura 1.12, puede verse un ejemplo de la adquisición de dos señales analógicas de voltaje con Arduino y las librerías LINX (*Hobbyist*). Nótese que ambas señales se representan en voltaje y no poseen ningún tipo de escalamiento para convertirlas a temperatura.

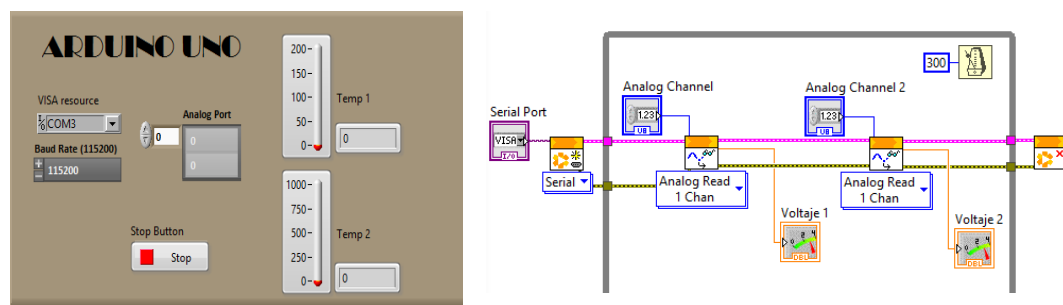


Figura 1.12. Interfaz de lectura de dos canales analógicos con Arduino

En la Figura 1.13, se muestra el código de la adquisición de dos señales analógicas de voltaje con la tarjeta NI-USB6009.

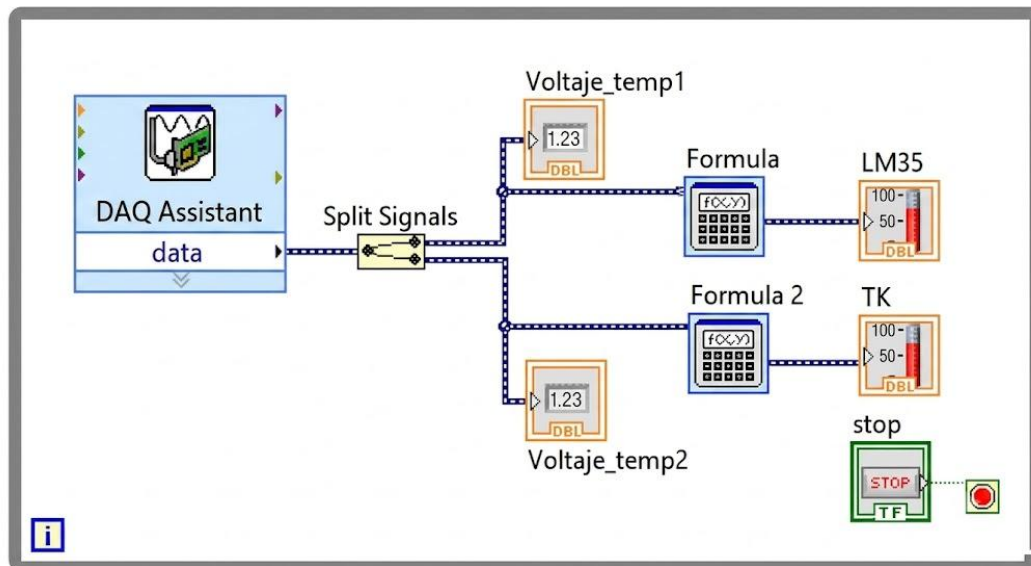


Figura 1.13. lectura de dos canales analógicos con la tarjeta de adquisición de datos

La figura 1.14 muestra el diagrama esquemático del sistema de sensado de temperatura usando Termopar tipo K, el sensor LM35, el amplificador AD595, LabVIEW y la tarjeta de adquisición de datos NI-USB6009.

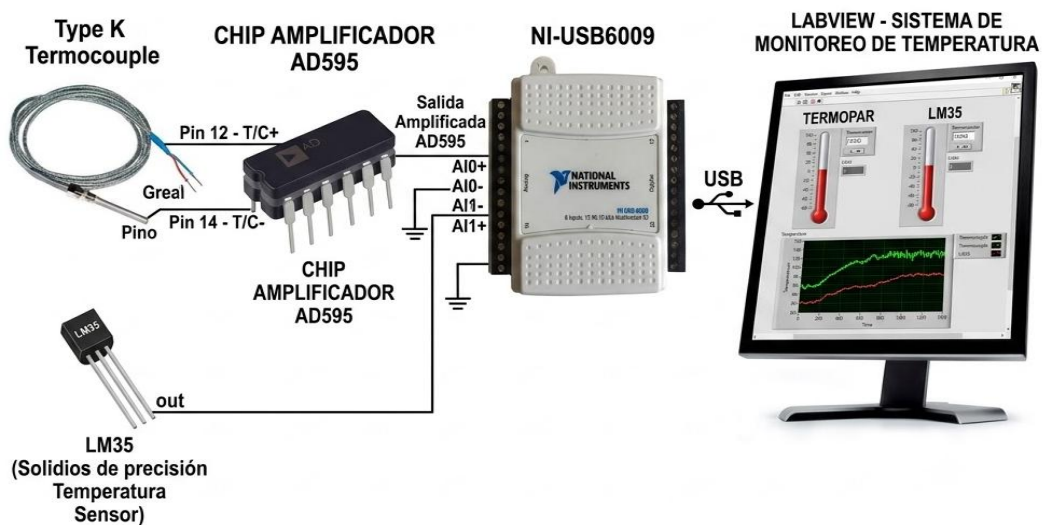


Figura 1.14. Diagrama esquemático del sistema de sensado de temperatura usando la tarjeta de adquisición de datos

Termistor NTC

9. La Figura 1.15 muestra de izquierda a derecha, un termistor NTC, un circuito divisor de voltaje formado por una resistencia fija y un termistor (resistencia variable) y el diagrama circuital esquemático del montaje en *board*. Realice las conexiones indicadas del circuito mostrado en la Figura 1.15c y polarícelo según lo indicado. Note que el cable color negro es la tierra (GND), el amarillo es la línea de salida del divisor de tensión y el rojo es la polarización positiva de 5V.

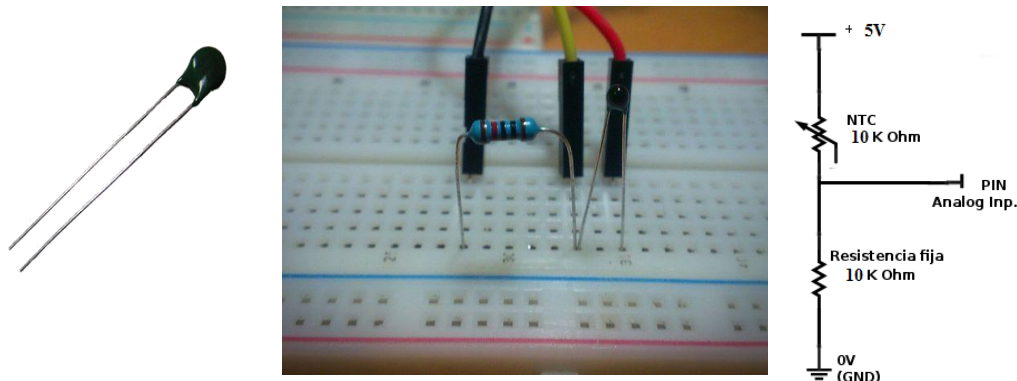
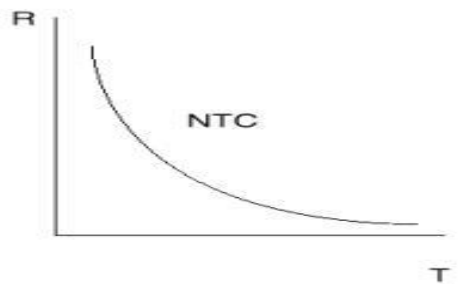


Figura 1.15. a) Termistor NTC. b) Circuito en board. c) diagrama esquemático

10. Paralelamente al montaje del numeral 9, realice el montaje de un sensor de temperatura LM35 de modo que éste quede ubicado lo más cerca posible del termistor y polarícelo correctamente; para ello siga el manual del fabricante.
11. Con la ayuda de Excel y del sensor LM35 (valor de referencia), construya una gráfica experimental de Voltaje (V) *vs* Temperatura (°C) basada en las mediciones que se realicen en el laboratorio. Tenga en cuenta que los dos sensores (LM35 y termistor NTC) deben estar muy cerca uno del otro para que la temperatura de ambos al calentarlos simultáneamente sea prácticamente la misma. Llene la Tabla 1.1.

Tabla 1.1. Característica del termistor NTC

Medición	Temperatura [°C]	Voltaje [V]
1		
2		
3		
4		
5		
6		



Nota: Se sugiere que mientras calienta los sensores, tome un video con su celular registrando tanto el voltaje generado como la temperatura sensada. Haga lo mismo para la curva de enfriamiento. El video servirá de insumo para crear la tabla de datos que posteriormente usará en la caracterización del termistor.

12. Con base en el diagrama circuital mostrado en la Figura 1. y en la figura 1.16, conecte el pin de salida del divisor de voltaje en uno de los pines de entrada analógicos de la placa Arduino o de la tarjeta NI-USB6009.

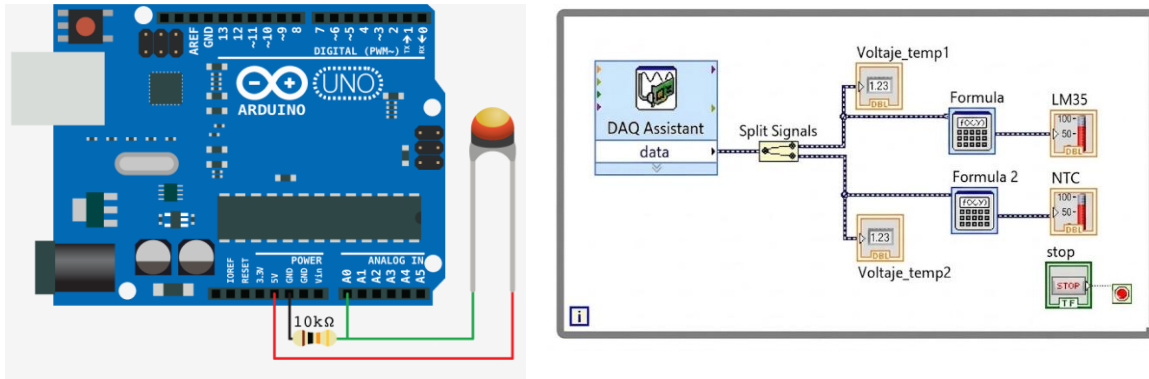


Figura 1.6. a) Conexión del Termistor NTC a la placa Arduino. b) Diagrama de bloques del código de adquisición para la tarjeta NI-USB-6009

13. Diseñe una interfaz en LabVIEW que permita comunicarse con la placa *Arduino* o la tarjeta de adquisición de datos *NI-USB-6009* y se pueda visualizar en ella la temperatura captada por el Termistor NTC. Asegúrese de elegir correctamente el puerto de comunicación (COM) y la velocidad de transmisión de datos (*Baud Rate*). En este punto, se debe hacer uso de los datos tomados en la Tabla 1.1 para caracterizar el termistor. Resulta necesario saber qué voltaje representa una determinada temperatura, para lo cual podría usar una extrapolación de los datos creando una ecuación o línea de tendencia. Para ello se puede ayudar de la gráfica de dispersión realizada en Excel, haciendo *click* derecho sobre ella, se debe seleccionar la opción “*Agregar línea de tendencia*” y luego “*Presentar ecuación en el gráfico*”, asegurándose de escoger la línea que mejor se ajuste a los datos (lineal, polinómica, logarítmica, etc).



Figura 1.17. Ejemplo de Excel para agregarle la línea de tendencia a una gráfica.

14. Aplicación: Circuito de control de temperatura On/Off usando Termistor NTC

Monte el circuito de la figura 1.18, polarícelo según lo indicado y explique el funcionamiento del mismo. Realice una interfaz en LabVIEW para monitorear/ controlar el estado del sistema.

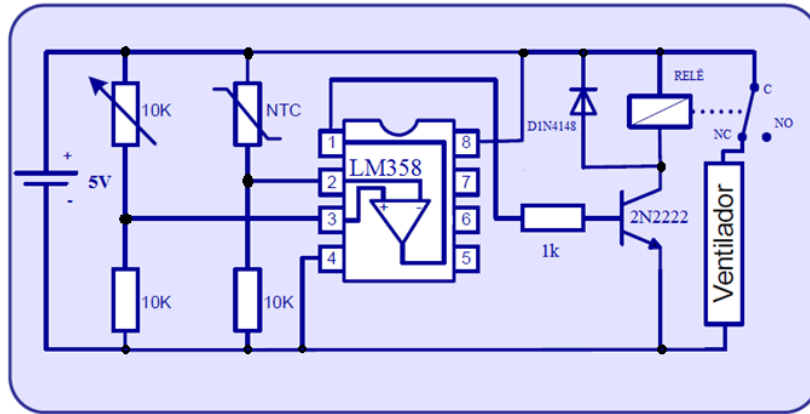


Figura 1.18. Circuito de un Control de temperatura on/off básico usando termistor NTC

Conclusiones

- Escriba las conclusiones más relevantes de la práctica de laboratorio, indicando hallazgos, dificultades y logros.

Práctica No.2

Sensado de niveles de presión usando sensor piezoeléctrico

Objetivo:

Implementar una aplicación que detecte cuándo se ha ejercido un nivel de presión sobre un elemento piezoeléctrico, dicho efecto debe ser visualizado en un entorno gráfico desarrollado en LabVIEW.

Materiales:

- PC con software LabVIEW Instalado
- Driver de comunicación **NI-VISA** instalado.
- Driver de comunicación **DAQmx** instalado.
- Librerías **LINX-Makerhub (Hobbyist)**, para la programación de las placas Arduino con LabVIEW
- Software Arduino IDE
- Placa **Arduino UNO o MEGA**-cable de comunicación USB
- Tarjeta **NI-USB-6009**
- 2 sensores de vibración piezoeléctricos SEN-10293
- Buzzer de 5V
- Board, Fuente DC
- Resistencias, potenciómetros, capacitores varios, cables y caimanes

Estudio previo

1. Consultar el principio de funcionamiento de un sensor piezoeléctrico.
2. Adjuntar algunas imágenes de este tipo de sensores y construir una tabla con las principales diferencias técnicas entre ellos.
3. Consultar y listar las principales aplicaciones donde se implementan los sensores de presión piezoeléctricos.
4. Consultar el *datasheet* del sensor de vibración piezoeléctrico SEN-10293 (TBB-20-6L0), sus principales usos y aplicaciones.
5. Realice el montaje en *board* del (os) circuito(s) propuesto(s) y programe los códigos necesarios en software antes de la práctica. Recuerde que el tiempo en el laboratorio es limitado y lo debe optimizar realizando los montajes y programas en un horario extracurricular.
6. Para presentar su preinforme, use el formato (*template*) de la revista científica IEEE dispuesto en el siguiente enlace, con una extensión máxima de 3-4 páginas:
<https://www.ieee.org/conferences/publishing/templates> .

Experimento

Antes de comenzar su laboratorio, asegúrese de tener instalados todos los programas y driver's de comunicación necesarios.

- LabVIEW
- Librerías LINX-Makerhub (Hobbyist)
- NI-VISA drivers (Comunicación Serial)
- DAQmx drivers.
- Arduino IDE

Sensor de vibración piezoeléctrico

1. El sensor de la Figura 1.a, es un sensor piezoeléctrico usado para detectar vibraciones o golpes, en términos generales, este sensor puede captar los cambios en los niveles de presión que se ejercen sobre él, generando una salida de voltaje proporcional a la excitación generada. Para realizar su conexión, tenga en cuenta el diagrama de la Figura 2.1a y 2.1b.

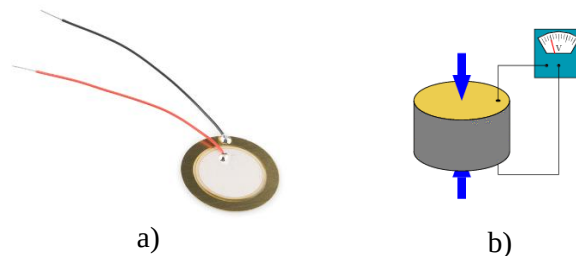


Figura 2.7. a) Sensor piezoeléctrico SEN-10293. b) Esquemático del funcionamiento del sensor

Conecte el cable negro del sensor a tierra (gnd) y el cable rojo al pin de entrada analógica de voltaje A0 del Arduino. Adicionalmente, conecte un resistor de $1M\Omega$ en paralelo al sensor piezoeléctrico con el fin de asegurar un nivel de voltaje bajo a la entrada del Arduino cuando el sensor no sea presionado.

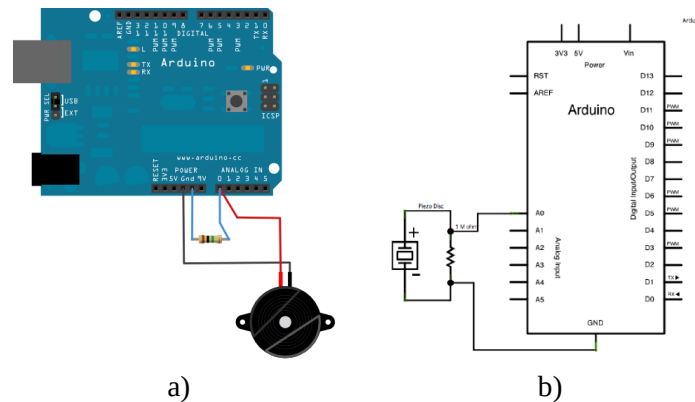


Figura 2.8. Conexiones del sensor piezoeléctrico

- Con base en el diagrama circuital mostrado en la Figura 2.2b, conecte un diodo LED y un buzzer en los pines de salida digitales de la placa Arduino UNO o MEGA como se indica en la Figura 2.3.

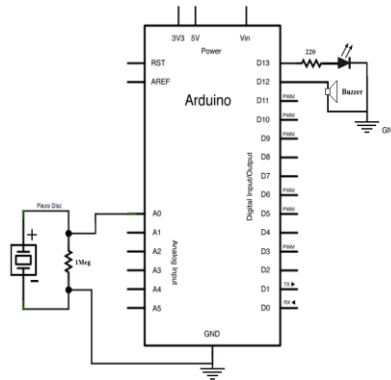


Figura 2.9. Conexión del sensor piezoeléctrico, LED y buzzer al Arduino

- Conecte la Placa *Arduino UNO o MEGA* al PC e identifique el puerto COM donde se ha iniciado la comunicación con el dispositivo.
- Diseñe una interfaz en LabVIEW que permita comunicarse con la placa *Arduino UNO o MEGA* y se pueda visualizar en ella un LED virtual que deberá encenderse y apagarse sincronizadamente con el diodo LED implementado físicamente cada vez que el usuario ejerza un nivel de presión sobre el sensor piezoeléctrico. El buzzer debe sonar también cada vez que lo anterior suceda. Asegúrese de elegir correctamente el puerto de comunicación y la velocidad de transmisión de datos (*Baud Rate*).

Para encender el LED virtual en LabVIEW es necesario medir experimentalmente los niveles de voltaje cada vez que se presiona el sensor, estableciendo umbrales de voltaje para la toma de decisiones. Para leer el voltaje de entrada por el pin A0, use el bloque “*Analog Read*” de las librerías LINX mostrado en la Tabla 2.1, el cual se encuentra ubicado en la siguiente ruta de acceso:

***Functions/MakerHub/LINX/Peripherals/Analog/Read. ó
Functions/Hobbyist/Peripherals/Analog/Read.***

Tabla 2.1. Estado de los LEDS de la aplicación y bloque de lectura de voltaje

LED Virtual encendido	LED Virtual apagado	LMH-LINX.lvlib:Analog Read.vi Read the value of the specified analog input channel(s).
 Voltaje medido 1= ?	 Voltaje medido 2= umbral inferior	

Si el voltaje de entrada (V_{in}) en el pin A0 es mayor o igual al ***voltaje medido 2***, entonces se debe encender el LED, si no, se apagará.

Para encender el LED físico y el Buzzer conectados a la placa Arduino, es necesario usar los pines respectivos del Arduino como **salidas digitales**. Para esto use el bloque de la librería **LINX** que se encuentra en la ruta

Functions/MakerHub/LINX/Peripherals/Digital/Write

ó

Functions/Hobbyist/Peripherals/Digital/Write y que es mostrado a continuación:

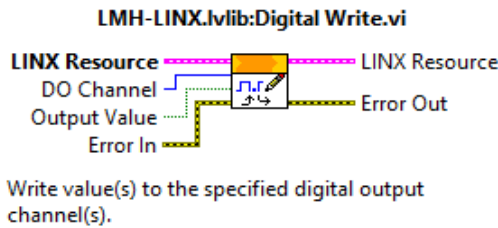


Figura 2.10. Bloque de la librería LINX para la escritura de salidas digitales

Para una mejor comprensión de lo que se requiere a nivel de hardware, refiérase al siguiente enlace donde podrá ver un **video** del funcionamiento del circuito:

https://www.youtube.com/watch?v=e5zuGxIGj_A

En la Figura 2.5, se puede ver un ejemplo de una interfaz diseñada en LabVIEW y su respectivo código para realizar la operación descrita anteriormente.

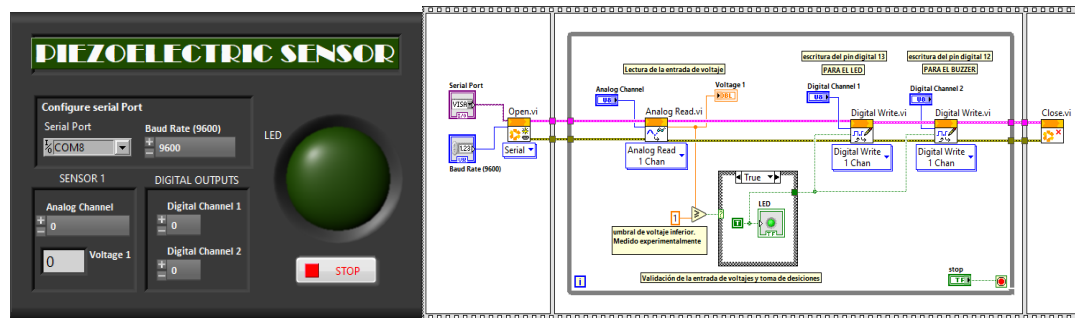


Figura 2.11. Ejemplo de LabVIEW para leer un pin analógico de voltaje y escritura de pines configurados como salidas digitales usando Arduino y las librerías Hobbyist

5. Diseñe una interfaz en LabVIEW que permita comunicarse con la tarjeta de adquisición de datos NI-USB-6009. La tarjeta debe ser configurada para leer un canal analógico de voltaje y activar/desactivar dos (2) líneas de salida digitales. El canal analógico se usará para leer el voltaje generado por el sensor piezoeléctrico, las salidas digitales se usarán para conectar el *LED* y el *buzzer* de la aplicación. La figura 2.6 muestra la configuración de la DAQ.

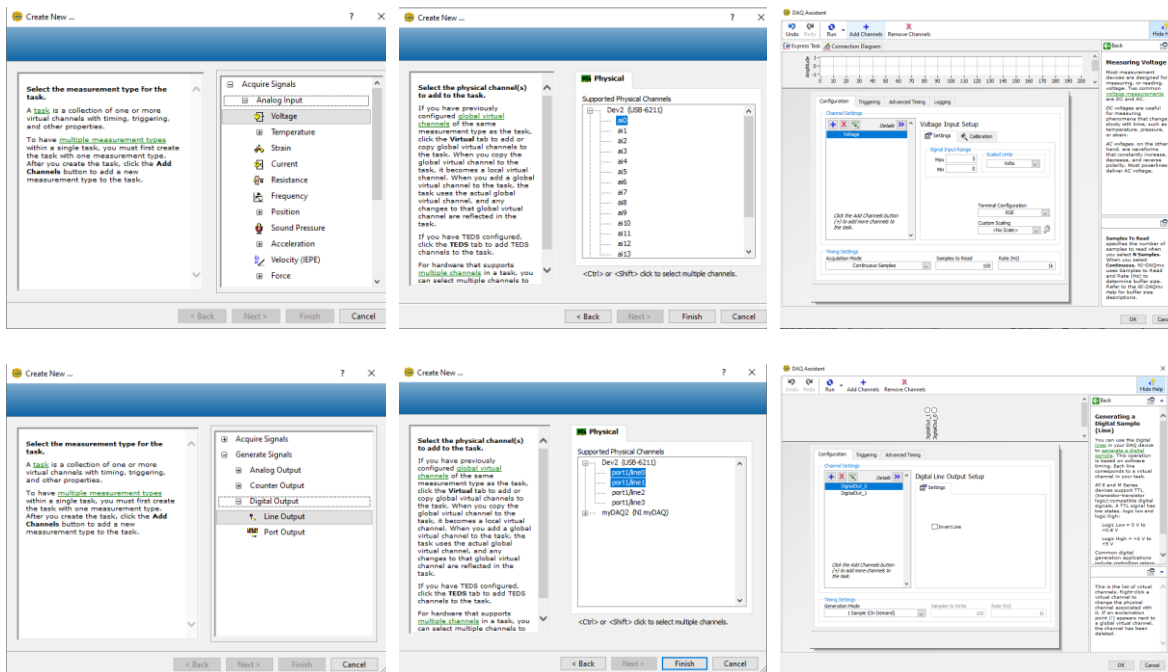


Figura 2.6. Configuración para leer el canal a0 como entrada analógica de voltaje y como salidas digitales de las líneas 0 y 1.

En la figura 2.7 se observa un ejemplo del código implementado con la DAQ para realizar la aplicación de lectura de un canal analógico de voltaje y la escritura de dos líneas digitales.

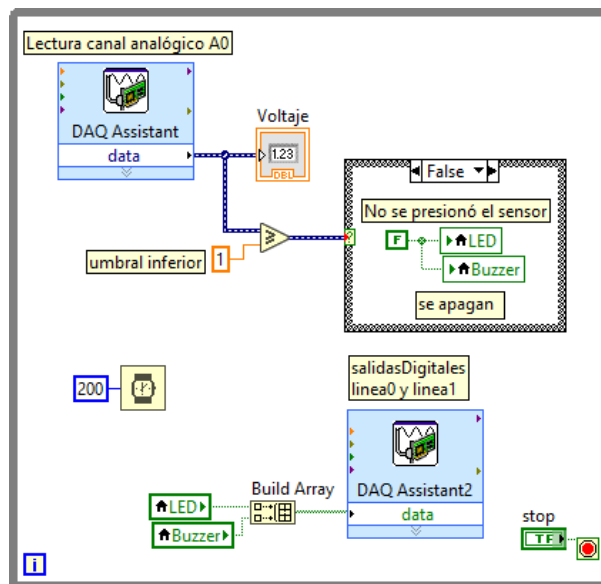


Figura 2.7. Ejemplo de LabVIEW para leer un pin analógico de voltaje y escritura de dos pines configurados como salidas digitales usando la DAQ

6. Aplicaciones

- a. **Reloj digital configurable con sensor piezoeléctrico:** Monte el circuito de la figura 2.8, adicionando un nuevo sensor piezoeléctrico conectado al pin A1 del Arduino (o de la tarjeta de adquisición de datos NI-USB-6009). Tenga en cuenta el diagrama de bloques de la figura 2.5 (o 2.7) y adicione un nuevo bloque **Analog Read** a su código para leer el nuevo sensor.

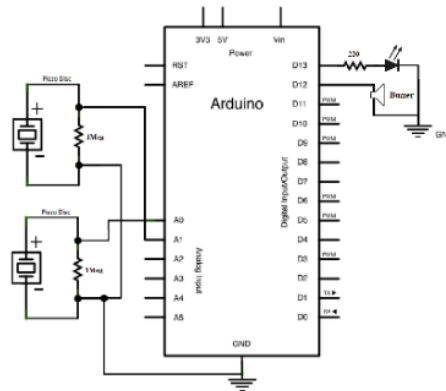


Figura 2.8. Conexión de los dos piezoeléctricos a la tarjeta Arduino

Se debe desarrollar una aplicación en software que permita interactuar con los sensores piezoeléctricos instalados, de modo tal que cada vez que el primer sensor (**config**) se presione, aumente la cuenta uno a uno de un contador (1) cíclico que va desde 1 hasta 12 (Horas), es decir el conteo que realiza el contador cada vez que se presiona el sensor es el siguiente:

1-2-3-4-5-6-7-8-9-10-11-12-1-2-3-4-5-6-7-8-9-10-11-12-1....

Cuando el segundo sensor (**enter**) sea presionado, el contador (1) se detiene y da paso para que otro contador (2) sea configurado al presionar nuevamente el primer sensor (**config**). El nuevo contador (2), incrementará su cuenta uno a uno desde 0 hasta 59 (minutos) así:

0-1-2-3-4-5.....10-11.....30...40.....57-58-59-0-1-2-3-4-5.....25.....40.....50.....57-58-59-0-1-2-3....

Al terminar de configurar el segundo contador, se presiona nuevamente el segundo sensor (**enter**) confirmando los valores de los contadores.

Al recibir un tercer toque en el segundo sensor se tendrá configurada la hora deseada, a partir de ese instante el reloj debe comenzar a correr y a dar la hora cada minuto que pase. Para esta temporización, se puede usar el bloque de LabVIEW llamado **"wait(ms)"** mostrado en la figura 2.9.

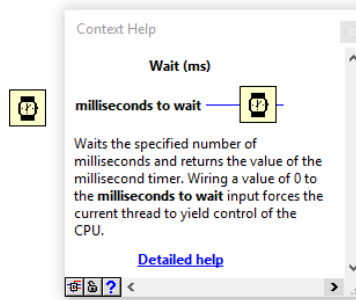


Figura 2.9. Bloque de temporización de LabVIEW Wait (ms)

Se aclara que partiendo de la Figura 2.5, se debe incluir en el código la lectura del segundo sensor a través del pin A1, además se debe implementar la lógica de los incrementos de los contadores. En la Figura 2.10, se muestra un ejemplo de la interfaz de usuario a desarrollar.



Figura 2.10. Aplicación final desarrollada en LabVIEW

- b. Desarrolle una aplicación **libre** donde haga uso de los dos sensores piezoeléctricos, de tal forma que partiendo de la propiedad que tienen estos sensores para generar un voltaje proporcional a la presión efectuada sobre ellos, se puedan realizar diferentes acciones con un mismo sensor. Por ejemplo, si se presiona levemente el sensor 1, éste generará un voltaje 1, lo cual implicará realizar la acción 1. Si se presiona más fuertemente el mismo sensor 1, se generará un voltaje 2, lo cual implicará realizar una acción 2. Como ejemplos prácticos para esta situación, está la incorporación de este tipo de sensores en las pantallas táctiles de los celulares modernos para aplicaciones de video juegos, donde el nivel de presión ejercida sobre la pantalla, efectuará una acción diferente y proporcional a dicha presión en el juego, aplicaciones musicales, juegos de fuerza en ferias, etc.

7. Conclusiones

Escriba las conclusiones más relevantes de la práctica de laboratorio, indicando hallazgos, dificultades y logros.

Práctica No.3

Sensado de peso con celdas de carga y transmisión serial

Objetivo:

Desarrollar una aplicación de pesaje utilizando una celda de carga y un amplificador HX711 monitoreando la medida a través del entorno gráfico de LabVIEW. Se busca que los estudiantes comprendan los principios de operación y acondicionamiento de señales de una celda de carga, así como la interfaz y amplificación de su salida mediante el uso del amplificador HX711.

Materiales:

- PC con software LabVIEW Instalado
- Driver de comunicación **NI-VISA** instalado.
- Software Arduino IDE
- Placa **Arduino UNO o MEGA**-cable de comunicación USB
- 1 celda de carga (5kg, 3kg, 2kg, 1kg)
- Módulo *driver* para celda de carga HX711
- Piezas impresas en 3D para crear la balanza electrónica (ver anexos)
- Tornillos de fijación de las piezas 3D a la celda de carga.

Estudio previo

1. Consultar qué es una celda de carga y cuál es su principio de funcionamiento.
2. Adjuntar algunas imágenes de este tipo de sensor y construir una tabla con las principales diferencias técnicas entre ellos.
3. Consultar y listar las principales aplicaciones donde se implementan las celdas de carga.
4. ¿Qué tipo de circuitos son usados para acondicionar eléctricamente la señal de una celda de carga?
5. Consultar el *datasheet* del módulo HX711 e identificar sus pines de conexión. Concluir qué tipo de chip amplificador es este circuito integrado.
6. Realice el montaje y la programación de la aplicación propuesta con los códigos suministrados para el Arduino (ver anexos). También implemente el aplicativo de software en LabVIEW para establecer comunicación serial con la balanza desarrollada. Recuerde que el tiempo en el laboratorio es limitado y lo debe optimizar realizando los montajes y programas en un horario extracurricular.
7. Para presentar su preinforme, use el formato (*template*) de la revista científica IEEE dispuesto en el siguiente enlace, con una extensión máxima de 3-4 páginas:
<https://www.ieee.org/conferences/publishing/templates>.

Experimento

Antes de comenzar su laboratorio, asegúrese de tener instalados todos los programas y driver's de comunicación necesarios.

- LabVIEW
- NI-VISA drivers (Comunicación Serial)
- Arduino IDE

Sensor celda de carga

1. El sensor de la Figura 1., es una celda de carga que soporta pesos de hasta 5kg, usada para convertir una fuerza o peso en una señal eléctrica, esta señal es proporcional a la magnitud de la fuerza aplicada, generando una salida de voltaje dependiente de la excitación. Las celdas de carga se basan en diferentes principios de operación, como la deformación por esfuerzo, la presión hidráulica o neumática, y las propiedades piezoeléctricas.

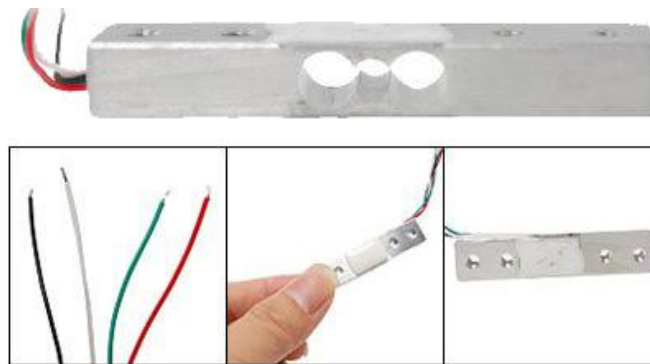


Figura 3.12. Celda de carga

2. El módulo **HX711** es un convertidor analógico a digital (ADC) de 24 bits de alta precisión, diseñado específicamente para aplicaciones de pesaje y medición de fuerza en sistemas que utilizan celdas de carga. El HX711 es popular en proyectos de electrónica debido a su facilidad de uso, bajo costo y capacidad para proporcionar una medición precisa y estable de la señal de salida de las celdas de carga. Incluye un **amplificador de instrumentación** con una ganancia seleccionable de 32, 64 o 128, lo que permite amplificar las débiles señales de salida típicas de las celdas de carga sin necesidad de un amplificador externo. Utiliza una interfaz digital de dos hilos (*clock* y *data*) para comunicarse con microcontroladores, como Arduino, simplificando la conexión y el diseño del sistema. La Figura 3.2 muestra una imagen del módulo amplificador HX711.

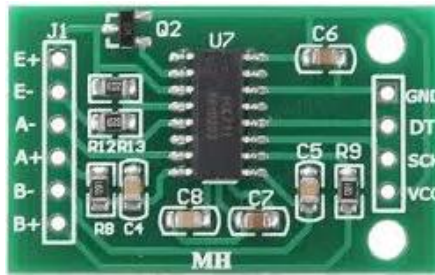


Figura 3.13. Módulo HX711

- Para realizar la conexión de la celda de carga al amplificador HX711 y a la placa Arduino, tenga en cuenta el diagrama de la Figura 3.3 y la Tabla 3.1.

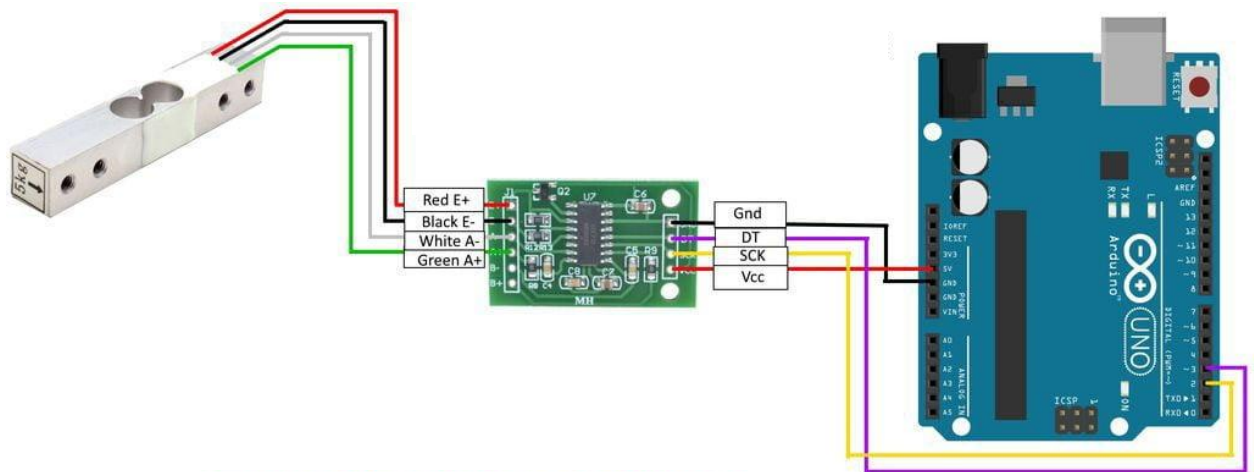


Figura 3.3. Diagrama de conexiones

Tabla 3.1. Asignación de pines

Calda de carga	Módulo HX711 pin		Arduino pin
Rojo	E+	GND	GND
Negro	E-	DT	3
Blanco	A-	SCK	2
Verde	A+	VCC	5v

- Imprima e instale las piezas 3D a la celda de carga, estas servirán de base para construir la balanza electrónica. Los archivos necesarios para imprimir las piezas, las puede encontrar en el siguiente enlace: <https://www.thingiverse.com/thing:3129439>

Nota: podrá solicitarle las piezas impresas al docente del curso

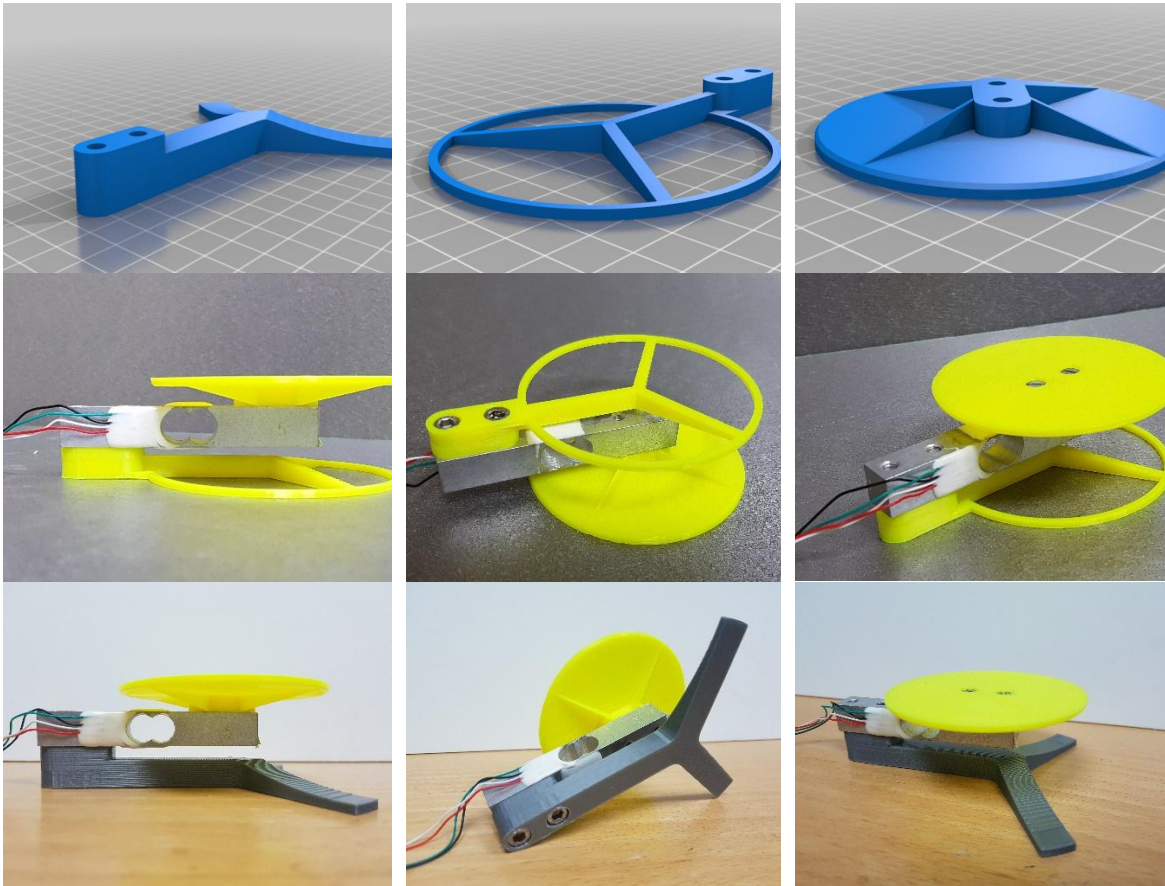


Figura 3.4. Ensamble de las piezas 3D a la celda de carga

5. Después de realizar las conexiones eléctricas y el ensamble mecánico, proceda a programar el Arduino así:
 - a. De acuerdo con el Anexo 3.1 (*Código para calibrar la celda de carga*) programe el Arduino y tenga a la mano un objeto al que le conozca su peso. Puede usar una balanza electrónica externa para medir el objeto. Al ejecutar este programa, deberá obtener el **factor de calibración** de la celda de carga, el cual se calcula **dividiendo** el valor reportado en el monitor serial del Arduino **entre el peso del objeto conocido**. Para más información, refiérase al video:

<https://www.youtube.com/watch?v=Z01gjTLE5eE&t=165s>



Figura 3.5. Peso patrón medido en balanza externa

La ruta para acceder a los bloques de la Figura 3.6 se puede observar en la Figura 3.7.

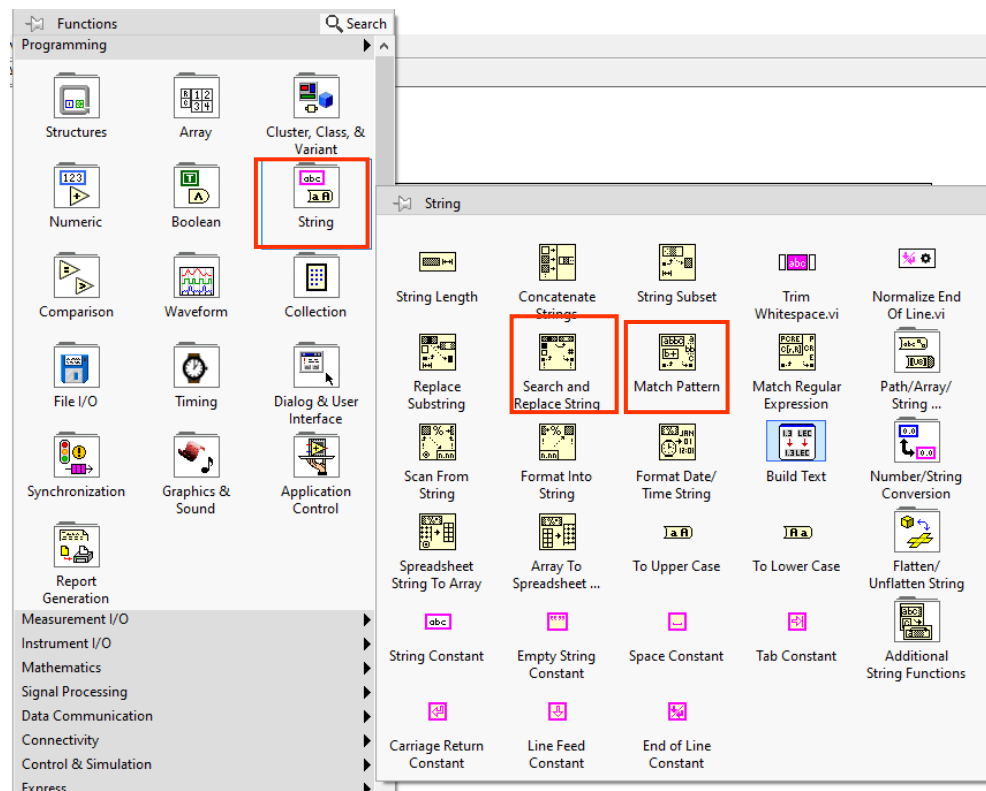


Figura 3.7. Ruta de acceso a los bloques Match Pattern y Search and Replace

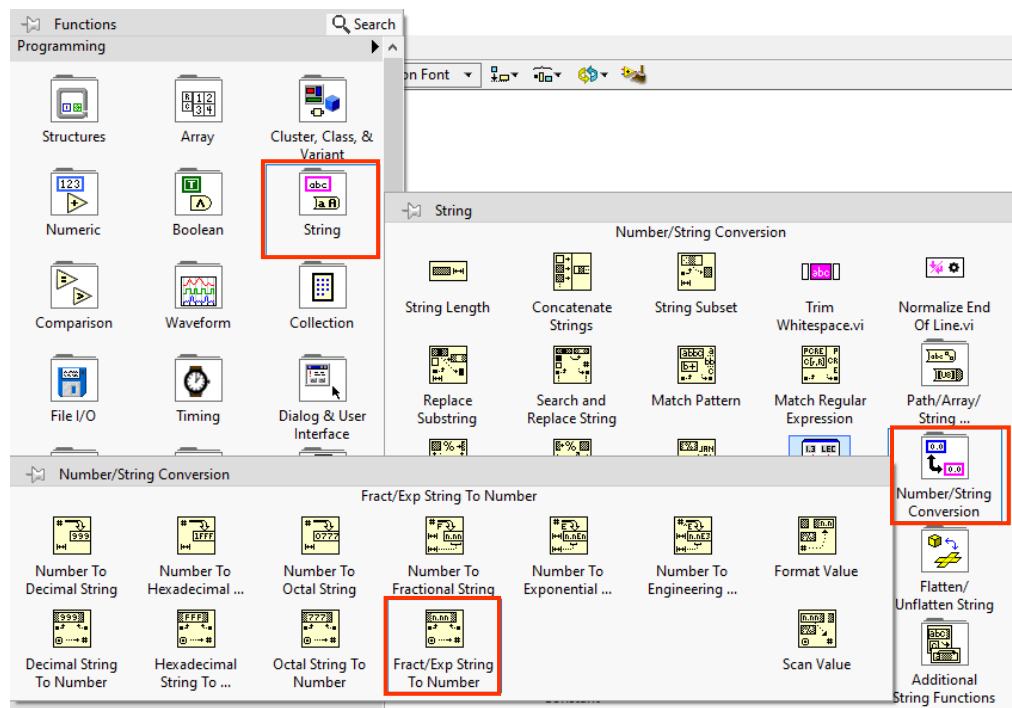


Figura 3.8. Ruta de acceso al bloque Fract/Exp String to Number

Conclusiones

- Escriba las conclusiones más relevantes de la práctica de laboratorio, indicando hallazgos, dificultades y logros.

ANEXOS

3.1. Código para calibrar la celda de carga

```
/*
Programa para obtener el factor de escala en base a un peso conocido. Con el valor
obtenido se divide por el peso colocado (en gramos) y aplicar en el segundo programa.
Requiere libreria: HX711 Arduino Library de Bogdan
*/
#include "HX711.h"          // incluye librería HX711
#define DT 3                // DT de HX711 a pin digital 3
#define SCK 2               // SCK de HX711 a pin digital 2
HX711 celda;               // crea objeto con nombre celda

void setup() {
  Serial.begin(9600);        // inicializa monitor serie a 9600 baudios
  celda.begin(DT, SCK);      // inicializa objeto con los pines a utilizar
  Serial.println("Para obtener factor de escala:"); // texto estático descriptivo
  celda.set_scale();         // establece en factor de escala por defecto
  celda.tare();              // realiza la tara o puesta a cero
  Serial.println("Colocar un peso conocido (10 seg.)"); // texto estático descriptivo
  delay(10000);              // demora de 10 segundos para colocar el peso conocido
  Serial.println(celda.get_units(10)); // obtiene valor (promedio de 10 lecturas) para calcular factor de escala
  Serial.println("Hecho. Dividir el valor mostrado por el peso colocado y obtener #####"); // texto descriptivo
}

void loop() {
}
}
```

3.2. Código para medir con la balanza

```
/*
Implementacion de una balanza con el factor de escala obtenido del primer programa.
Requiere libreria: HX711 Arduino Library de Bogdan
*/
#include "HX711.h"          // incluye librería HX711
#define DT 3                // DT de HX711 a pin digital 3
#define SCK 2               // SCK de HX711 a pin digital 2
HX711 celda;               // crea objeto con nombre celda

void setup() {
  Serial.begin(9600);        // inicializa monitor serie a 9600 baudios
  Serial.println("Balanza con celda de carga"); // texto descriptivo
  celda.begin(DT, SCK);      // inicializa objeto con los pines a utilizar
  celda.set_scale(#####.f); // ##### es el factor de escala obtenido del primer programa
  celda.tare();              // realiza la tara o puesta a cero
}

void loop() {
  Serial.print("Peso:"); // texto descriptivo
  Serial.println(celda.get_units(10), 1); // muestra el valor obtenido promedio de 10 lecturas con 1 cifra decimal
  celda.power_down();       // apaga el módulo HX711
  delay(3000);              // demora de 3 segundos
  celda.power_up();         // despierta al módulo HX711
}
}
```

Práctica No.4

Medición de posición usando acelerómetros tri-axiales

Objetivo:

Controlar el movimiento de un objeto digital tridimensional mediante la medición de un acelerómetro tri-axial. La aplicación debe ser monitoreada mediante un entorno gráfico desarrollado en LabVIEW.

Materiales:

- PC con software LabVIEW Instalado.
- Driver de comunicación **NI-VISA** instalado para el establecimiento de la comunicación serial.
- Software Arduino IDE.
- Librerías LINX-MakerHub instaladas (Hobbyist).
- Placa **Arduino UNO o MEGA**-cable de comunicación USB.
- 1 acelerómetro tri-axial ADXL345 o MPU6050
- Board, Fuente DC, cables o caimanes.
- 2 micro-servomotores de 5 v.
- 1 apuntador láser.

Estudio previo

1. Consultar: ¿Qué es un acelerómetro y cuál es su principio de funcionamiento?
2. ¿Qué tipos de acelerómetros se encuentran en el mercado?. Elabore una tabla donde identifique sus principales características técnicas relacionando: ancho de banda, sensibilidad, materiales, niveles de polarización, etc.
3. Consulte en qué tipo de aplicaciones industriales podría usarse un acelerómetro. Liste varias de ellas.
4. Consulte qué aplicaciones están disponibles para descarga en su *Smart-phone* que usen el acelerómetro de su celular.
5. Consultar los *datasheets* de los acelerómetros *ADXL345* y *MPU6050*. Realice una tabla donde resalte sus principales características técnicas.
6. Vea el siguiente video de *Youtube* donde se publica una aplicación que usa el acelerómetro ADXL345 usando las Librerías LINX de MakerHub y LabVIEW.
<https://www.youtube.com/watch?v=264OzQsHhUM>
7. Realice el montaje en *board* del (os) circuito(s) propuesto(s) y programe los códigos necesarios en software antes de la práctica. Recuerde que el tiempo en el laboratorio es limitado y lo debe optimizar realizando los montajes y programas en un horario extracurricular.

8. Para presentar su preinforme, use el formato (*template*) de la revista científica IEEE dispuesto en el siguiente enlace, con una extensión máxima de 3-4 páginas:
<https://www.ieee.org/conferences/publishing/templates>.

Experimento

Antes de comenzar su laboratorio, asegúrese de tener instalados todos los programas y driver's de comunicación y firmware necesarios:

- LabVIEW
- Librerías LINX (Hobbyist)
- NI VISA drivers (Comunicación Serial)
- Arduino IDE

En la Figura 1., se muestran los acelerómetros **ADXL345** y **MPU6050** los cuales serán usados en esta práctica (puede ser sólo uno de ellos) para adquirir las variables de posición X, Y y Z. Para cumplir con el objetivo propuesto, la práctica se realizará en cinco (5) partes, las cuales se detallan brevemente a continuación:

Primera parte: Generación de un objeto 3D en LabVIEW y control de su posición de forma manual.

Segunda Parte: Lectura de las señales del acelerómetro tri-axial (ejes x, y, z) usando las librerías LINX de MakerHub (Hobbyist).

Tercera Parte: Lectura de acelerómetro MPU6050 usando comunicación serial y librerías VISA.

Cuarta parte: Control del movimiento de un objeto digital 3D usando un acelerómetro.

Quinta parte: Aplicación a la robótica. Control de servomecanismos mediante la adquisición de señales provenientes de un acelerómetro.

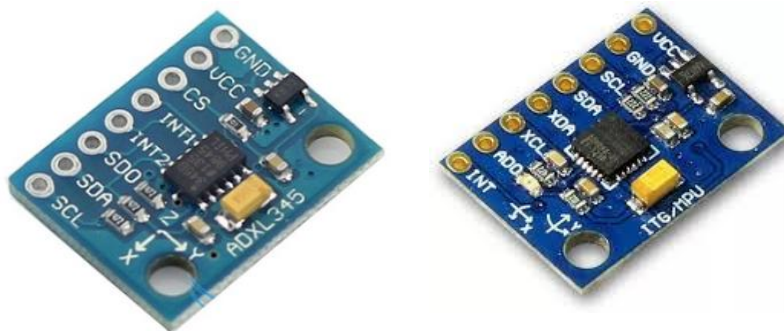


Figura 4.14. Acelerómetro ADXL345 (izquierda) y Acelerómetro MPU6050 (derecha)

PRIMERA PARTE:

Generación de un objeto 3D en LabVIEW y control de su posición de forma manual.

Para crear un objeto 3D en LabVIEW, siga los siguientes pasos:

Tome de la paleta “Object VIs”, el bloque “Create Object.vi”



La ruta para encontrar este VI es la siguiente: *Programming/Graphics & Sound/3D Picture Control/Object/Create Object.Vi*

Este bloque tiene la función de crear un objeto 3D, que se mostrará en un escenario 3D.

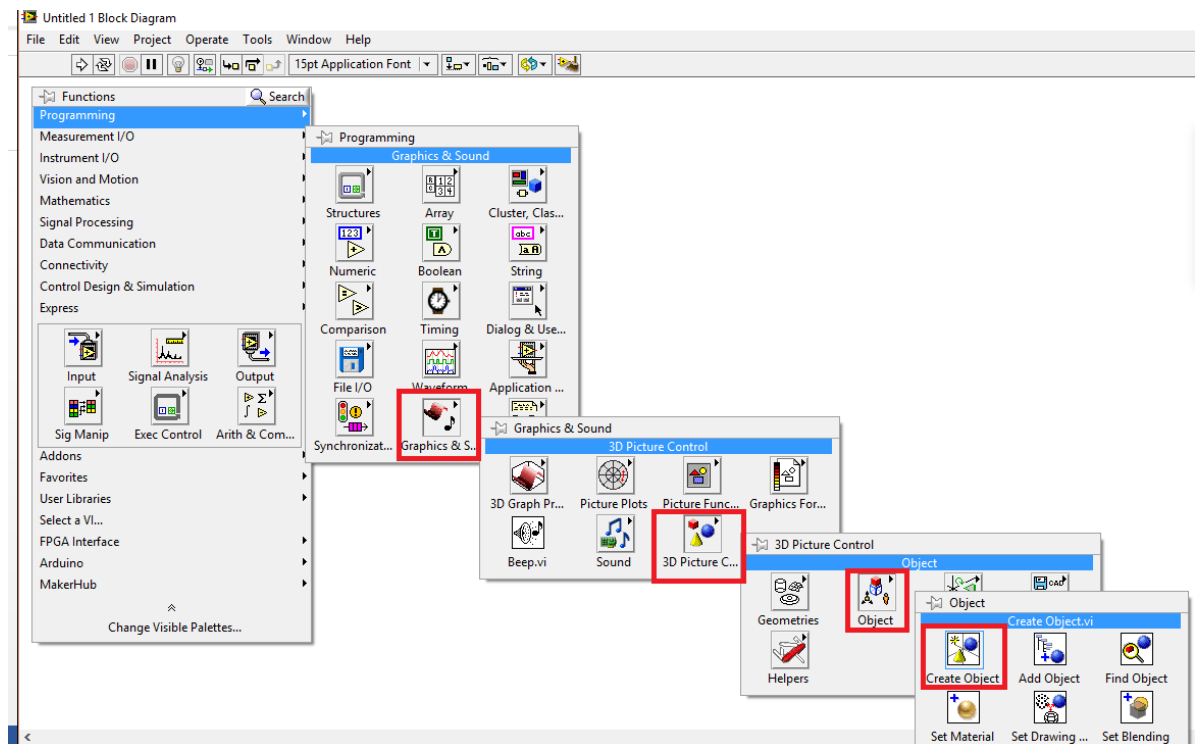


Figura 4.15. Ruta para encontrar el bloque Create object.vi

De la paleta “Application Control VIs” se toma el bloque “Invoke Node.vi” que invoca una acción sobre una referencia específica.

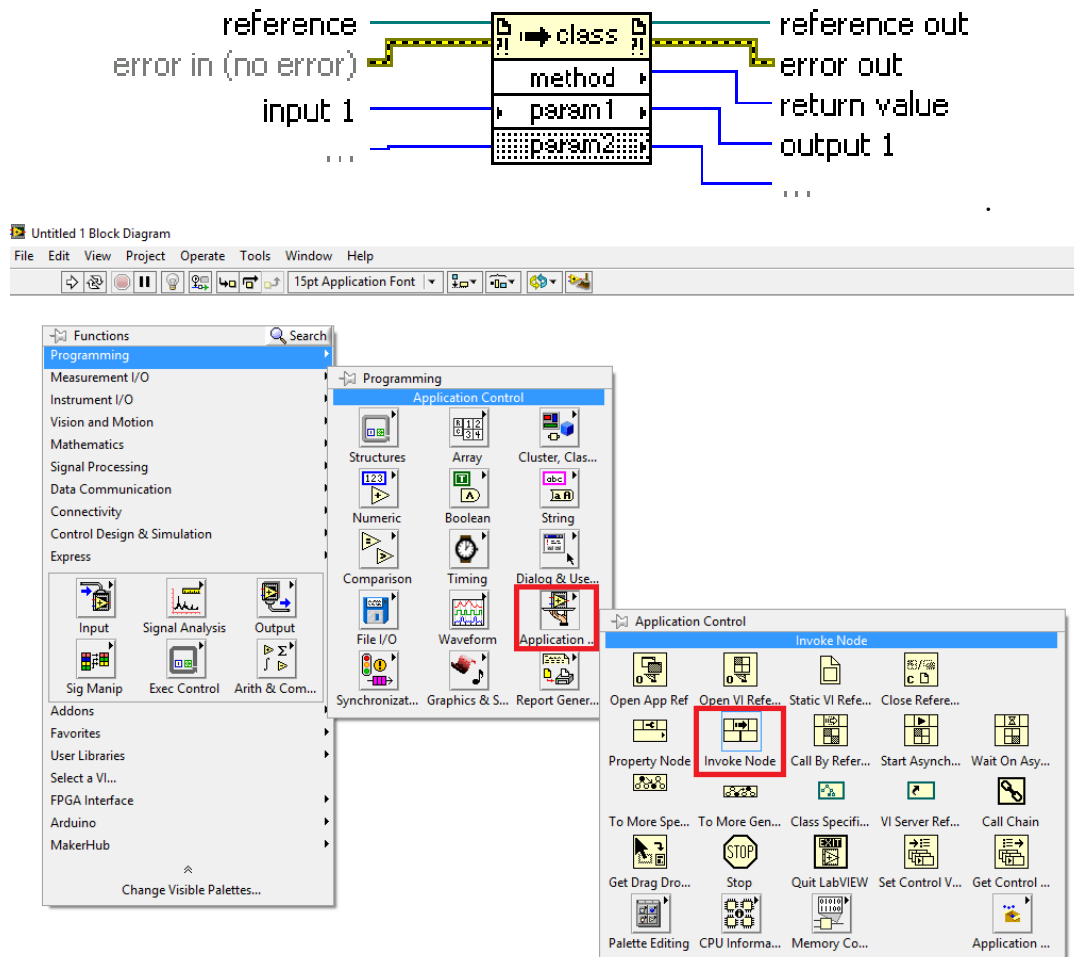
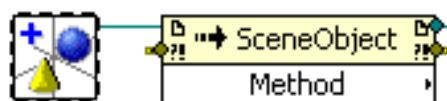
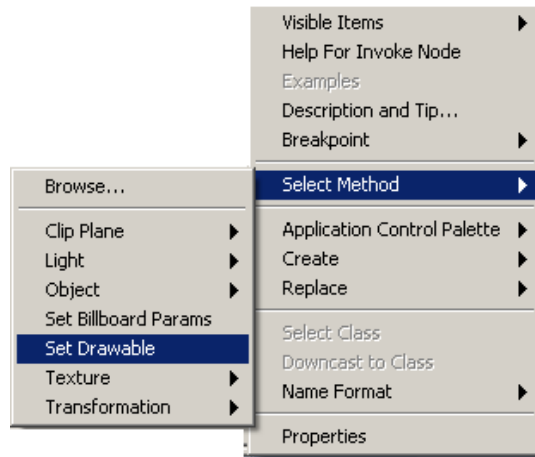


Figura 4.16. Ruta para encontrar el bloque Invoke Node.vi

Al conectar como referencia el bloque “Create Object” inmediatamente el bloque *invoke node* se convierte en un “SceneObject”



Para comenzar a agregar características a nuestro escenario 3D, haga *click* derecho sobre el bloque “Scene object”, y seleccione del menú desplegable la opción “*Select Method*” y luego “*Set Drawable*”.



Allí se agregará el bloque que definirá la forma del objeto que aparecerá en el escenario 3D (un cubo para nuestro caso).

De la paleta “*Geometries Vis*” se usará el bloque “*Create Box.vi*”, allí mediante constantes se define las dimensiones del cubo y la combinación RGB de colores para ésta (Roja por defecto).

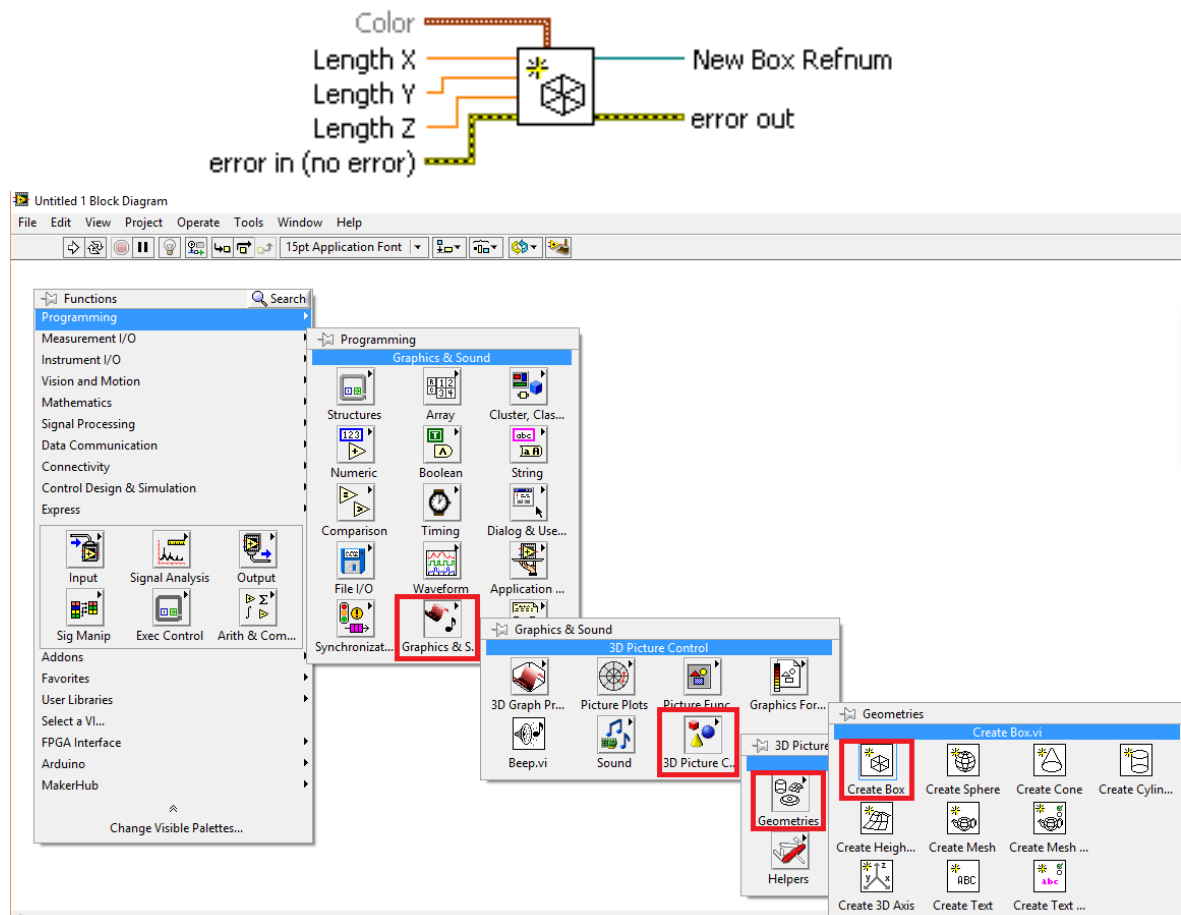
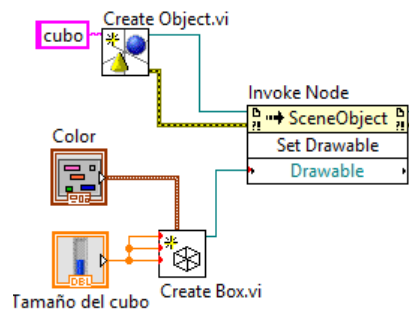


Figura 4.17. Ruta para encontrar el bloque *Create Box.vi*

De esta manera, conectando al bloque *SceneObject*, se podrían agregar más características (Texturas, luminosidad, cambios al objeto, etc).



El algoritmo gráfico para implementar es el siguiente:

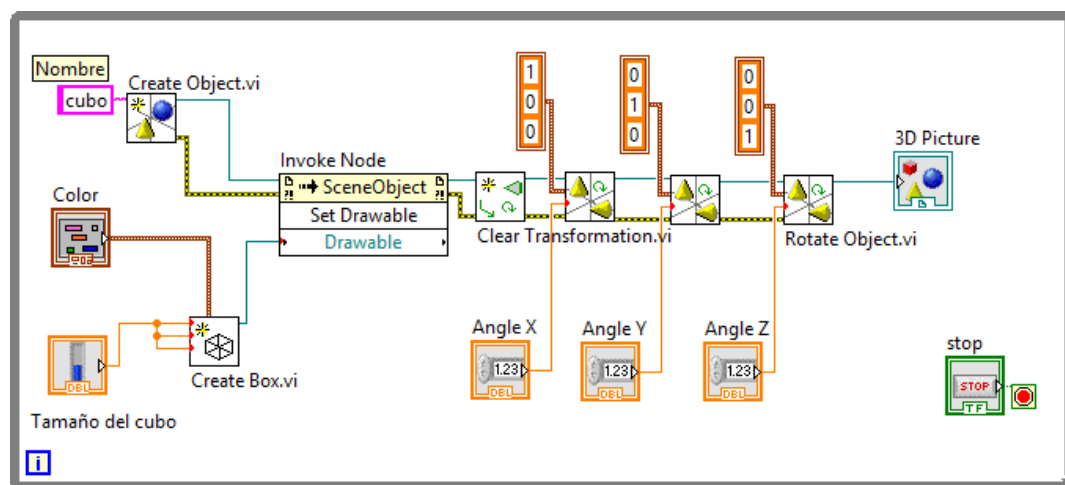


Figura 4.18. Código para la generación de un objeto en 3D

Los bloques “*Rotate Object.vi*” y “*Clear Transformation.vi*” se sacan de la siguiente ruta:

Programming/Graphics & Sound/3D Picture Control/Transformations/Rotate Object.Vi

Programming/Graphics & Sound/3D Picture Control/Transformations/Clear

Transformation.Vi

Para sacar el objeto 3D Picture  siga la siguiente ruta desde la interfaz de usuario o panel frontal de LabVIEW.

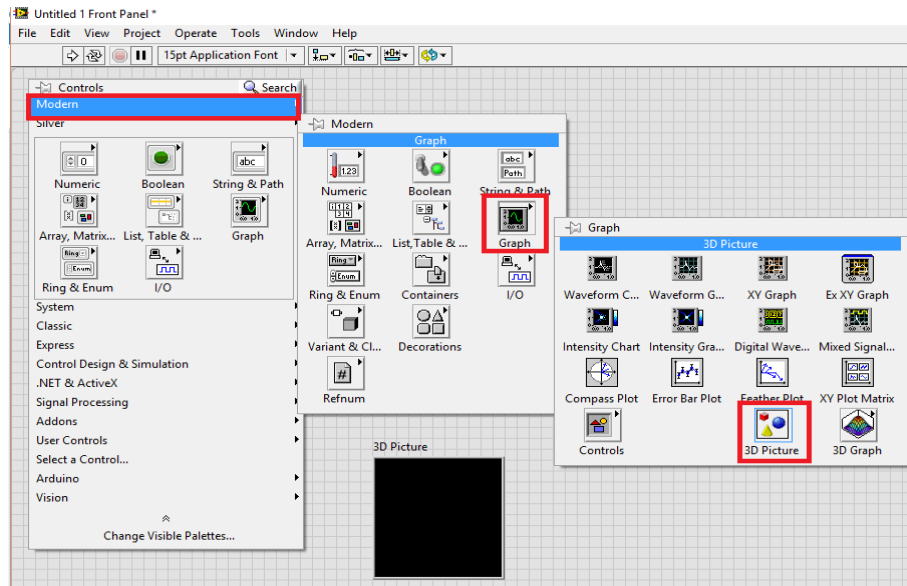


Figura 4.19. Ruta para encontrar el bloque 3D Picture.vi

Finalmente, se tendrá un paralelepípedo en forma de cubo como el observado en la Figura 4.7.

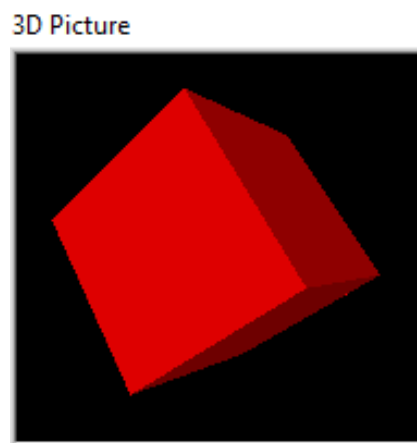


Figura 4.20. Objeto en 3D creado en LabVIEW

Con los controles manuales Angle X, Angle Y, Angle Z se podrá variar manualmente la posición del cubo.

Un ejemplo de la interfaz gráfica a desarrollar es como se muestra a continuación:

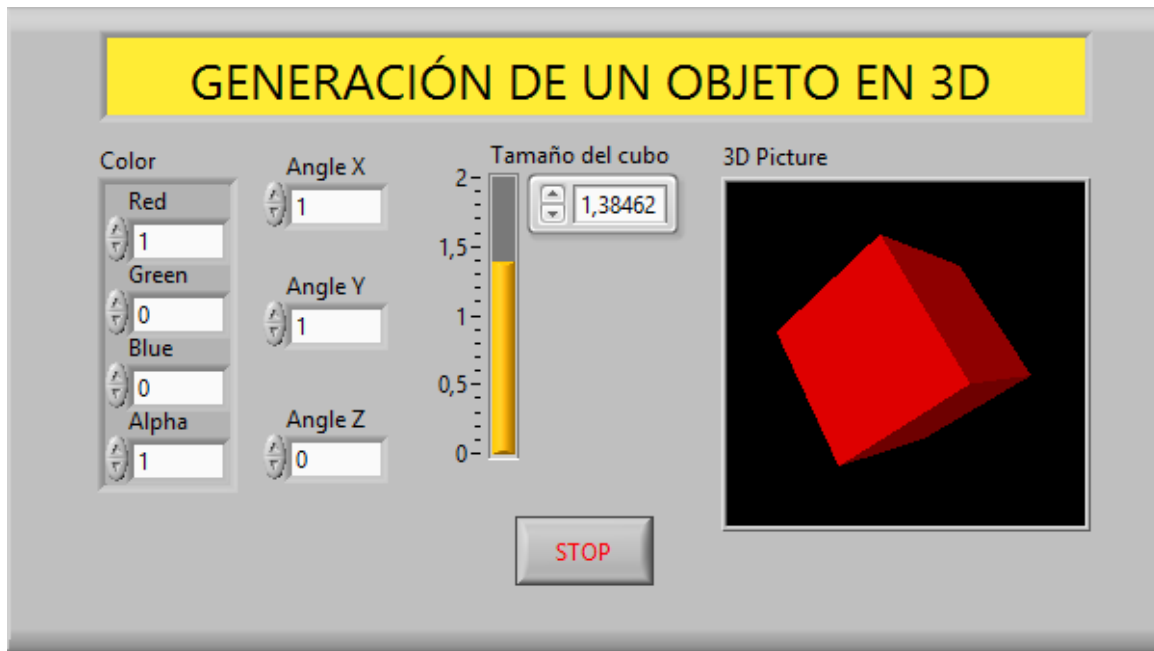


Figura 4.21. Ejemplo de interfaz diseñada en LabVIEW para generación de un objeto en 3D



NOTA: Antes de ejecutar su aplicación, asegúrese que el tamaño del cubo sea diferente de 0, esto generaría un error.

SEGUNDA PARTE

Lectura de señales del acelerómetro tri-axial (ejes x, y, z) usando las librerías LINX de MakerHub (Hobbyist).



NOTA: en esta sección usted podrá trabajar con el acelerómetro que tenga disponible, ambos códigos son similares.

2. Realice el montaje de la Figura 4.10 o Figura 4.11 según sea el caso. En caso de trabajar con el **ADXL345**, conecte los terminales Vcc, GND y CS del acelerómetro a la placa Arduino tal y como se indica en la Figura 4.9. Con el pin CS conectado a Vcc, el dispositivo quedará habilitado y funcionará como esclavo (ver Figura 4.12). Tenga en cuenta que la alimentación será de 3.3V. Arduino dispone de soporte I2C por hardware vinculado físicamente a los pines A4 y A5 para el UNO y los pines 20 y 21 en el MEGA para las señales SDA y SCL respectivamente. Verifique si el pin SDO se encuentra conectado a GND, si es así, la dirección del acelerómetro será

0x53; si se encuentra conectado a Vcc, entonces la dirección será **0x1D**. En el caso de trabajar con el acelerómetro **MPU6050**, la dirección del esclavo será **0x68**.



NOTA: tenga presente que la placa Arduino también dispone de conexiones para I2C (SDA y SCL) localizados en los 2 últimos pines de la bornera digital, cerca al pulsador de Reset.

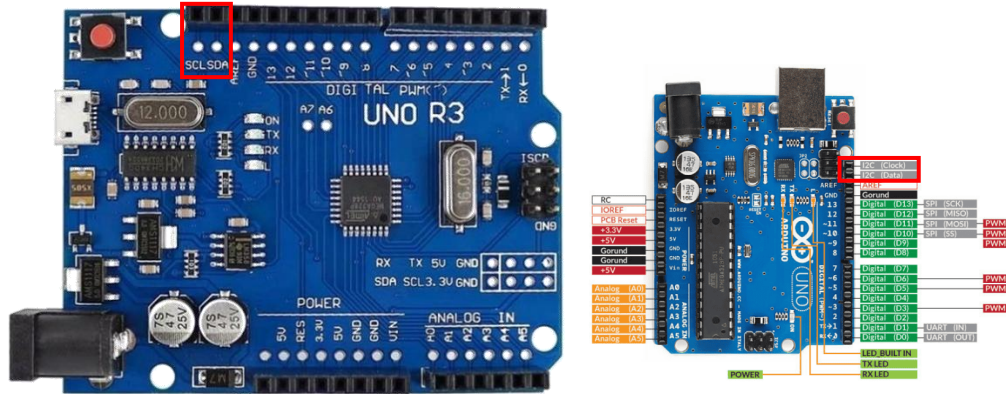


Figura 4.9. Localización de los pines SDA y SCL en el Arduino uno.

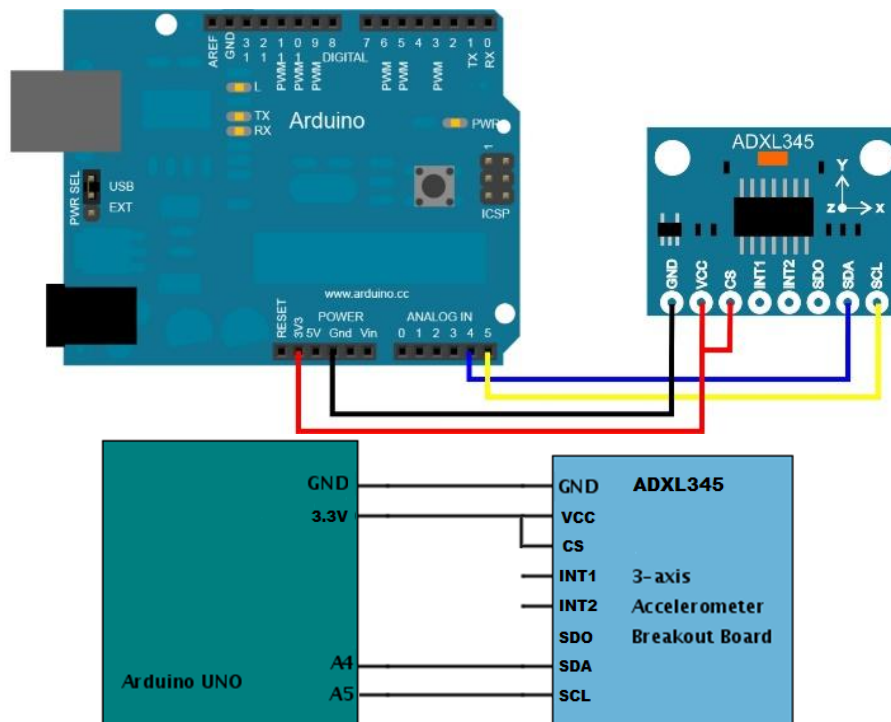


Figura 4.10. Diagrama de conexiones de la placa Arduino al Acelerómetro ADXL345

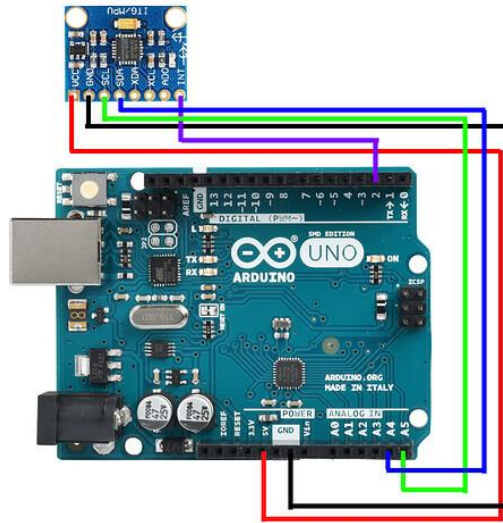


Figura 4.11. Diagrama de conexiones de la placa Arduino al Acelerómetro MPU6050

¿Cómo leer múltiples acelerómetros con el Arduino?

Una forma simple de realizar esta tarea es multiplexando cada acelerómetro, es decir, se realizará la lectura de un acelerómetro a la vez, primero se lee uno, luego el otro y luego el otro, el proceso será cíclico y se repite a alta velocidad.

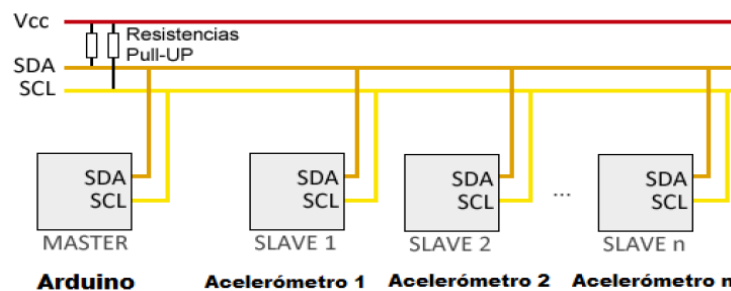


Figura 4.12. Diagrama de conexiones de la placa Arduino trabajando como maestro y varios acelerómetros operando como esclavos en una comunicación I2C

Se trata de destinar un pin de salida del Arduino para activar/desactivar cada acelerómetro. Para habilitar el chip deseado escriba CS en HIGH, mientras que los otros dispositivos se encuentren desactivados con CS en LOW. Es importante que sólo se encuentre un sólo acelerómetro activo a la vez ya que, sobre el mismo bus de datos, no pueden aparecer diferentes dispositivos con la misma dirección.

Recordar: con SDO/ALT ADDRESS = GND, la dirección será **0x53** y con SDO/ALT ADDRESS = VCC, la dirección será **0x1D**.

Antes de proceder con la implementación del código de lectura para el acelerómetro, se sugiere visitar el siguiente enlace y ver el video.

- Video de lectura de acelerómetro ADXL345, con LINX en LabVIEW
<https://www.youtube.com/watch?v=CAW-a4jrHL8>

- Diseñe una interfaz en *LabVIEW* que permita comunicarse con la placa *Arduino UNO* o *MEGA* usando las librerías *LINX*. En ella se debe visualizar la magnitud de la aceleración en los ejes X, Y y Z sensados por el acelerómetro. La Figura 4.13, muestra un ejemplo de una interfaz desarrollada en LabVIEW para generar esta aplicación.

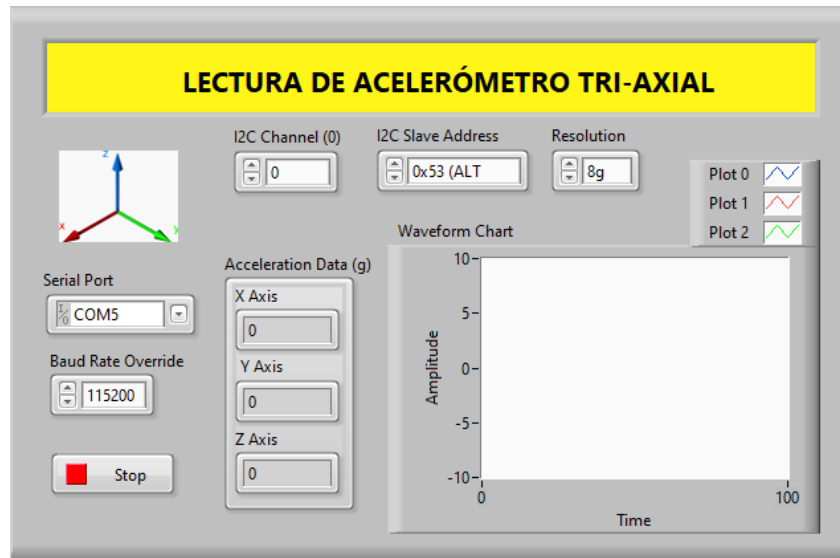


Figura 4.13. Ejemplo de interfaz en LabVIEW para lectura de las señales entregadas por el acelerómetro

El algoritmo gráfico que representa la interfaz de la figura 4.14 es el siguiente:

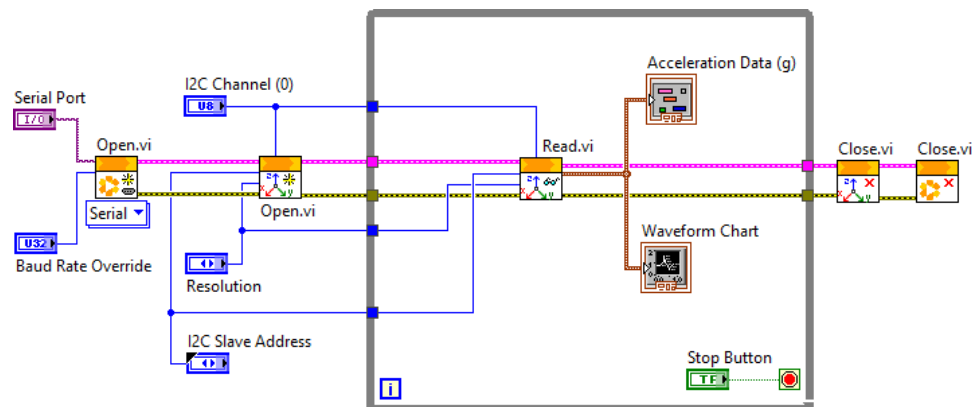


Figura 4.14. Código de la interfaz de la figura 4.13 usando LINX

Los bloques “open.vi” y “Read.vi” se sacan de la siguiente ruta:

MakerHub/LINX/Sensors/Accelerometer/ADXL345/Open.vi

MakerHub/LINX/Sensors/Accelerometer/ADXL345/Read.vi

MakerHub/LINX/Sensors/Accelerometer/MPU6050/Open.vi

MakerHub/LINX/Sensors/Accelerometer/MPU6050/Read.vi

Dependiendo de la magnitud de los niveles de aceleración en X, Y y Z y del tipo de acelerómetro usado (ADXL345 o MPU6050), el estudiante puede verificar si la aplicación funciona mejor usando las variables del Acelerómetro o del Giróscopo. Las unidades de la aceleración vienen dadas en (g- gravedad) y las del giróscopo están dadas en (dps- grados por segundo).

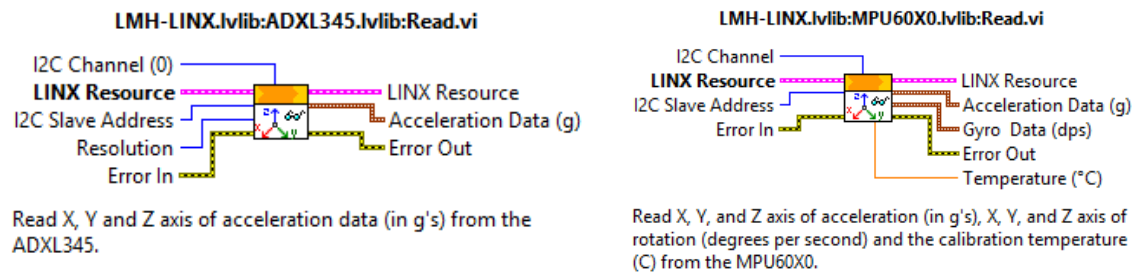


Figura 4.15. Entradas y salidas del bloque de lectura de los acelerómetros

La Figura 4.16 muestra la ruta para sacar los bloques necesarios según el acelerómetro con el que se desee trabajar.

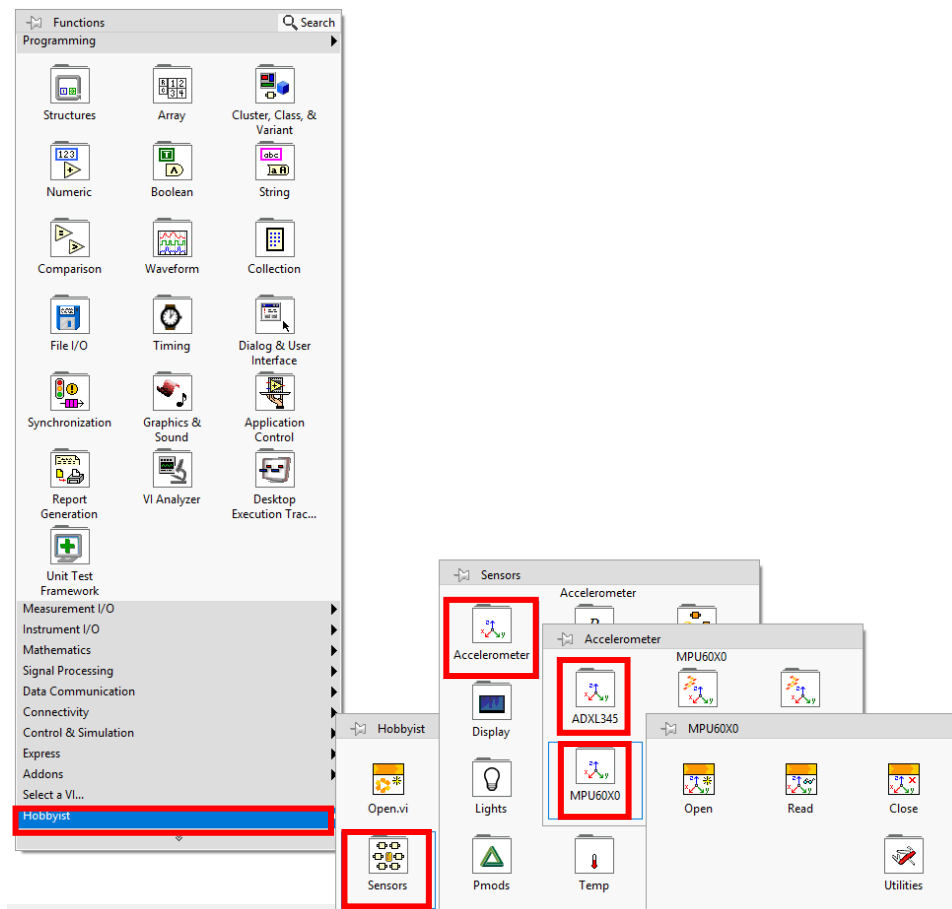


Figura 4.16. Ruta para encontrar los bloques de los acelerómetros

- b. Conecte la Placa *Arduino UNO* o *MEGA* al PC e identifique el puerto COM donde se ha iniciado la comunicación con el dispositivo y ejecute la aplicación desarrollada verificando su buen funcionamiento.

TERCERA PARTE:

Lectura de acelerómetro MPU6050 usando comunicación serial y librerías VISA

En esta sección se ilustra un método alternativo para la comunicación con el acelerómetro donde no se usarán las librerías LINX (Hobbyist).

Para desarrollar esta parte, conecte el acelerómetro a la placa Arduino como se indica en la Figura 4.18 y codifique su algoritmo gráfico como se muestra en la Figura 4.19. Una vez hecho esto, programe el Arduino con el código presentado en el **Anexo 4.1**. Para que el programa funcione correctamente, se debe instalar la siguiente librería en la plataforma de Arduino IDE: **Adafruit MPU6050**

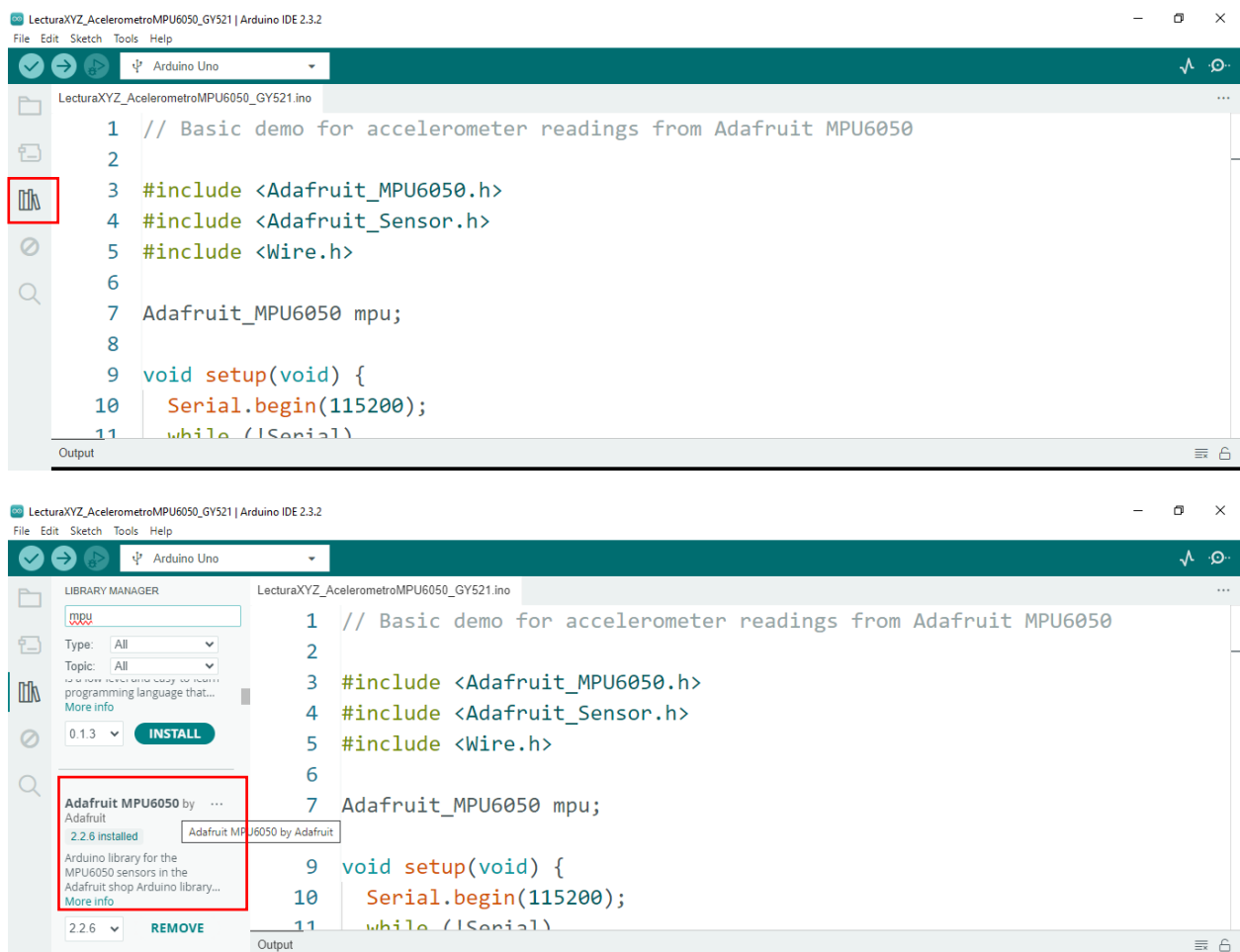


Figura 4.17. Instalación de las librerías de Adafruit MPU6050 en Arduino IDE

NOTA: Nótese que al final del código se arma una trama serial a partir de instrucciones Serial.print, la cual es enviada por el puerto y que luego será desconcatenada por la aplicación en LabVIEW para mostrar las variables X, Y y Z separadamente.

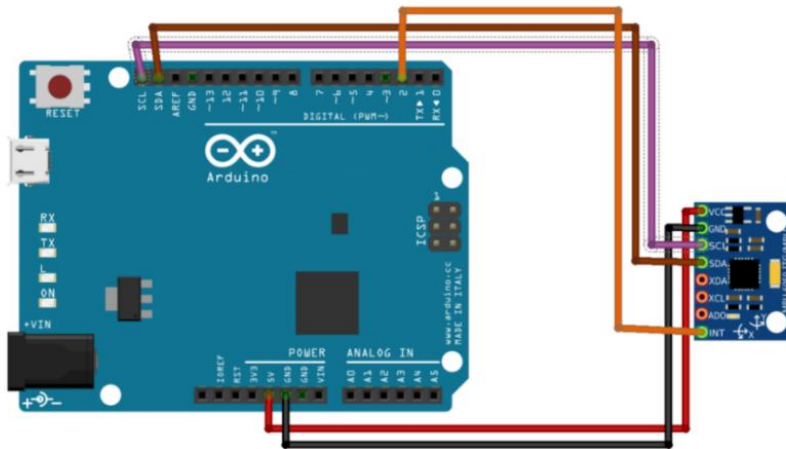


Figura 4.18. Conexión del acelerómetro al Arduino para establecer comunicación serial

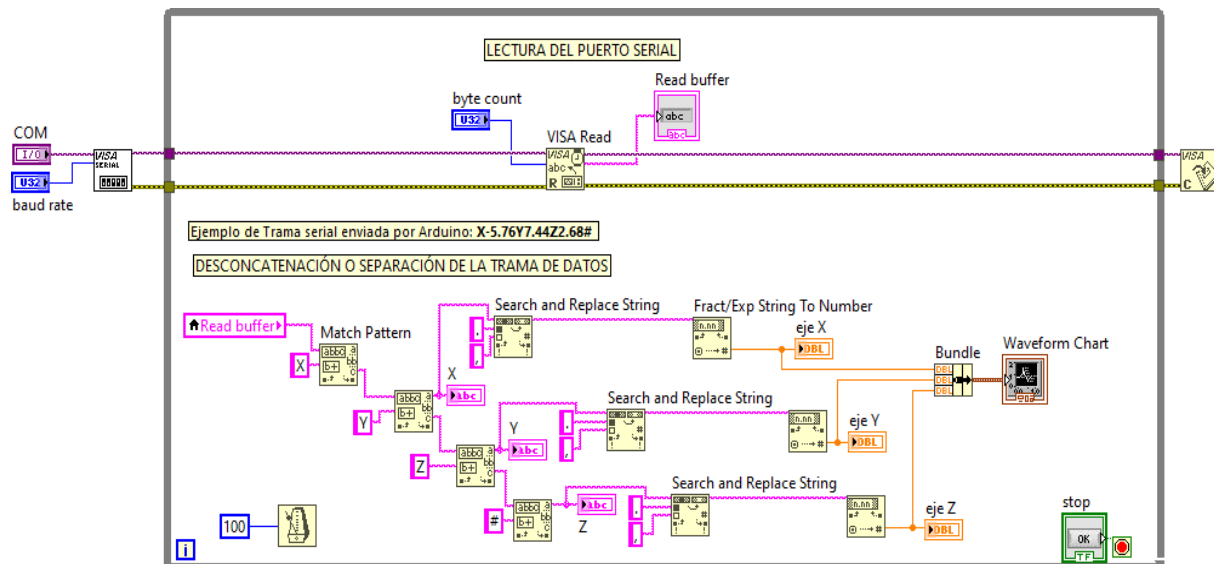


Figura 4.19. Diagrama de bloques en LabVIEW para lectura del acelerómetro usando comunicación serial

CUARTA PARTE:

Control del movimiento de un objeto digital 3D usando un acelerómetro

Antes de continuar, se sugiere ver los siguientes enlaces, los cuales integran el sensado de las variables X, Y y Z de un acelerómetro tri-axial con el movimiento de un objeto en un escenario 3D diseñado en LabVIEW.

- <https://www.youtube.com/watch?v=uzCplBoC0mM>
- <https://www.youtube.com/watch?v=X2nyi2DltII>

Para realizar esta aplicación, se debe unir en un sólo código los dos programas generados previamente (Parte 1 y Parte 2 ó 3). La figura 4.20, muestra un ejemplo de una interfaz para desarrollar esta aplicación usando el acelerómetro ADXL345.

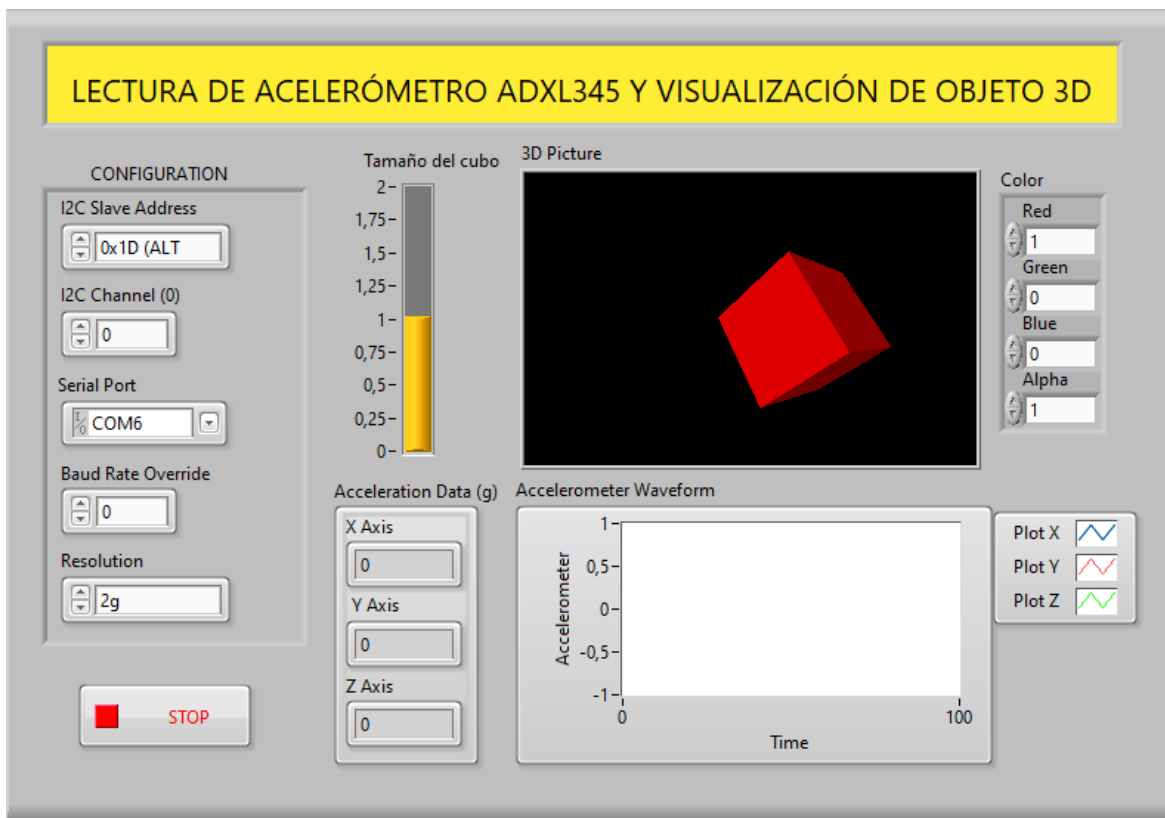


Figura 4.20. Ejemplo de una interfaz para sensar aceleraciones de un objeto indicando su posición

El algoritmo gráfico que representa la interfaz de la figura 4.20 se muestra en la Figura 4.21.

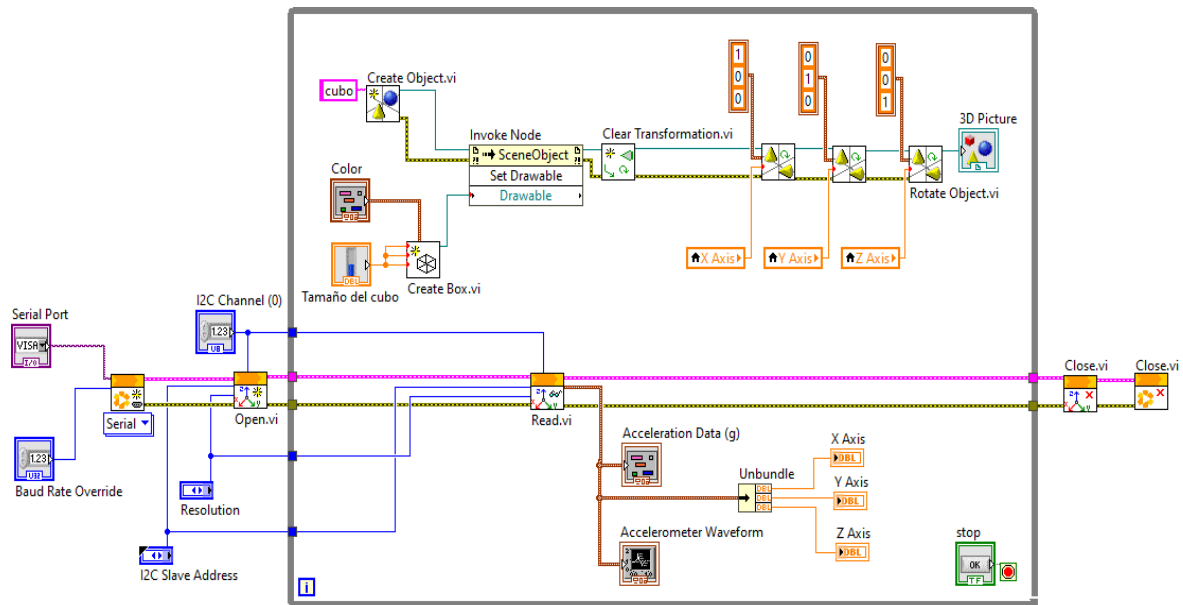


Figura 4.21. Código de la interfaz de la figura 4.17 usando LINX y el acelerómetro ADXL345

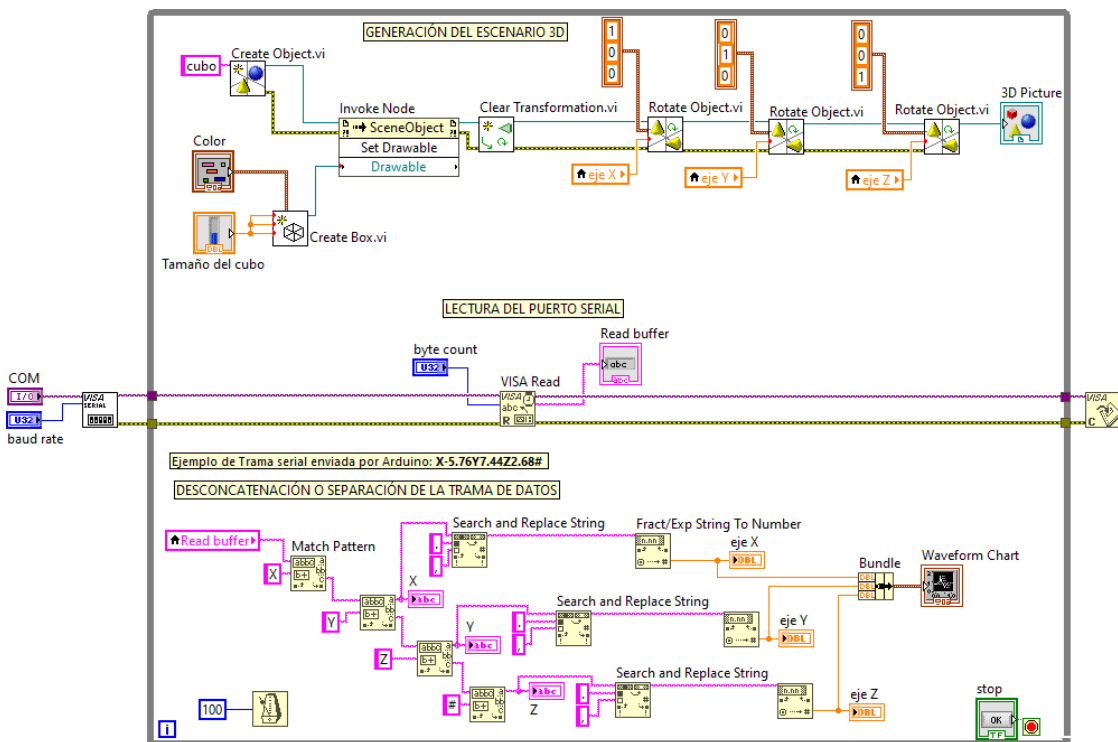


Figura 4.22. Código de la interfaz de la figura 4.19 usando VISA y el acelerómetro MPU6050

QUINTA PARTE

Aplicación a la robótica: Control de servomecanismos mediante la adquisición de señales provenientes de un acelerómetro.

Para realizar este trabajo, considere la obtención de dos servomotores y un apuntador laser de juguete, como los que se muestran en la figura 4.23.



Figura 4.23. Servomotor de 5V y apuntador laser

Visite el siguiente enlace donde podrá encontrar una aplicación orientada a la robótica que le servirá de contexto para realizar el siguiente trabajo:

Video de aplicación con acelerómetro ADXL345, arduino y servo-motor con apuntador laser
<https://www.youtube.com/watch?v=8cBBB3KOgzs>

En este otro video se explica el código usado y se incluyen las librerías necesarias para programar el Arduino

<https://www.youtube.com/watch?v=GrYXvdKj3T4>

También podrá programar la aplicación usando las librerías LINX (Hobbyist) en lugar del código de texto en lenguaje C del Arduino. En la figura 4.24 se observa el ejemplo de un código usando las Librerías LINX de MakerHub para sensar el acelerómetro y controlar el movimiento de dos (2) servomotores.

Nótese en el siguiente ejemplo como se controlan los canales digitales donde se encuentran conectados físicamente los dos servomotores con la lectura del acelerómetro (ejes X y Y). Cada eje moverá su respectivo servomotor y cada fórmula matemática usada, tiene la función de convertir o escalar las variables de aceleración (o ángulo) en grados de movimiento.

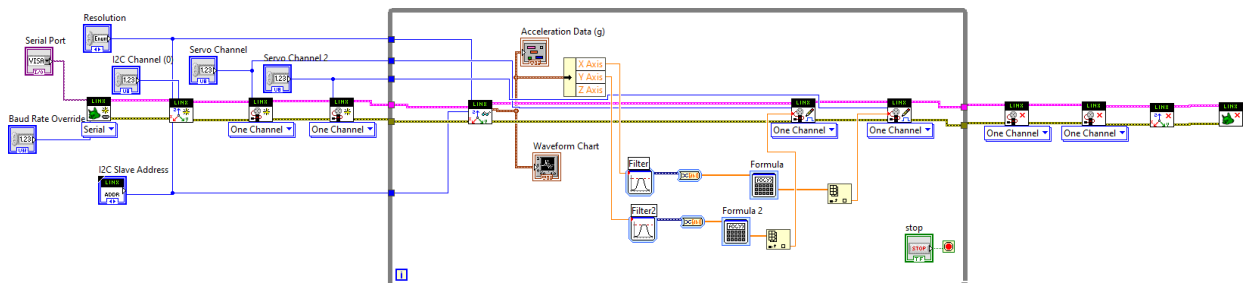


Figura 4.24. Código en LabVIEW para sensar acelerómetros tri-axiales y controlar dos servomotores

Conclusiones

- Escriba las conclusiones más relevantes de la práctica de laboratorio, indicando hallazgos, dificultades y logros, comparando los procedimientos desarrollados en las cinco partes de esta práctica.

ANEXO 4.1

Código Arduino Lectura acelerómetro MPU6050

```
// Basic demo for accelerometer readings from Adafruit MPU6050
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>
Adafruit_MPU6050 mpu;

void setup(void) {
  Serial.begin(115200);
  while (!Serial)
    delay(10); // will pause Zero, Leonardo, etc until serial console opens
  //Serial.println("Adafruit MPU6050 test!");
  // Try to initialize!
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1) {
      delay(10);
    }
  }
  //Serial.println("MPU6050 Found!");
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  //Serial.print("Accelerometer range set to: ");
  switch (mpu.getAccelerometerRange()) {
    case MPU6050_RANGE_2_G:
      Serial.println("+2G");
      break;
    case MPU6050_RANGE_4_G:
      Serial.println("+4G");
      break;
    case MPU6050_RANGE_8_G:
      //Serial.println("+8G");
      break;
    case MPU6050_RANGE_16_G:
      Serial.println("+16G");
      break;
  }
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  //Serial.print("Gyro range set to: ");
  switch (mpu.getGyroRange()) {
    case MPU6050_RANGE_250_DEG:
      Serial.println("+ 250 deg/s");
      break;
    case MPU6050_RANGE_500_DEG:
      //Serial.println("+ 500 deg/s");
      break;
    case MPU6050_RANGE_1000_DEG:
      Serial.println("+ 1000 deg/s");
      break;
    case MPU6050_RANGE_2000_DEG:
      Serial.println("+ 2000 deg/s");
      break;
  }
```

```

}

mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
//Serial.print("Filter bandwidth set to: ");
switch (mpu.getFilterBandwidth()) {
case MPU6050_BAND_260_HZ:
  Serial.println("260 Hz");
  break;
case MPU6050_BAND_184_HZ:
  Serial.println("184 Hz");
  break;
case MPU6050_BAND_94_HZ:
  Serial.println("94 Hz");
  break;
case MPU6050_BAND_44_HZ:
  Serial.println("44 Hz");
  break;
case MPU6050_BAND_21_HZ:
  //Serial.println("21 Hz");
  break;
case MPU6050_BAND_10_HZ:
  Serial.println("10 Hz");
  break;
case MPU6050_BAND_5_HZ:
  Serial.println("5 Hz");
  break;
}

//Serial.println("");
//delay(100);
}
void loop() {

  /* Get new sensor events with the readings */
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);
  /* Print out the values */
  Serial.print("X");
  Serial.print(a.acceleration.x);
  Serial.print("Y");
  Serial.print(a.acceleration.y);
  Serial.print("Z");
  Serial.print(a.acceleration.z);
  Serial.println("#");
  //Serial.println(" m/s^2");
  /*Serial.print("Rotation X: ");
  Serial.print(g.gyro.x);
  Serial.print(", Y: ");
  Serial.print(g.gyro.y);
  Serial.print(", Z: ");
  Serial.print(g.gyro.z);
  Serial.println(" rad/s");
  Serial.print("Temperature: ");
  Serial.print(temp.temperature);
  Serial.println(" degC");
  Serial.println("");*/
  delay(100);
}

```


Práctica No.5

Medición de distancia por ultrasonido e imágenes de profundidad

Objetivo:

Implementar una aplicación para medir distancia usando sensores ultrasónicos y cámaras de profundidad. La aplicación debe ser monitoreada mediante un entorno gráfico desarrollado en LabVIEW.

Materiales:

- PC con software LabVIEW Instalado.
- Driver de comunicación **NI-VISA** instalado para el establecimiento de la comunicación serial.
- Software Arduino IDE.
- Librerías LINX-MakerHub instaladas (Hobbyist).
- Software Anaconda (Spyder- Python) instalado
- Driver para Cámaras ASTRA-series
- Librerías OpenNI instaladas
- Placa **Arduino UNO o MEGA**-cable de comunicación USB.
- 1 sensor de ultrasonido HC-SR04
- 1 sensor de profundidad (3D) Orbbec Astra
- Board, Fuente DC, cables o caimanes.

Estudio Previo

1. Consultar: ¿Cuál es el rango de audición del oído humano? , ¿Cómo se define un ultrasonido y un infrasonido?
2. Consultar: ¿Cuál es la velocidad del sonido para diferentes medios de propagación de la onda?. Realice una tabla donde relacione dichas constantes según el medio de propagación y/o condiciones ambientales.
3. Consulte en qué tipo de aplicaciones podría usarse un sensor ultrasónico. Liste varias de ellas.
4. Consultar los *datasheets* de los módulos ultrasónicos *HC-SR04* y *HC-SR05*. Explicar sus principales diferencias técnicas.
5. Ver el siguiente video de *Youtube* y explicar en sus propias palabras y desde un punto de vista metodológico cómo se realizará la medición de distancia usando el sensor ultrasónico:
Video tutorial de la conexión, programación y aplicación

https://www.youtube.com/watch?v=b-5lcVebV_w

6. Consultar las características principales del sensor de profundidad Orbecc Astra: Qué es, cómo funciona, cuál es su principio de funcionamiento y en cuáles aplicaciones de ingeniería se podría implementar.
7. De acuerdo al *datasheet* de la cámara Orbbec Astra, construya una tabla donde se especifiquen sus principales características técnicas: rangos de operación, resolución, tecnología usada, dimensiones físicas, etc. Incluya una imagen ilustrativa.
8. Realice el montaje en *board* del (os) circuito(s) propuesto(s) y programe los códigos necesarios en software antes de la práctica. Recuerde que el tiempo en el laboratorio es limitado y lo debe optimizar realizando los montajes y programas en un horario extracurricular.
9. Para presentar su preinforme, use el formato (*template*) de la revista científica IEEE dispuesto en el siguiente enlace, con una extensión máxima de 3-4 páginas:
<https://www.ieee.org/conferences/publishing/templates> .

Experimento

Antes de comenzar su laboratorio, asegúrese de tener instalados todos los programas y driver's de comunicación y firmware necesarios:

- LabVIEW
- Librerías LINX (Hobbyist)
- NI VISA drivers (Comunicación Serial)
- Arduino IDE
- Anaconda (Spyder-Python)
- Librerías para la cámara de profundidad Orbbec-Astra.

En la Figura 1., se muestra el sensor ultrasónico HC-SR04 el cual será usado en esta práctica para medir distancias. Para cumplir con el objetivo propuesto, la programación del *Arduino* se realizará por dos métodos diferentes, el primero es usando código en modo texto (lenguaje C) y monitor serial y el segundo, usando lenguaje gráfico en el entorno de LabVIEW por medio de las librerías **LINX**.

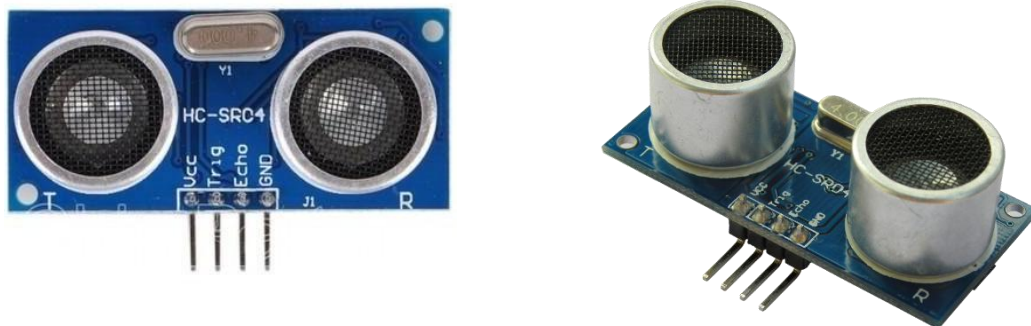


Figura 5.22. Vista superior y e isométrica del sensor ultrasónico HC-SR04

¿Cómo se convierte el tiempo medido por el sensor en una distancia?

Considerando que la velocidad del sonido en el aire es de 340 m/s, o **0.034 cm/ μ s** (ya que trabajaremos con centímetros y microsegundos). Para calcular la distancia, recordaremos que $v=d/t$ (definición de velocidad: distancia recorrida en un determinado tiempo).

De la fórmula anterior se despeja d , obteniendo $d=v \cdot t$, siendo v la constante anteriormente citada y t el valor devuelto por el sensor a la placa Arduino. También habrá que dividir el resultado entre 2 dado que el tiempo recibido es el tiempo de ida y vuelta; es decir, $v=0.034/2= 0.017$

Medición de distancias programando el Arduino en lenguaje C y visualizando la medida en un monitor serial

1. Conecte la Placa *Arduino UNO* o *MEGA* al PC e identifique el puerto COM donde se ha iniciado la comunicación con el dispositivo. Abra la interfaz de Arduino IDE (Ver figura 5.2) y digite el código mostrado a continuación. Programe la placa.

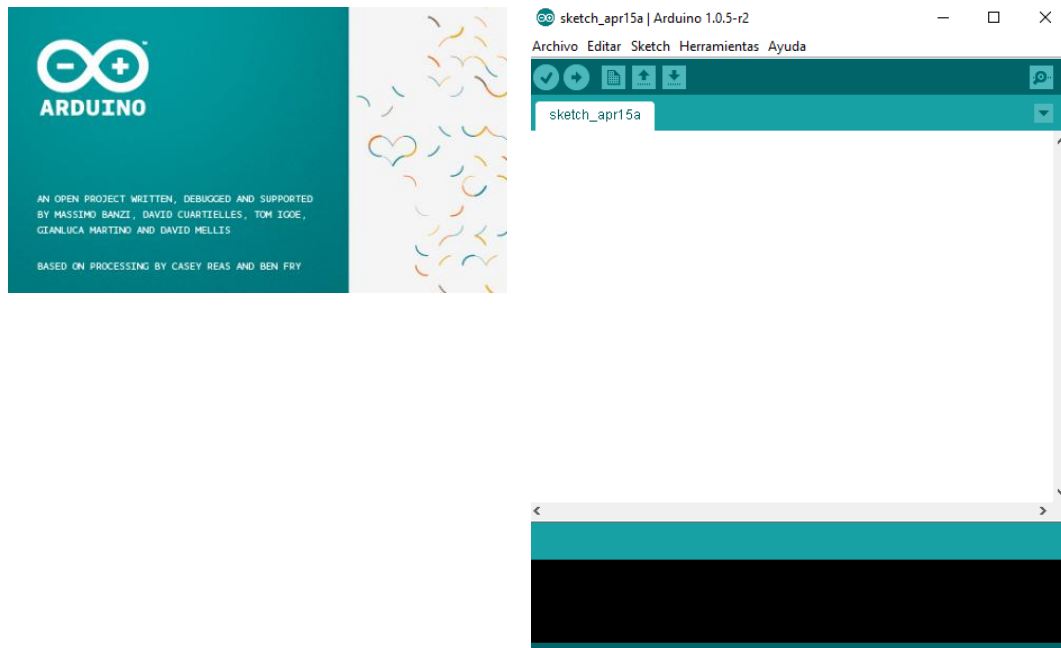


Figura 5.23. Interfaz de Arduino IDE para edición de código de texto en lenguaje C

Código para programar el Arduino en lenguaje C (modo texto)

```
long distancia;
long tiempo;
void setup() {
    Serial.begin(9600);
    pinMode(12, OUTPUT); /*pin 12 como salida: para el pulso ultrasónico, (TRIG)*/
    pinMode(11, INPUT); /*pin 11 como entrada: tiempo del rebote, (ECO)*/
}
void loop() {
    digitalWrite(12, LOW); /* estabilización del sensor*/
    delayMicroseconds(5); /* se espera 5 us para estabilización*/
    digitalWrite(12, HIGH); /* envío del pulso ultrasónico*/
    delayMicroseconds(10); /* tiempo que dura en alto el pulso (10us)*/
    digitalWrite(12, LOW); /* apago el pulso ultrasónico*/
    tiempo=pulseIn(11, HIGH); /* Función para medir la longitud del pulso entrante.
Mide el tiempo que ha transcurrido entre el
envío del pulso ultrasónico y cuando el sensor recibe el rebote.*/
    distancia= int(0.017*tiempo); /*fórmula para calcular la distancia
obteniendo un valor entero. d=V*t*/

    /*Monitorización en centímetros por el monitor serial*/
    Serial.println("Distancia:");
    Serial.println(distancia);
    Serial.println("cm");
    delay(1000);
}
```

2. Realice el montaje de la Figura 5.3. Conecte los terminales Vcc y Gnd del sensor a la placa Arduino tal y como se indica en la figura y los terminales de TRIG y ECHO a los pines digitales 12 y 11 respectivamente (pueden ser otros dependiendo de su preferencia y elección según el código).

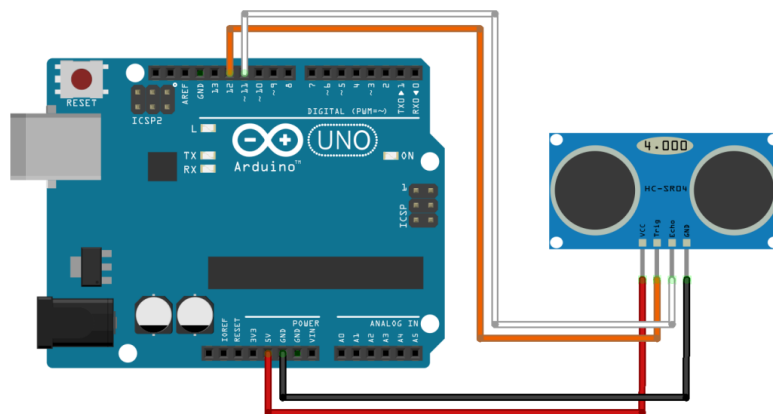


Figura 5.24. Diagrama de conexiones del Arduino y el sensor ultrasónico

3. Se requiere visualizar los resultados de la medición de distancia en el monitor serial que viene incorporado en el software de Arduino IDE (o cualquier otro monitor serial

similar como la HyperTerminal, Puty, etc). Para ello abra el monitor a 9600 baudios, como se muestra en la Figura 5.4 y verifique el funcionamiento de la aplicación.

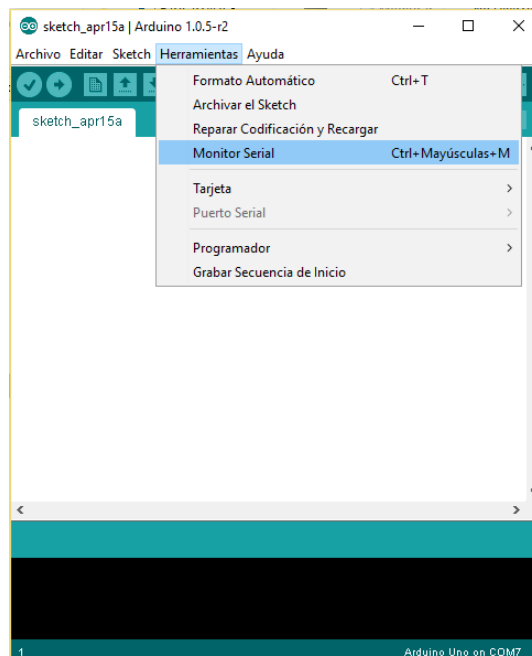


Figura 5.25. Monitor serial de Arduino

4. Diseñe la interfaz en LabVIEW respectiva para visualizar las mediciones de la distancia enviada por el puerto serial. En la Figura 5.5, se muestra un ejemplo de la interfaz de usuario.



Figura 5.26. Interfaz de usuario desarrollada en LabVIEW recibiendo datos por comunicación serial

En la Figura 5.6, se muestra el código gráfico de la interfaz representada en la figura 5.5.

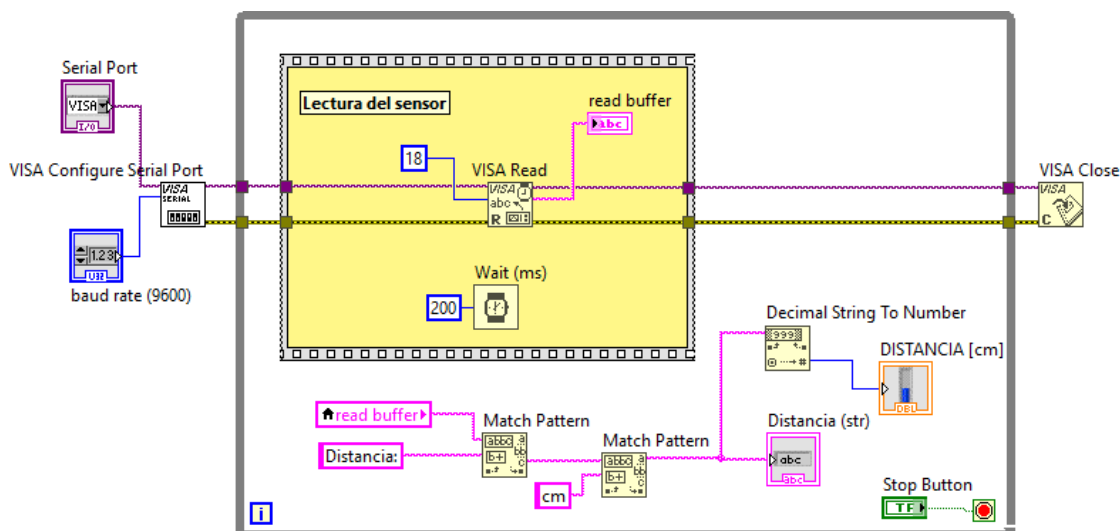


Figura 5.27. Código gráfico en LabVIEW de la interfaz presentada en la figura 5.5

Si requiere más información respecto a la programación, conexión y visualización de los resultados de la aplicación, visite el siguiente enlace donde encontrará un video tutorial:

Video tutorial de la conexión, programación y aplicación con alarma

<https://www.youtube.com/watch?v=IF1eN0WK3bU&nohtml5=False>

5. Después de tener todo el montaje y la programación lista, verifique mediante un flexómetro o una regla las medidas de distancia realizadas por el sensor hasta un objeto cualquiera que esté a su alcance como se ilustra en la Figura 5.7.

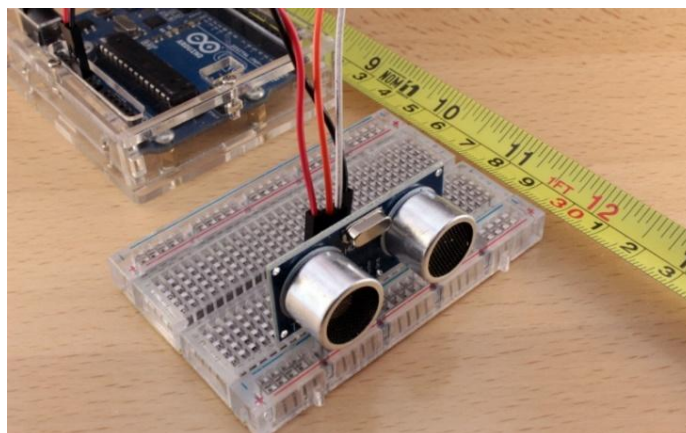


Figura 5.28. Montaje final de la aplicación. Verificación de distancia mediante flexómetro

Medición de distancias programando el Arduino en código gráfico usando librerías LINX de LabVIEW

6. Diseñe una interfaz en *LabVIEW* que permita comunicarse con la placa *Arduino UNO* o *MEGA* usando las librerías *LINX*. En ella se debe visualizar la distancia medida por el sensor ultrasónico a un objeto que se encuentre a su alcance. La Figura 5.8, muestra un ejemplo de una interfaz desarrollada en LabVIEW para generar esta aplicación.

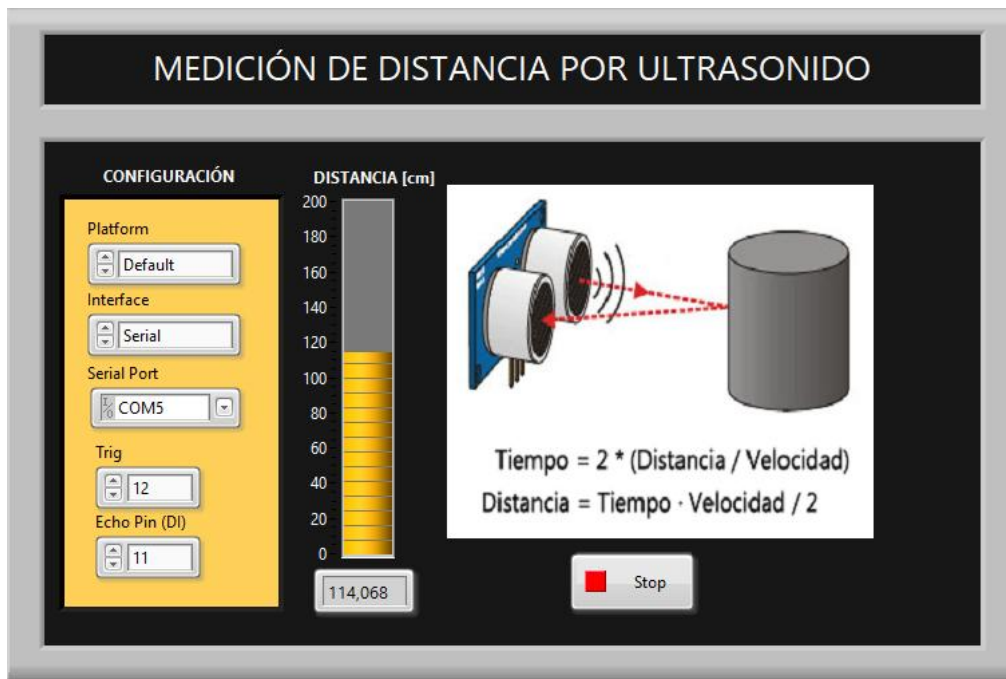


Figura 5.29. Ejemplo de interfaz en LabVIEW para medir distancias por ultrasonido

La Figura 5.9, muestra el código gráfico de la interfaz mostrada en la Figura 5.8.

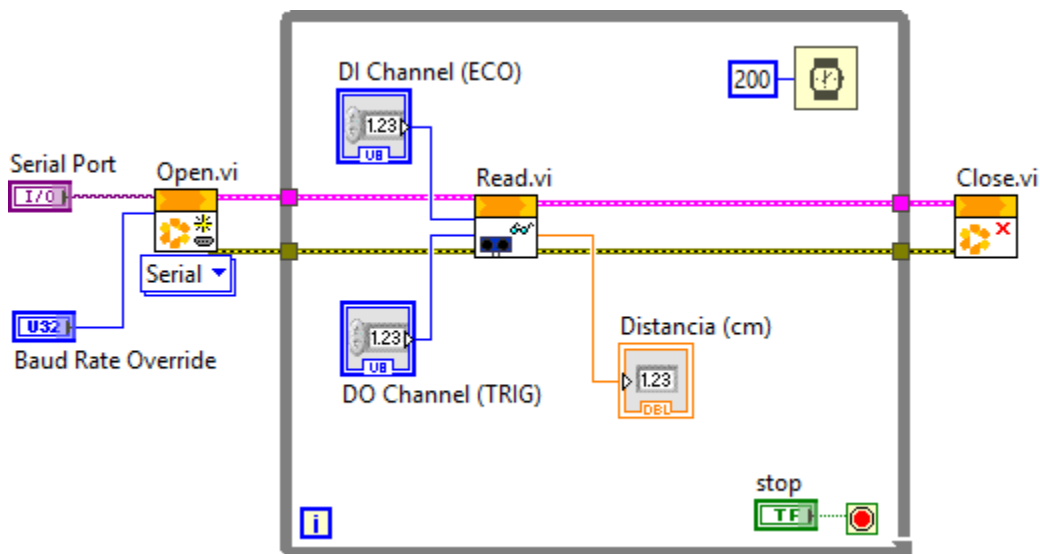


Figura 5.30. Código de la aplicación mostrada en la figura 5.8

Los bloques usados en el código son los siguientes:

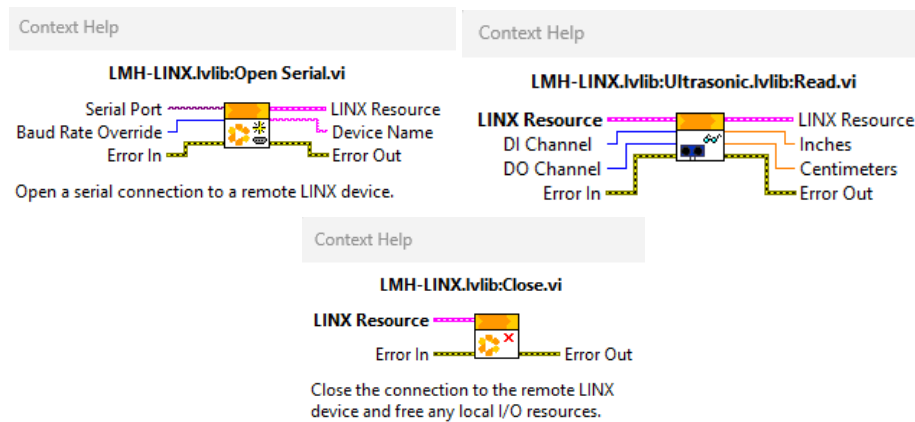


Figura 5.31. Bloques usados en la aplicación

La ruta para acceder al bloque de lectura del sensor ultrasónico es:

Hobbyist/Sensors/Distance/Ultrasonic/Read.vi, esta ruta se muestra en la figura 5.11.

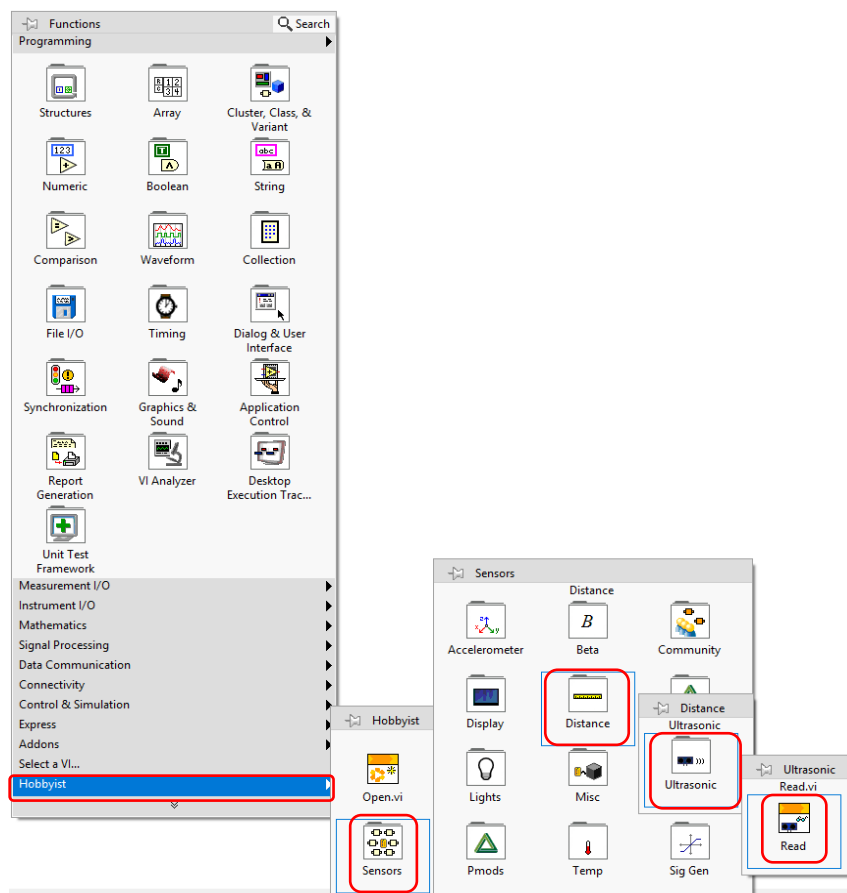


Figura 5.32. Ruta para encontrar los bloques usados en la aplicación

7. Ejecute la aplicación desarrollada en LabVIEW y verifique su funcionamiento. Compare los resultados obtenidos con el procedimiento desarrollado en la primera parte de esta práctica.

i Nota: Si al ejecutar la aplicación mostrada en la figura 5.9 presenta problemas de lectura del sensor ultrasónico, siga los pasos mostrados en la Figura 5.12.

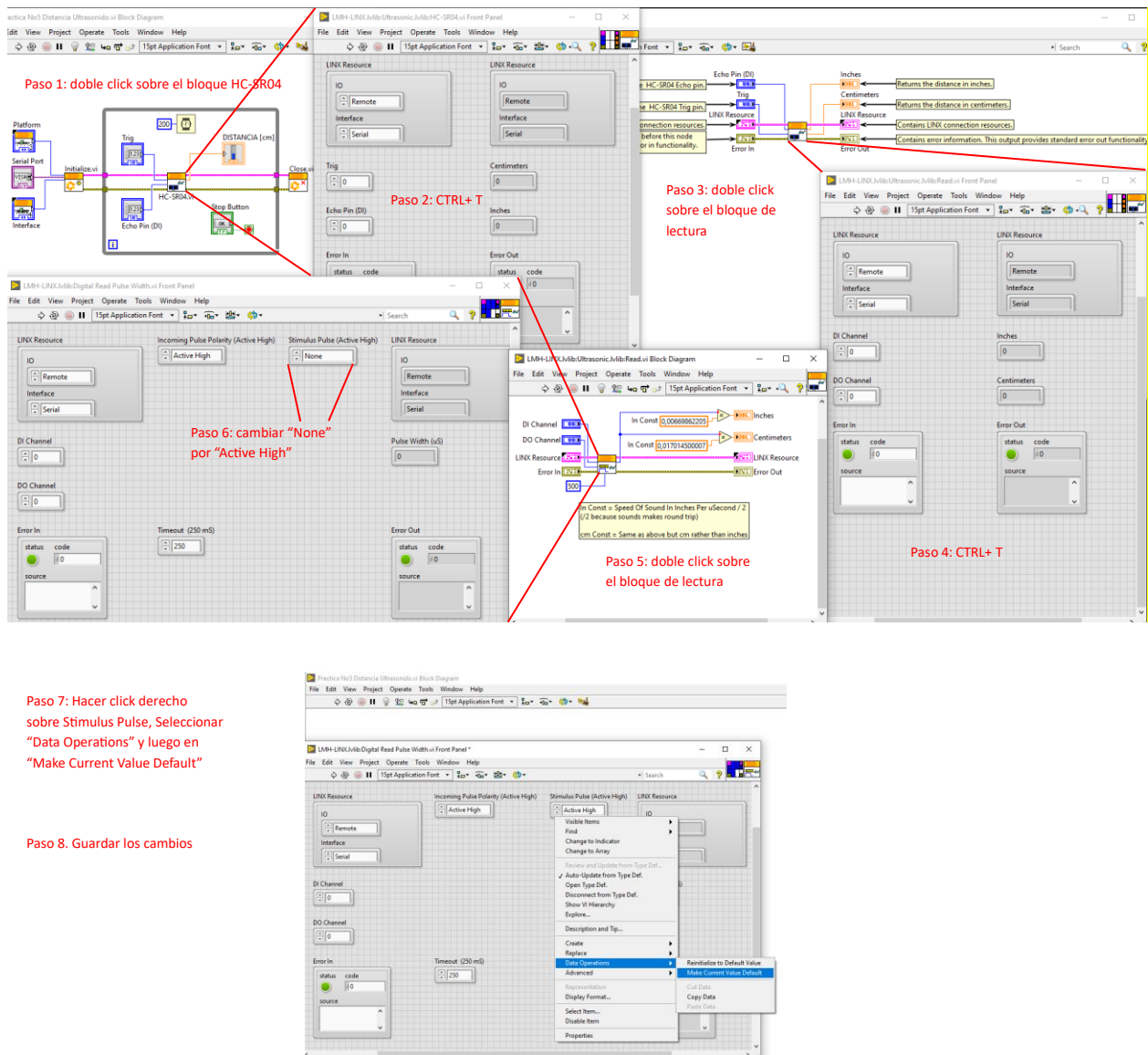


Figura 5.33. Pasos para solucionar el error de lectura del sensor ultrasónico usando las librerías LINX

Se adjunta un link de video con el procedimiento descrito anteriormente:

(3) SOLUCIÓN AL ERROR DEL SENSOR ULTRASÓNICO HC-SR04 EN LABVIEW 2020 - YouTube

Aplicaciones (elija una opción)):

Dispositivo ultrasónico para invidentes

Con los conceptos aprendidos y desarrollados en los numerales anteriores, implemente una aplicación “Sencilla” basada en Software y Hardware, que posibilite la orientación de una persona invidente. El dispositivo puede emitir sonidos a cierta frecuencia o zumbidos que indiquen hacia dónde debe ser su desplazamiento dependiendo de la posición y distancia de un obstáculo en su camino. Como ejemplos prácticos podría pensarse en un bastón o en unos anteojos, la aplicación será de libre elección.



Figura 5.13. Aplicación 1 con sensor ultrasónico

Medición de nivel por ultrasonido

Implemente una aplicación para medir el nivel (% o cm) y el volumen (ml) de una sustancia líquida contenida en un recipiente con graduaciones tipo Erlenmeyer, probeta, vaso beaker, etc. Tanto el nivel medido en %, como el volumen en [ml, cm^3] debe ser mostrado en la interfaz diseñada en LabVIEW.

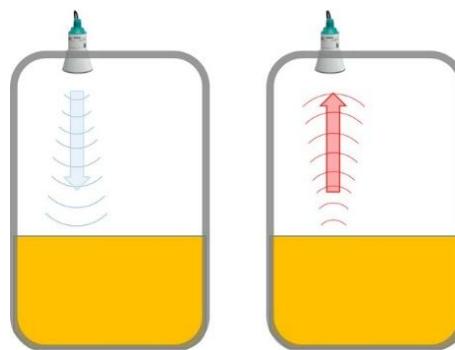


Figura 5.14. Aplicación 2 con sensor ultrasónico

Medición de distancias usando el sensor de profundidad Orbbec-Astra

Una cámara Orbbec Astra es un sensor de visión 3D y tecnología de luz estructurada que mide profundidad, imágenes a color y datos infrarrojos en tiempo real, dentro de sus características principales, podemos contar con:

- *Captura RGB-D*: Combina una cámara de profundidad con una lente a color tradicional.
- *Rango de trabajo*: Detecta objetos y espacios desde pocos centímetros hasta los 8 metros, según la versión.
- *Uso común*: Ideal para robótica, navegación autónoma, reconocimiento de gestos, escaneo tridimensional y control por computadora.
-



Figura 5.15. Sensor de profundidad Orbbec Astra series

Para comenzar a trabajar con la cámara de profundidad, siga los siguientes pasos:

8. Verifique que se hayan instalado los controladores *astra-win32-driver-4.3.0.22* (o superior) en su computador, estos *drivers* permitirán la identificación y operación del sensor en su sistema operativo. Para descargar e instalar los archivos en su PC, visite el siguiente enlace:

<https://www.orbbec.com/developers/openni-sdk/>

Allí encontrará *OpenNI SDK*- Kit de Desarrollo de Software (se deben descargar también), el cual es un paquete de herramientas que ayuda a los programadores a crear aplicaciones. Incluye elementos como bibliotecas de código, documentación y compiladores.

En el mismo enlace también está disponible “*Orbbec Camera Driver for Windows*”.

9. Conecte al puerto USB de su computador la cámara Orbbec Astra y verifique en el *administrador de dispositivos* que ha sido reconocida por su sistema operativo (ver figura 5.16).

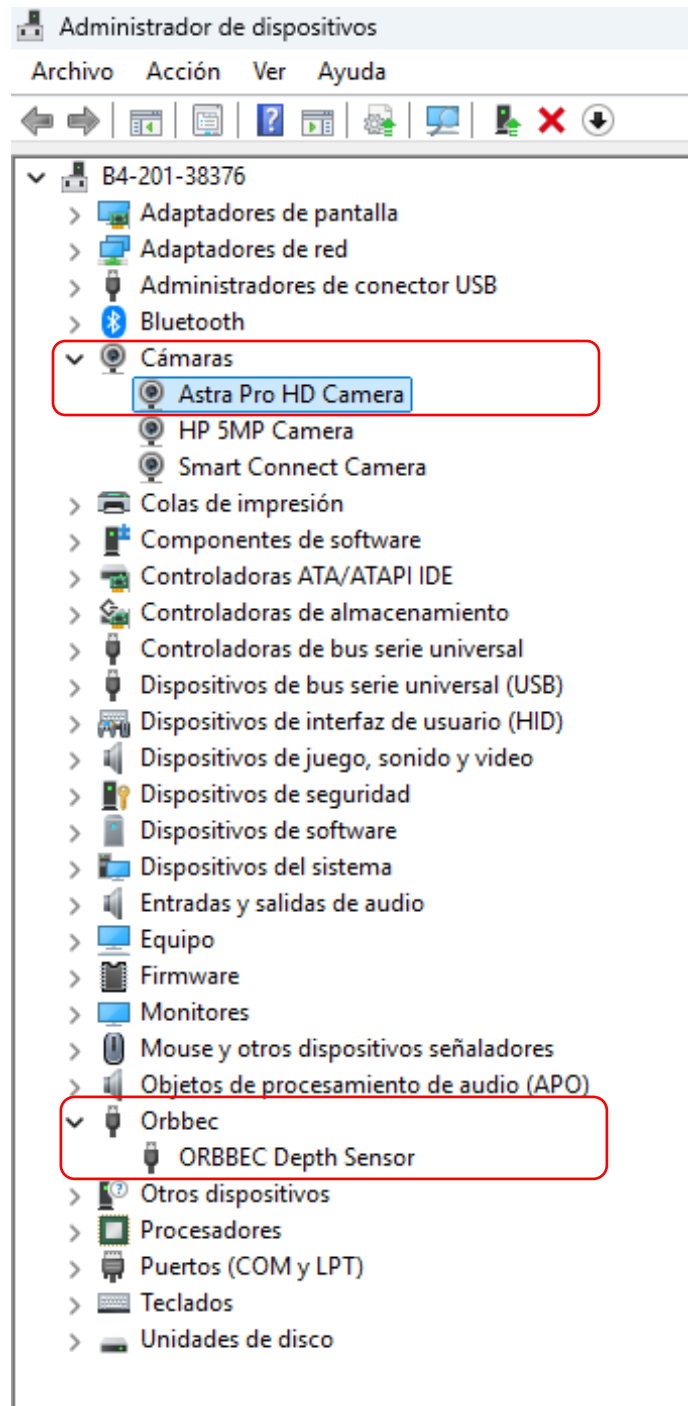


Figura 5.16. Administrador de dispositivos

10. Instale en su equipo **Anaconda**, el cual es una plataforma de distribución libre y abierta de los lenguajes **Python** y **R**, diseñada para la **ciencia de datos**, el **aprendizaje automático** y la computación científica. Esta Plataforma Incluye

herramientas clave como **Jupyter Notebook**, **Spyder** y el gestor de paquetes **Conda**. En el siguiente enlace podrá descargar el software:
<https://www.anaconda.com/download/success?reg=auth>

11. Ejecute el software Anaconda y abra la aplicación *Spyder*

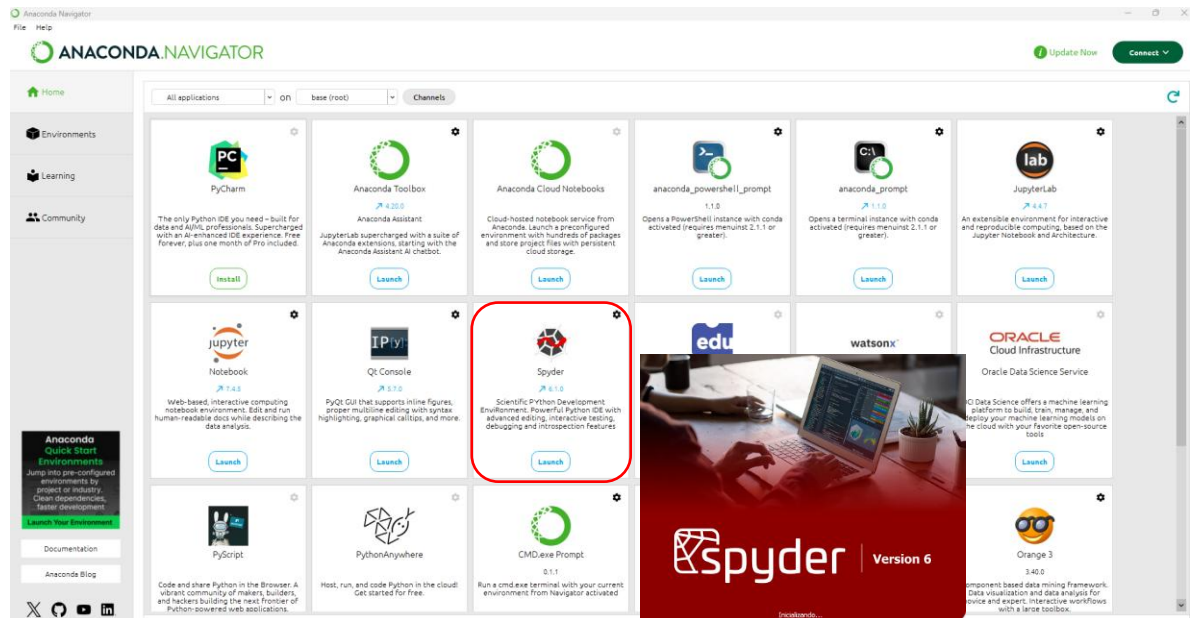


Figura 5.17. Interfaz de usuario del software Anaconda

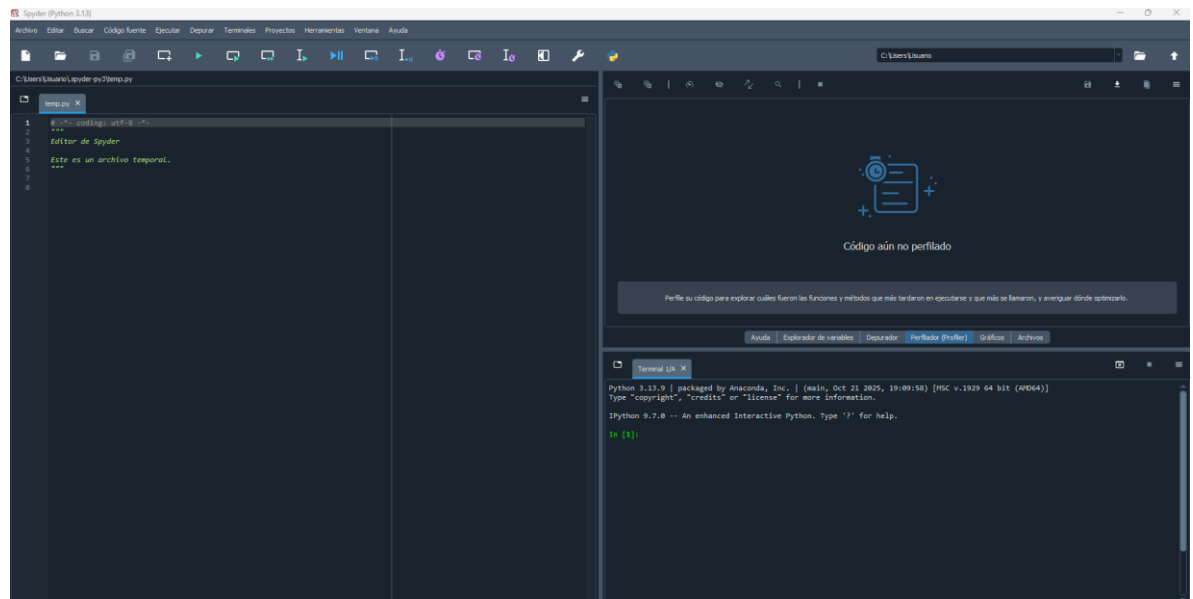


Figura 5.17. Interfaz de usuario del software Spyder

12. Si su equipo de cómputo no tiene preinstalados paquetes de Visual Studio 2013, se hace necesario descargar e instalar los paquetes redistribuibles de Visual C++, los cuales instalan componentes en tiempo de ejecución necesarios para ejecutar aplicaciones desarrolladas usando Visual Studio 2013 y reconocer la cámara Astra.

Para instalar los paquetes redistribuibles de Visual C++ para Visual Studio 2013 compatibles con Spyder 6 y Python 3.13, visite el siguiente enlace:

<https://www.microsoft.com/es-co/download/details.aspx?id=40784>

(Opcional) Conectar la cámara Astra por USB y verificar su correcto funcionamiento ejecutando el programa **NiViewer.exe** ubicado en los archivos de desarrollo de Orbbec. Si **NiViewer.exe** muestra la imagen de profundidad, el sistema operativo está listo.

13. **Instalación de Librerías en Python:** se deben instalar los paquetes necesarios para el procesamiento de visión por computadora y comunicación con OpenNI2.

- Abrir la terminal de Anaconda (Anaconda Prompt) o la consola de Spyder.
- Ejecutar el siguiente comando para instalar OpenCV, NumPy y el wrapper PrimeSense:

```
pip install opencv-python numpy openni primesense
```

La librería `primesense` funciona en Python 3.13 porque no requiere compilación previa (es un wrapper que llama directamente a la DLL `OpenNI2.dll` de 64 bits), pero si al ejecutar el comando `pip install` alguna librería secundaria da un error de compilación por incompatibilidad con Python 3.13. En ese caso, la solución más rápida dentro de Anaconda es crear un entorno en Python 3.10 con:

```
conda create -n vision310 python=3.10 spyder -y
conda activate vision310
pip install opencv-python numpy openni primesense
```

14. Copie el código `.py` suministrado (ver Anexo) para esta práctica y péguelo en el espacio de trabajo de Spyder (ver figura 5.18), se **debe modificar obligatoriamente** en el script de Python la variable de la ruta inicial:

```
OPENNI_REDIST_PATH = r"RUTA_DONDE_EL_ESTUDIANTE_DESCARGO_NIVIEWER"
```

Esta ruta debe apuntar a la carpeta del SDK de *Orbbec* donde se encuentran el archivo *OpenNI2.dll* y la carpeta *OpenNI2/Drivers*.

- Secuencia Correcta de Ejecución:
 - Ejecutar el script en Spyder en primer lugar.

En la consola de Spyder aparecerá el mensaje:

¡ÉXITO: OpenNI2 inicializado correctamente!

[TCP Servidor] Escuchando en 127.0.0.1:5005...

ESPERANDO CONEXIÓN CON LABVIEW... <<<

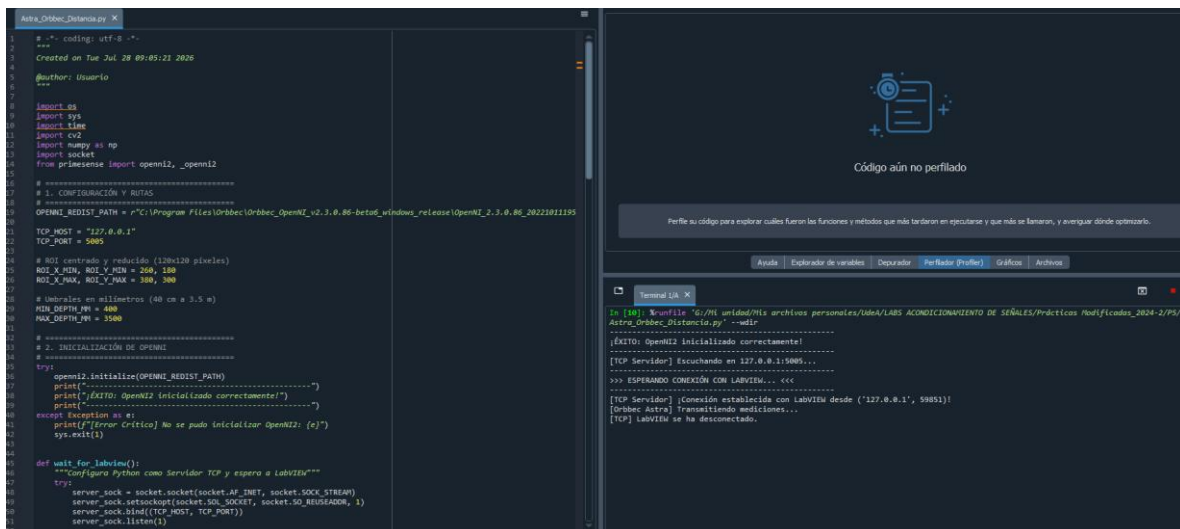


Figura 5.18. Consola de *Spyder* ejecutando el script de *Python*, *Esperando conexión con LabVIEW*

- Luego, ejecutar el .vi diseñado en LabVIEW

El bloque *TCP Open Connection.vi* se conectará automáticamente a Python en el puerto 5005.

- ¡Listo! Se desplegará la ventana gráfica de OpenCV y LabVIEW comenzará a recibir la distancia en tiempo real.

15. El .vi diseñado previamente en *LabVIEW* debe corresponder a una aplicación que reciba una trama *TCP* bajo el siguiente formato:

<Distancia_med,Distancia_max, Distancia_min>\n

La trama debe ser desconcatenada para mostrar individualmente las variables recibidas. La dirección IP debe ser: *127.0.1* y el puerto: *5005*. En la figura 5.19 se muestra un ejemplo de la interfaz requerida.



Figura 5.19. Interfaz de usuario de la aplicación desarrollada en LabVIEW para medir distancia con la cámara ASTRA e Imagen de profundidad del sensor de profundidad

En la figura 5.20 se muestra el código desarrollado en LabVIEW para recibir los datos enviados por la cámara ASTRA empleando el protocolo TCP.

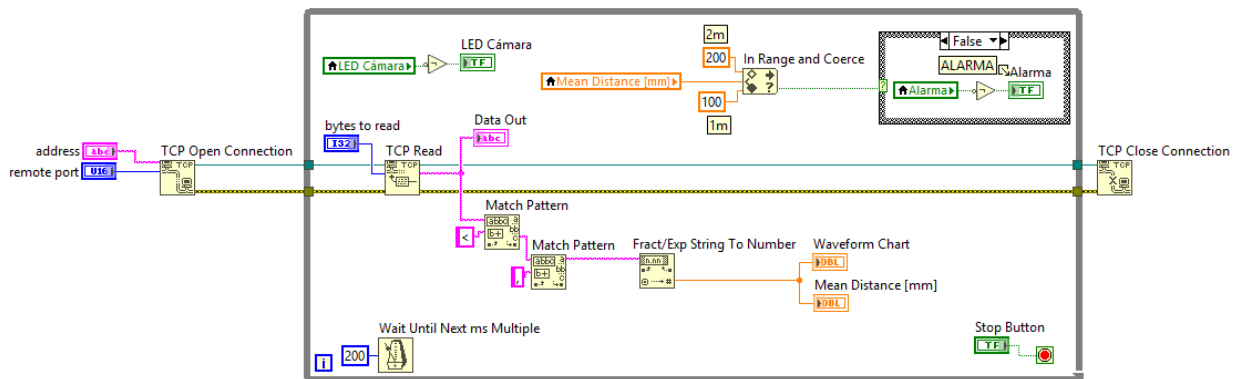


Figura 5.20. Código gráfico desarrollado en LabVIEW

16. Partiendo como base del código suministrado en el *Anexo* de esta práctica (el cual ya es funcional), modifique el algoritmo (con ayuda de la *IA*) para capturar tanto las medidas de la Distancia de un objeto a la cámara (eje *Z*), como las medidas en *cm* del desplazamiento de una mano en los ejes “*X*” y “*Y*”. Es decir, en la interfaz de *OpenCV* (ventana con la imagen de profundidad de la Figura 5.19) se debe visualizar la distancia medida en los tres ejes (*X*, *Y* y *Z*). Por otro lado, en la interfaz de LabVIEW, se deben presentar estas distancias y además encender/apagar *LEDs* indicadores cada vez que las distancias superen cierto umbral preestablecido (tipo alarma de proximidad).

Anexo

Código en python

```
=====
=
PRACTICA DE LABORATORIO: SISTEMA DE MEDICION DE DISTANCIA 3D
DISPOSITIVO: Orbbec Astra (OpenNI2)
LENGUAJE: Python 3.x + OpenCV + Socket TCP + NumPy

DESCRIPCIÓN:
Este programa captura el mapa de profundidad generado por una cámara de luz
estructurada Orbbec Astra, procesa la matriz de distancias dentro de una
Región de Interés (ROI) específica y transmite los datos procesados en tiempo
real a un servidor/cliente en LabVIEW mediante sockets TCP/IP.
=====
=
"""

import os
import sys
import time
import cv2
import numpy as np
import socket
from primesense import openni2, _openni2

# =====
# 1. PARAMETROS Y CONFIGURACIÓN GLOBAL
# =====

# Ruta absoluta donde residen los controladores DLL de OpenNI2 y Orbbec (Win64)
# (Si cambias de computadora, debes verificar que esta ruta apunte a NiViewer)
OPENNI_REDIST_PATH = r"C:\Program Files\Orbbec\Orbbec_OpenNI_v2.3.0.86-
beta6_windows_release\OpenNI_2.3.0.86_202210111950_4c8f5aa4_beta6_windows\Win64-
Release\tools\NiViewer"

# Configuración de Red TCP/IP (comunicación local con LabVIEW)
TCP_HOST = "127.0.0.1" # IP Local (localhost)
TCP_PORT = 5005 # Puerto de escucha (debe coincidir con LabVIEW)

# -----
# CONFIGURACIÓN DE LA REGIÓN DE INTERÉS (ROI)
# Resolución total del sensor: 640 x 480 píxeles.
#
# Para modificar la ventana de medición, ajusta estos límites pixel a pixel:
# ROI_X_MIN / ROI_X_MAX -> Controlan el ancho (Horizontal)
# ROI_Y_MIN / ROI_Y_MAX -> Controlan el alto (Vertical)
# -----
ROI_X_MIN, ROI_Y_MIN = 260, 180 # Esquina superior izquierda de la ROI
ROI_X_MAX, ROI_Y_MAX = 380, 300 # Esquina inferior derecha (Ventana de 120x120px)

# -----
```

```
# UMBRALES DE VALIDACIÓN DEL SENSOR (en milímetros)
# El sensor Astra mide con precisión entre 0.4 metros (400 mm) y 3.5 metros (3500 mm).
# Valores fuera de este rango se consideran 'Ruido' o 'Puntos no reflejados'.
# -----
MIN_DEPTH_MM = 400
MAX_DEPTH_MM = 3500
```

```
# =====
# 2. INICIALIZACIÓN DE LA LIBRERÍA OPENNI2
# =====
try:
    # Carga las bibliotecas dinámicas de C++ (.dll) para comunicarse con el hardware
    openni2.initialize(OPENNI_REDIST_PATH)
    print("-----")
    print("[OpenNI2] Modulo inicializado correctamente.")
    print("-----")
except Exception as e:
    print(f"[Error Critico] No se pudo cargar OpenNI2. Revisa la ruta de las DLL: {e}")
    sys.exit(1)
```

```
# =====
# 3. FUNCIONES DE SERVIDOR TCP / IP
# =====
def wait_for_labview():
    """
    Configura Python como Servidor TCP y detiene la ejecución hasta que
    el cliente en LabVIEW (TCP Open Connection.vi) establezca la conexión.
    """
    try:
        # Crear Socket en la familia IPv4 (AF_INET) y protocolo TCP (SOCK_STREAM)
        server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Permitir la reutilización inmediata del puerto tras reiniciar el programa
        server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

        # Vincular el socket a la IP y Puerto especificados
        server_sock.bind((TCP_HOST, TCP_PORT))

        # Poner en modo de escucha (máximo 1 conexión entrante)
        server_sock.listen(1)

        print(f"[TCP Servidor] Escuchando en {TCP_HOST}:{TCP_PORT}...")
        print("-----")
        print(">>> ESPERANDO CONEXIÓN CON LABVIEW... <<<")
        print("  (Inicia el programa .vi en LabVIEW ahora)")
        print("-----")

        # Asignar un tiempo máximo de espera (Timeout de 30 segundos)
        server_sock.settimeout(30.0)

        # Bloquear el programa hasta que LabVIEW responda
        client_sock, addr = server_sock.accept()

        print(f"[TCP Servidor] ;Cliente conectado exitosamente desde {addr}!")
```

```

# Quitar el tiempo límite de espera para permitir transmisión continua
client_sock.settimeout(None)

return server_sock, client_sock

except socket.timeout:
    print("\n[TCP Warning] Se agoto el tiempo de espera (30s) sin respuesta de LabVIEW.")
    print("[TCP Warning] El programa continuara ejecutandose solo en modo OpenCV.")
    return None, None

except Exception as e:
    print(f"[TCP Error] Falla en la inicializacion del servidor: {e}")
    return None, None

# =====
# 4. ALGORITMO PRINCIPAL Y PROCESAMIENTO
# =====
def main():
    # Paso 1: Iniciar el socket y aguardar por LabVIEW
    server_sock, client_sock = wait_for_labview()

    # Paso 2: Conectar con el sensor físico Orbbec Astra
    try:
        dev = openni2.Device.open_any() # Abre la primera cámara conectada por USB
        depth_stream = dev.create_depth_stream() # Crea el canal de profundidad

        # Configurar la resolución y velocidad de captura del flujo de video
        depth_stream.set_video_mode(
            _openni2.OniVideoMode(
                pixelFormat=_openni2.OniPixelFormat.ONI_PIXEL_FORMAT_DEPTH_1_MM, # 1 entero = 1
mm                resolutionX=640,
                resolutionY=480,
                fps=30
            )
        )
        depth_stream.start() # Iniciar la emisión de infrarrojos
        print("[Orbbec Astra] Flujo de profundidad activado (30 FPS).")

        # Ventana de visualización de OpenCV
        cv2.namedWindow("Orbbec Astra - Profundidad ROI", cv2.WINDOW_NORMAL)
        cv2.resizeWindow("Orbbec Astra - Profundidad ROI", 640, 480)

    except Exception as e:
        print(f"[Hardware Error] No se pudo abrir la camara Astra: {e}")
        if client_sock: client_sock.close()
        if server_sock: server_sock.close()
        return

    # Paso 3: Bucle de captura y procesamiento en tiempo real
    try:
        while True:
            # A) Captura del buffer de datos crudos (Raw Data)
            frame = depth_stream.read_frame()

```

```

frame_data = frame.get_buffer_as_uint16() # Convertir puntero C a formato uint16

# Reorganizar el arreglo unidimensional en una Matriz 2D de [480 filas x 640 columnas]
depth_map = np.frombuffer(frame_data, dtype=np.uint16).reshape((480, 640))

# B) Extracción de la sub-matriz correspondiente a la ROI
depth_roi = depth_map[ROI_Y_MIN:ROI_Y_MAX, ROI_X_MIN:ROI_X_MAX]

# C) Filtrado de ruido (Creación de Máscara Booleana)
# Solo conserva píxeles cuya distancia esté entre 400mm y 3500mm
valid_mask = (depth_roi >= MIN_DEPTH_MM) & (depth_roi <= MAX_DEPTH_MM)
valid_depths = depth_roi[valid_mask] # Matriz resultante recortada

# D) Procesamiento Estadístico de Mediciones
if valid_depths.size > 0:
    # Convertir de milímetros a centímetros (dividiendo entre 10.0)
    dist_min_cm = np.min(valid_depths) / 10.0
    dist_max_cm = np.max(valid_depths) / 10.0

    # Usamos la MEDIANA estadística en lugar del promedio.
    # La mediana es inmune a valores atípicos/bordes ruidosos.
    dist_median_cm = np.median(valid_depths) / 10.0

    # Formato del mensaje para LabVIEW: "<Mediana,Max,Min>\n"
    # El caracter '\n' (salto de línea) le indica a LabVIEW el fin de trama.
    payload = f"<{dist_median_cm:.2f},{dist_max_cm:.2f},{dist_min_cm:.2f}>\n"

    # Envío de la trama TCP si el socket está activo
    if client_sock:
        try:
            client_sock.sendall(payload.encode('utf-8'))
        except Exception:
            print("[TCP Warning] LabVIEW se desconecta.")
            client_sock = None
    else:
        # Caso extremo: Objeto fuera de rango o sensor bloqueado
        dist_min_cm = dist_max_cm = dist_median_cm = 0.0

# E) Generación del Mapa de Calor (Pseudo-color para vista humana)
# Normalizar los valores de profundidad a una escala de 8 bits (0 a 255)
depth_vis = np.clip(depth_map, MIN_DEPTH_MM, MAX_DEPTH_MM)
depth_vis = cv2.normalize(depth_vis, None, 0, 255, cv2.NORM_MINMAX, dtype=cv2.CV_8U)

# Aplicar la paleta JET (Azul = Cerca, Rojo = Lejos)
depth_colormap = cv2.applyColorMap(depth_vis, cv2.COLORMAP_JET)

# F) Renderizado del Recuadro ROI
cv2.rectangle(depth_colormap, (ROI_X_MIN, ROI_Y_MIN), (ROI_X_MAX, ROI_Y_MAX), (0,
255, 255), 3)

# G) Renderizado de la Interfaz y Texto en Pantalla
# Banner negro superior para mejorar la legibilidad del texto
cv2.rectangle(depth_colormap, (0, 0), (640, 75), (0, 0, 0), -1)

# Imprimir valor de la Mediana (Texto destacado)
text_principal = f"DISTANCIA: {dist_median_cm:.1f} cm"

```

```

cv2.putText(depth_colormap, text_principal, (15, 38),
            cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0, 255, 255), 3, cv2.LINE_AA)

# Imprimir métricas secundarias (Min y Max)
text_secundario = f"Min: {dist_min_cm:.1f}cm | Max: {dist_max_cm:.1f}cm"
cv2.putText(depth_colormap, text_secundario, (15, 65),
            cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 1, cv2.LINE_AA)

# Mostrar cuadro resultante
cv2.imshow("Orbbec Astra - Profundidad ROI", depth_colormap)

# H) Control de salida (Esperar 30ms / Salir con 'q' o ESC)
key = cv2.waitKey(30) & 0xFF
if key in (ord('q'), 27):
    break

except Exception as err:
    print(f"[Loop Error] Excepcion inesperada en el bucle principal: {err}")

# Paso 4: Liberación de recursos y apagado seguro de componentes
finally:
    print("\n[Info] Apagando sensor y liberando sockets de red...")
    if client_sock:
        client_sock.close()
    if server_sock:
        server_sock.close()
    try:
        depth_stream.stop()
        openni2.unload()
    except:
        pass
    cv2.destroyAllWindows()
    print("[Info] Programa finalizado correctamente.")

if __name__ == "__main__":
    main()

```

Conclusiones

- Escriba las conclusiones más relevantes de la práctica de laboratorio, indicando hallazgos, dificultades y logros, comparando los procedimientos desarrollados en las dos partes de esta práctica.

Práctica No.6

Sensado y control de velocidad de un motor DC y control de marcha y sentido de giro de un motor Brushless

Objetivos:

- Implementar una aplicación que mida la velocidad de un motor DC usando un codificador angular (encoder) empleando dos técnicas diferentes: conteo de pulsos (frecuencia) y medición de voltaje, dicha velocidad debe ser visualizada en un entorno gráfico desarrollado en LabVIEW.
- Implementar una aplicación que controle manualmente la velocidad de un motor DC mediante hardware (potenciómetro) y software (instrumento virtual), usando la técnica de modulación de ancho de pulso *PWM*. El control por software se debe realizar mediante un entorno gráfico desarrollado en LabVIEW.
- Implementar una aplicación en LabVIEW que controle la marcha (porcentaje de velocidad) y el sentido de giro de un motor sin escobillas tipo *Brushless*.

Materiales:

- PC con software LabVIEW Instalado.
- Driver de comunicación NI-VISA instalado
- Software Arduino IDE.
- Librerías LINX-MakerHub instaladas (Hobbyist).
- Drivers DAQmx instalados
- Placa Arduino UNO o MEGA-cable de comunicación USB.
- Tarjeta NI-USB-6009
- 1 motor DC de 5V
- 1 motor sin escobillas tipo Brushless
- Driver de control para el motor sin escobillas 57BLS01
- 1 Foto interruptor ITR9606 o H21A1 en herradura
- 1 Rueda encoder
- 1 CI 74LS14 comparador Schmitt-trigger
- 1 CI LM2907 conversor de frecuencia a voltaje
- 1 CI MAX232
- 1 potenciómetro de 50K
- 1 transistor NPN 2N2222 o 2N3904
- 1 diodo LED
- Board, Fuente DC variable
- Resistencias, potenciómetros, capacitores varios, cables y caimanos

Estudio Previo

1. Consultar los principales métodos para medición de velocidad de un motor DC, listar los tipos de sensores usados, e identificar algunas de las principales similitudes y diferencias encontradas por las diferentes técnicas.
2. Adjuntar algunas imágenes de los tipos de sensores y equipos usados para medición de velocidad en motores.
3. Si se utiliza un encoder de N huecos para medir la velocidad de un motor [rpm], qué variable es necesaria medir antes de establecer dicha velocidad?. Relacione su respuesta mediante una ecuación matemática.
4. Responder: ¿Qué es el *duty cycle* o ciclo de dureza de una señal? ¿En qué unidades se expresa esta magnitud?
5. Consultar en qué consiste la técnica de modulación de ancho de pulso *PWM* y listar sus principales aplicaciones.
6. Cómo se relaciona matemáticamente el ciclo de dureza (D) de una señal con su periodo (T)?. Realizar una gráfica en el dominio del tiempo donde se ilustren estas variables.
7. Explicar cómo se podría controlar la velocidad de un motor DC y el encendido de un diodo LED usando *PWM* considerando el efecto del valor promedio V_{DC} ó eficaz V_{RMS} .
8. Consultar los *datasheets* del opto-interruptor ITR9606, del CI 74LS14, MAX232 y Transistor 2N2222 o 2N3904 describiendo sus principales características técnicas, usos y aplicaciones.
9. Consultar el *datasheet* del Driver de control para el motor sin escobillas 57BLS01 e identificar su *pinout*.
10. ¿Cómo funciona y cómo se podría controlar un motor tipo Brushless?
11. Realice el montaje en *board* del (os) circuito(s) propuesto(s) y programe los códigos necesarios en software antes de la práctica. Recuerde que el tiempo en el laboratorio es limitado y lo debe optimizar realizando los montajes y programas en un horario extracurricular.
12. Para presentar su preinforme, use el formato (*template*) de la revista científica IEEE dispuesto en el siguiente enlace, con una extensión máxima de 3-4 páginas:
<https://www.ieee.org/conferences/publishing/templates> .

Experimento

Antes de comenzar su laboratorio, asegúrese de tener instalados todos los programas y driver's de comunicación y firmware necesarios:

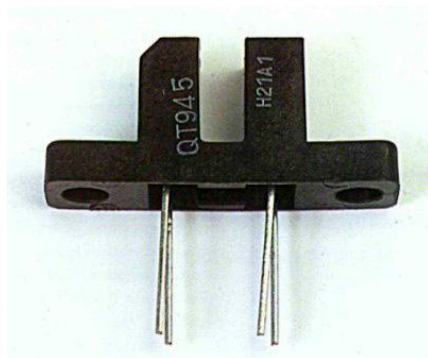
- LabVIEW
- Librerías LINX(Hobbyist)
- NI VISA drivers (Comunicación Serial)
- Arduino IDE
- Drivers DAQmx (para las tarjetas de adquisición de datos)

Esta práctica se desarrollará en dos partes, la primera parte está relacionada con el sensado y el control de velocidad de un motor DC mediante el uso de un encoder óptico y la técnica de modulación de ancho de pulso (*PWM*). La segunda parte se enfoca con el control de movimiento y cambio de sentido de giro de un motor sin escobillas tipo brushless.

Primera parte: Motor DC

Medición de velocidad usando conteo de pulsos (frecuencia)

1. Los sensores de la Figura 1.a y 6.1b, son interruptores ópticos en forma de herradura usados principalmente para medición de la velocidad en motores mediante discos ranurados o *encoders*. En términos generales, este tipo de sensores generan una señal de frecuencia proporcional a la velocidad de rotación del motor. Para realizar la conexión, tenga en cuenta el diagrama de la Figura 6.2 o Figura 6.3.



a)



b)

Figura 6.34. a) Optointerruptor H21A1. b) Optointerruptor ITR9606

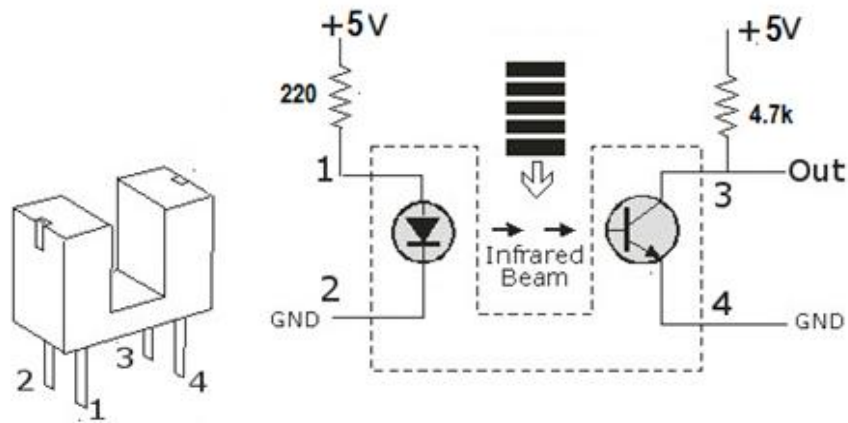


Figura 6.35. Conexiones del fotosensor. Salida por el Colector

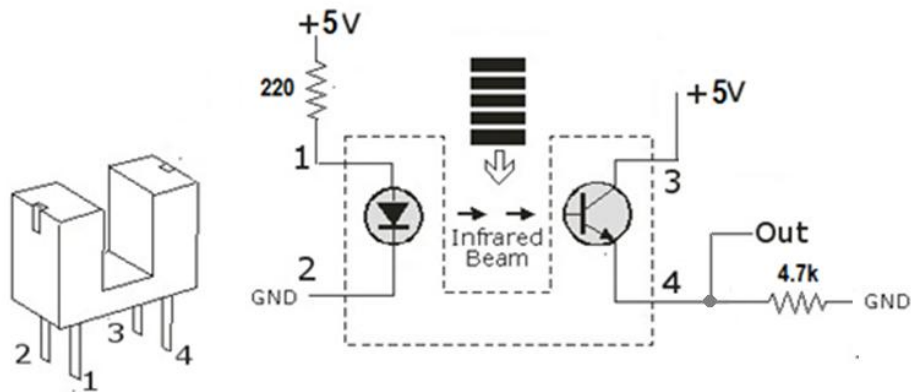


Figura 6.36. Conexiones del fotosensor. Salida por el Emisor

- Adicione el circuito integrado 74LS14 al circuito mostrado en la Figura 6.2 o 6.3, de acuerdo con el diagrama esquemático mostrado en la Figura 6.4a o Figura 6.4b según sea el caso. El objetivo de este circuito es acondicionar los pulsos de voltaje generados por el opto-interruptor a niveles de voltaje entre 0-5 voltios.

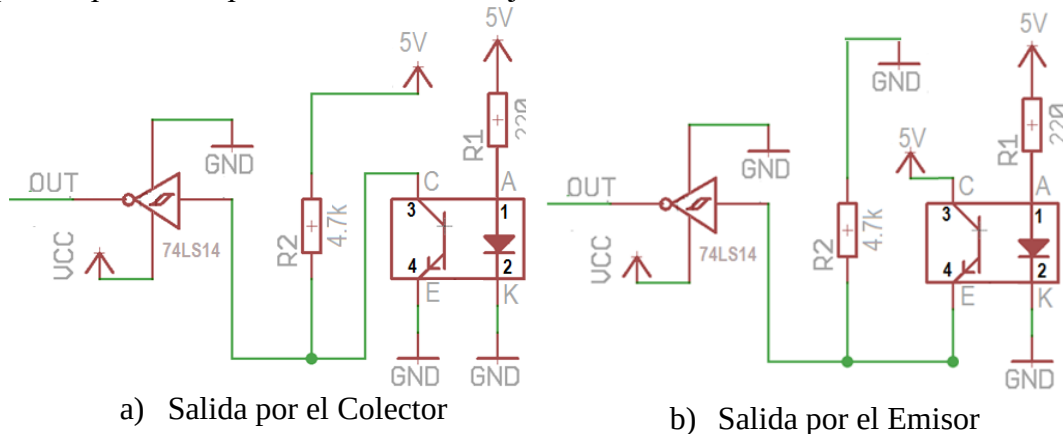


Figura 6.37. Diagrama de Conexión el optointerruptor y el comparador 74LS14

Para probar el anterior esquema circuital, conecte a la salida del circuito (OUT), un diodo LED con su respectivo resistor a tierra (GND). El Diodo LED debe permanecer encendido hasta que un objeto pasante por la “herradura” active su estado de apagado.

Se sugiere visitar el siguiente enlace para ver un video:

<https://www.youtube.com/watch?v=-vAxXwLKzkl>

- Una vez comprobado el correcto funcionamiento del circuito de la Figura 6.4, realice el montaje del codificador angular al eje del motor DC (ver Figura 6.5) y verifique que el diodo LED encienda y apague cada vez que el encoder gire (esto lo puede hacer con el motor apagado, gire el eje de éste con la mano).

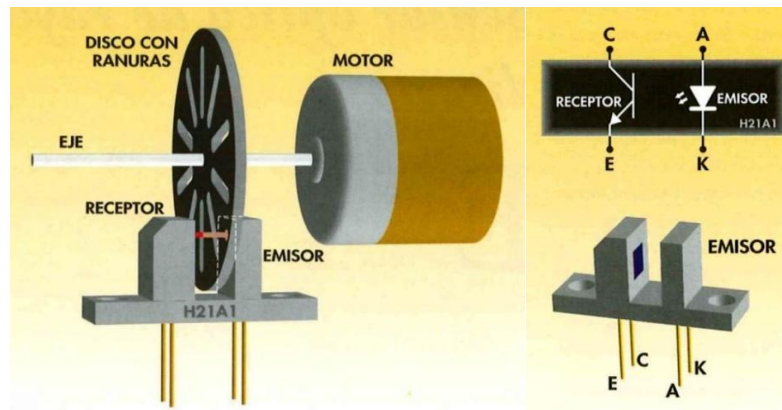


Figura 6.38. Montaje del encoder, motor DC y opto-interruptor

- Una vez esté funcionando correctamente todo el montaje (Figuras 6.4 y 6.5), mida con un multímetro o con un osciloscopio la señal de **frecuencia [Hz]** generada y calcule la velocidad en RPM del motor usando la **ecuación encontrada en el numeral 3 del estudio previo**. Para ello polarice el motor DC con los voltajes indicados en la Tabla 6.1, realizando cinco medidas diferentes por este método.
 ::: Use la fuente variable del laboratorio!!!

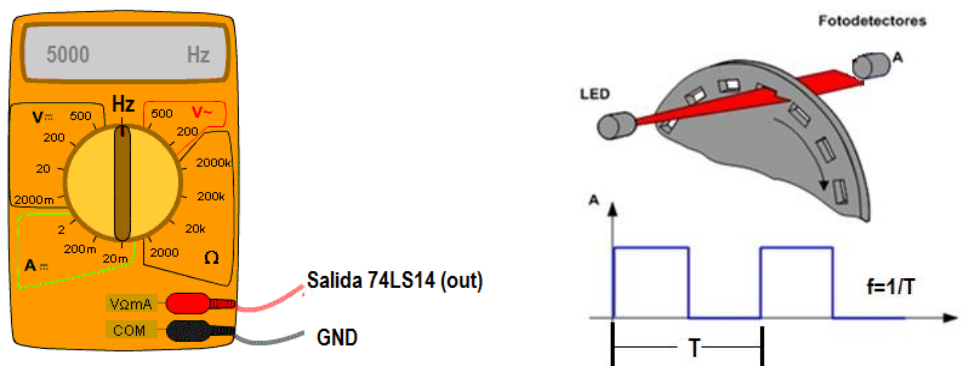


Figura 6.39. Medición de frecuencia para el cálculo de las rpm's del motor

Tabla 6.2. Medición de la velocidad por la técnica de conteo de pulsos (frecuencia)

Voltaje del motor [V]	# pulsos/vuelta	Frecuencia [Hz]	Velocidad [rpm] (calculada)
5			
4			
3			
2			
1			

5. Programe la placa Arduino con el siguiente código, el cual está diseñado para medir la velocidad del motor [rpm] a partir de los pulsos generados por el codificador angular. La medición se podrá observar en el monitor serial de Arduino.

Código

```

////////////////////////////////////
//Programa para el reporte de RPM a partir de la medición de frecuencia//
////////////////////////////////////
#include "TimerOne.h" //libreria para manejar interrupciones temporales
int pinInterrupcion = 0; // uso del pin de interrupción externa 0 = pin digital 2 para medición de RPM
long pulsos = 0; // variable para contar los pulsos del encoder
int resolucion = 4; // resolución del encoder número de pulsos por revolución
double rpm; // variable para almacenar el calculo de la velocidad rpm
unsigned long tiempoAnterior2 = 0; //guarda tiempo de referencia para comparar
long timeSerial=1000; //ms
void setup() {
    Serial.begin(9600); // configuración del serial
    Timer1.initialize(1000000); // cada 1 segundo se ejecuta la interrupción
    Timer1.attachInterrupt(segundo); //se establece función de interrupción
    attachInterrupt(pinInterrupcion, deteccion, RISING); // se programa un interrupción externa, cada que hay un
    cambio de bajo a alto
}
void loop() {
    if((millis()-tiempoAnterior2>=timeSerial)) //si ha transcurrido el periodo programado
    {
        Serial.print("rpm: ");
        Serial.println(rpm); //se imprime la velocidad del motor
        tiempoAnterior2=millis();
    }
}
void deteccion() { // función que se ejecuta cuando hay interrupción externa
    pulsos++; // se incrementa la variable pulso en uno.
}
void segundo () { // función que se ejecuta cuando se cumple el tiempo de interrupción temporal
    rpm=((double)pulsos/resolucion)*60; // calculo de velocidad rpm
    pulsos=0; // pulsos
}

```

- Después de haber programado el microcontrolador, diseñe una aplicación en LabVIEW donde pueda observar en ella la medición de velocidad realizada en los numerales anteriores. Tenga en cuenta las prácticas anteriores de transmisión serial para desconcatenar la trama y visualizar correctamente la variable de interés. En la figura 6.7 se presenta un ejemplo de la interfaz a desarrollar.



Figura 6.40. Ejemplo de la interfaz en LabVIEW para visualizar la velocidad [rpm] del motor

Medición de velocidad usando conversor de frecuencia a voltaje (LM2907 y MAX232)

- Para realizar los siguientes numerales, mantenga el montaje hasta ahora realizado en los puntos anteriores, pero además, adicione el circuito que acondiciona y convierte la señal pulsada de frecuencia en niveles de voltaje. En la Figura 6.8, se muestra un circuito esquemático que incorpora un CI MAX232 y un CI LM2907 los cuales proporcionan un **cruce por cero** de la señal y genera un nivel de voltaje proporcional a la velocidad del motor respectivamente. De acuerdo con el *datasheet* de este conversor, use el valor de los capacitores y resistores indicados conforme a las ecuaciones presentadas en dicho manual.

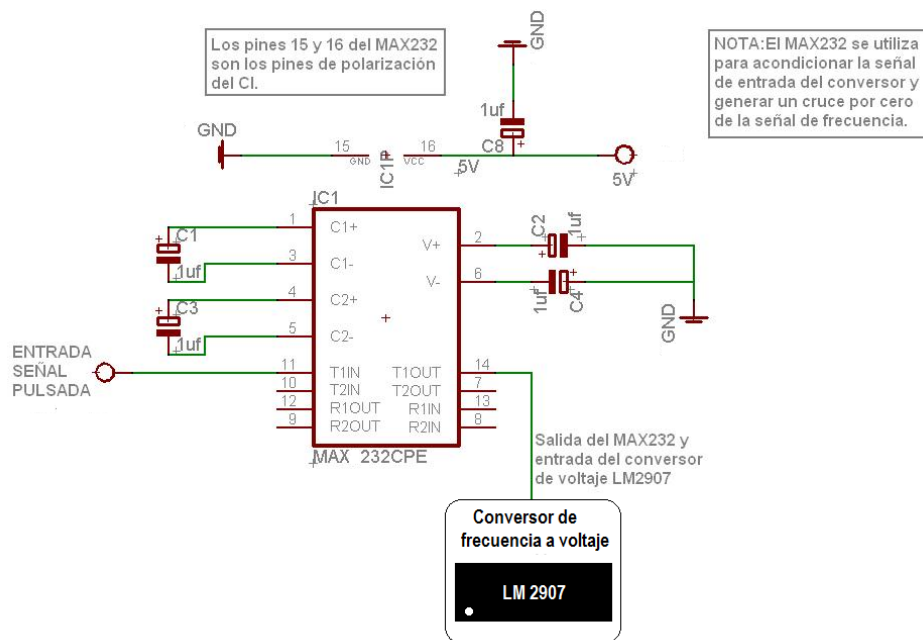


Figura 6.41. Conexión MAX232 y LM2907 para acondicionar la señal pulsada de frecuencia y convertirla en niveles de voltaje

Después de realizar el montaje de la Figura 6.8, conecte al pin A0 de la placa Arduino, la señal de salida del conversor de frecuencia a voltaje. En la Figura 6.9, se muestra el diagrama de conexión completo para el circuito que incorpora la conexión del fotosensor con salida por el Colector. Si se realizó la conexión de salida por el Emisor, tenga en cuenta esto para interpretar el circuito mostrado en la Figura 6.9 y realice las modificaciones respectivas.

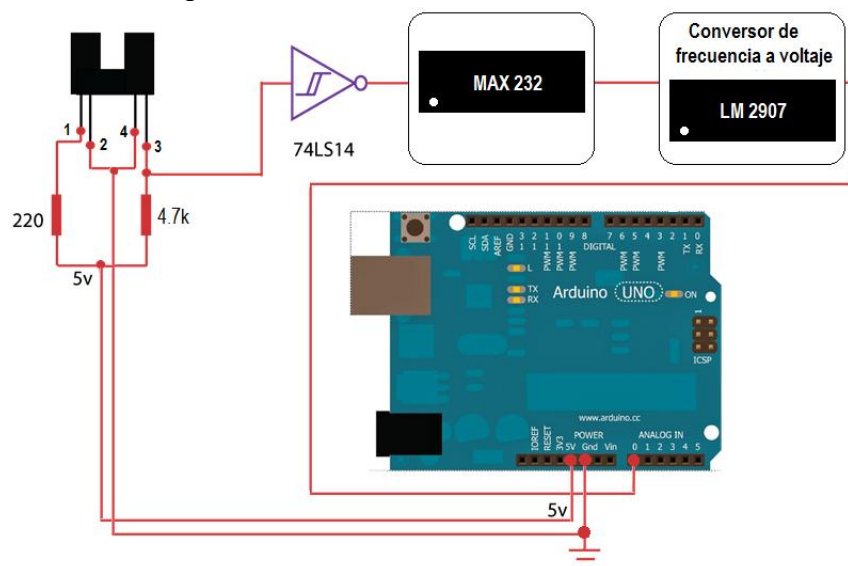


Figura 6.42. Conexión del circuito de acondicionamiento de la señal de frecuencia a la placa Arduino

8. Conecte la Placa *Arduino UNO o MEGA* al PC e identifique el puerto COM donde se ha iniciado la comunicación con el dispositivo.
9. Diseñe una interfaz en LabVIEW que permita comunicarse con la placa *Arduino UNO o MEGA* y se pueda visualizar en ella el voltaje entregado por el LM2907 y la velocidad [rpm] del motor. Asegúrese de elegir correctamente el puerto de comunicación y la velocidad de transmisión de datos (*Baud Rate*). Para visualizar la velocidad del motor en la interfaz, tenga en cuenta la Tabla 6.1, y esta vez relacione la velocidad calculada en [rpm] y el voltaje [V] entregado por el LM2907. Llene la Tabla 6.2.

Tabla 6.3. Relación entre la velocidad calculada [rpm] y el voltaje entregado por el LM2907

Voltaje del motor [V]	Frecuencia [Hz] (de la Tabla 1)	Velocidad [rpm] (de la Tabla 1)	Voltaje a la salida del LM2907 [V]
5			
4			
3			
2			
1			

10. Genere una ecuación que relacione las cinco (5) velocidades calculadas con los cinco (5) voltajes medidos a la salida del convertor de voltaje. Para esto se puede ayudar de una tabla de Excel generando la ecuación de una línea de tendencia lineal de los datos.

$$RPM=f(V)$$

Para realizar la conversión de unidades, es decir, para pasar de voltios a RPM, se podría usar el bloque llamado “*Fórmula*” dispuesto en LabVIEW, el cual se visualiza en la Figura 6.43. La entrada X1 será el voltaje proveniente de la lectura del puerto y entregado por el LM2907, la salida “*Result*” tendrá el resultado de la conversión, es decir, tendrá la velocidad del motor en [rpm]; en el interior del bloque se puede editar la ecuación que representa la característica lineal de voltaje (RPM *vs* voltaje).

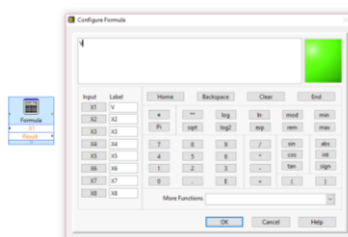


Figura 6.43. Bloque Fórmula de LabVIEW

En la Figura 6.11 y 6.12, se puede ver un ejemplo de una interfaz diseñada en LabVIEW y su respectivo código para realizar la operación descrita anteriormente.



Figura 6.44. Ejemplo de LabVIEW para leer un pin analógico de voltaje y representar la velocidad de un motor en [rpm].

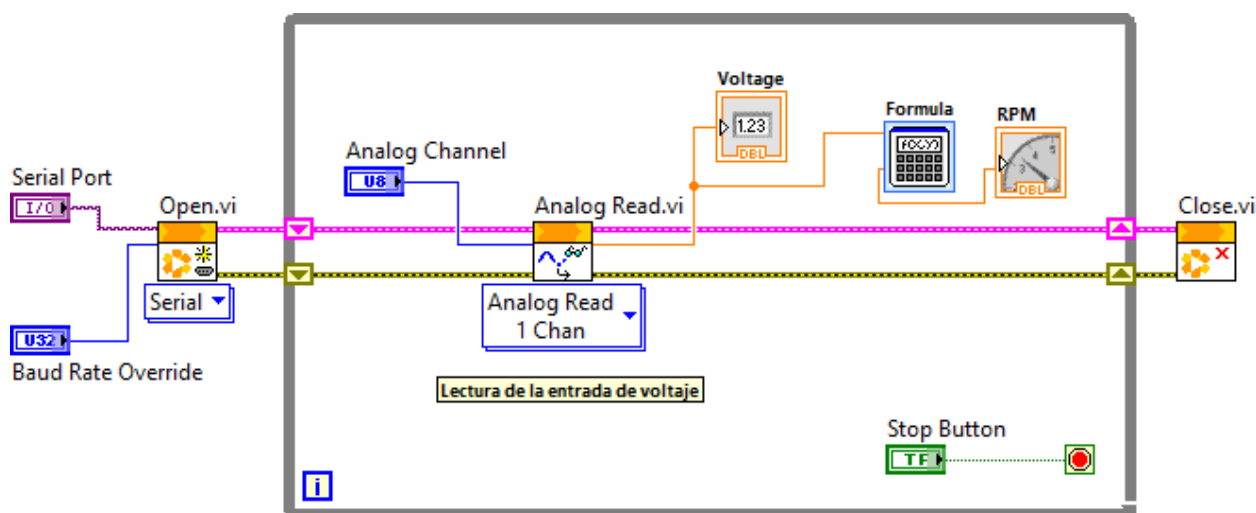


Figura 6.45. Código de la interfaz mostrada en la figura 6.11

11. Otra forma de realizar el sensado de velocidad, es usando en lugar del Arduino®, una tarjeta de adquisición de datos (DAQ). En esta práctica se cuenta con la disponibilidad de la tarjeta NI-USB6009 (ver figura 6.13). Para realizar el sensado de la velocidad del motor mediante la lectura de una señal analógica de voltaje configure la tarjeta siguiendo los pasos presentados en la guía o **práctica No1**.



Figura 6.46. Tarjeta de adquisición de datos NI-USB6009

- Programe en LabVIEW la aplicación mostrada en la figura 6.14.

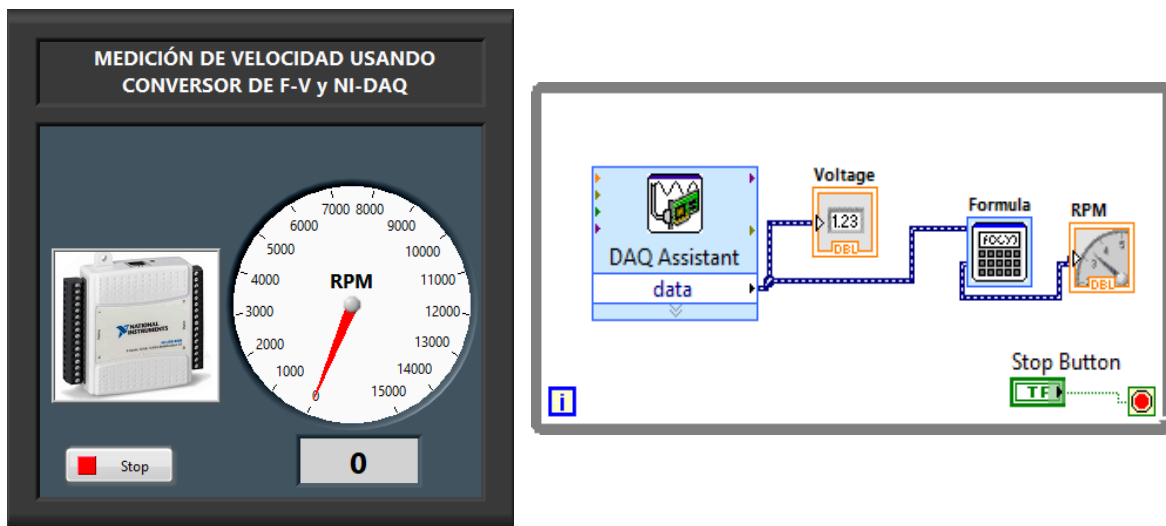


Figura 6.14. Programa para sensar una señal analógica de voltaje con la NI-DAQ

12. Para realizar el sensado de la velocidad del motor mediante el conteo de pulsos y el cálculo de la frecuencia con la DAQ, siga los siguientes pasos:
 - Use el bloque “**Timing and Transition**”, el cual se encuentra en la siguiente ruta: *Programming/Express/Signal Analysis/Timing Transition*

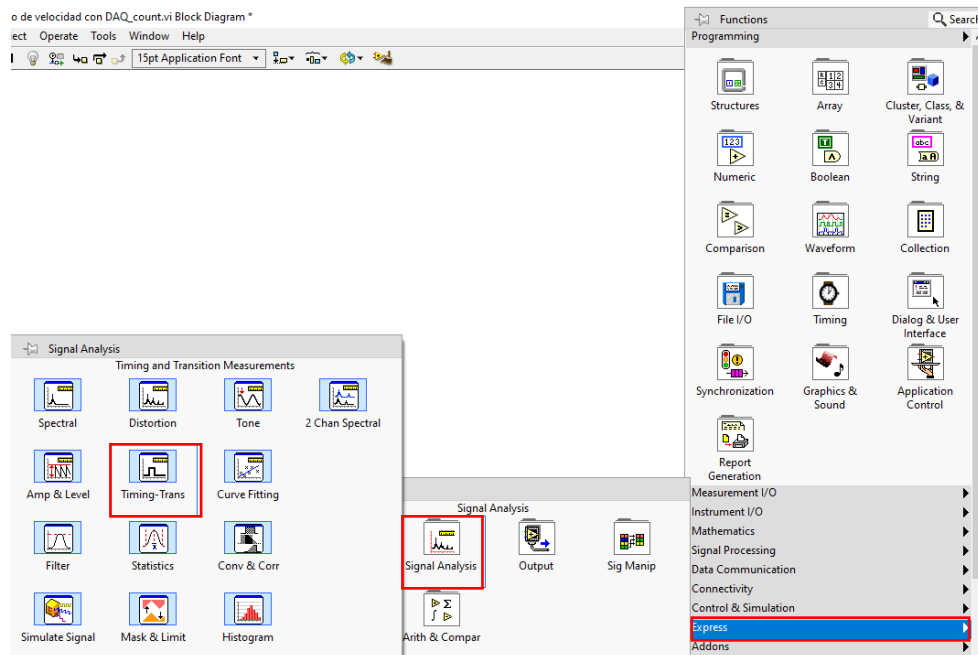


Figura 6.47. Ruta para acceder al bloque Timing Transition

- Haga doble *click* sobre el bloque *Timing Transition* y seleccione la salida de Frecuencia (ver figura 6.16).

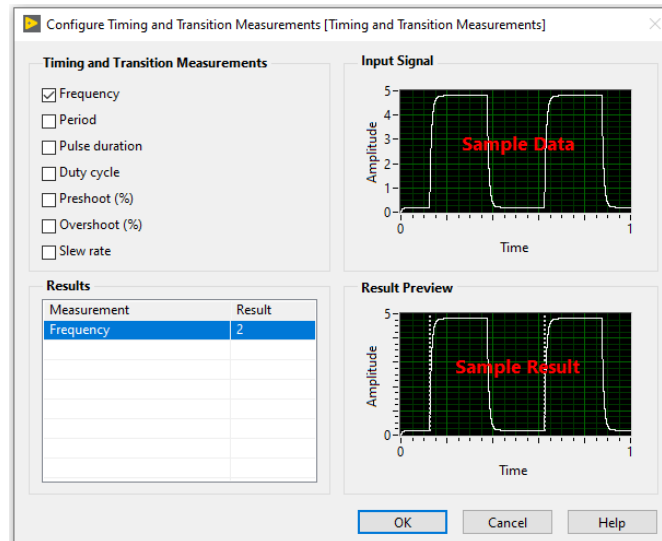


Figura 6.16. Configuración interna del tipo de salida del bloque Timing Transition

- Programe en LabVIEW la aplicación mostrada en la figura 6.17.

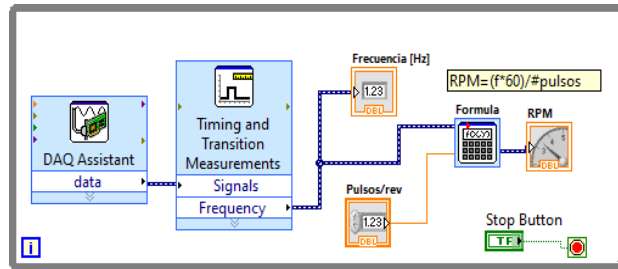


Figura 6.17. Programa para medir la frecuencia de una señal pulsada con cruce por cero con la NI-DAQ



NOTA 1: La técnica de medición de frecuencia (o periodo) con el bloque “Timing Transition” solo es válida cuando la señal pulsada a medir tiene un cruce por cero. Este método falla si la señal está desplazada un Nivel DC. Por esta razón se usó el MAX232.

Si se prefiere, puede usar un capacitor de acople (del orden de uF- preferiblemente “grande”) para eliminar los niveles DC de la señal cuadrada y obtener el cruce por cero necesario sin usar el MAX232.

NOTA 2: En caso de querer construir manualmente la rueda encoder, se debe asegurar que los huecos realizados sean espaciados equitativamente unos de otros para no incurrir en un error en la lectura de la velocidad.

Control en lazo abierto de la velocidad de un motor DC usando modulación de ancho de pulso (PWM)

Antes de desarrollar el control de velocidad del motor DC usando la técnica de modulación de ancho de pulso (PWM) exitosamente, primero aprenderemos a encender controlada y gradualmente un diodo LED usando esta misma técnica. Hay que saber que los pines del *Arduino* que tienen esta función están etiquetados con el símbolo “~”.

En la Figura 6.18a y 6.18b, respectivamente se muestra la placa Arduino UNO con los pines habilitados para realizar *PWM* y una modulación de ancho de pulso para una señal con 5 porcentajes de dureza diferentes. Nótese que el ancho de dureza de la señal está dado por el tiempo en que la señal permanece en “alto” durante un periodo T .

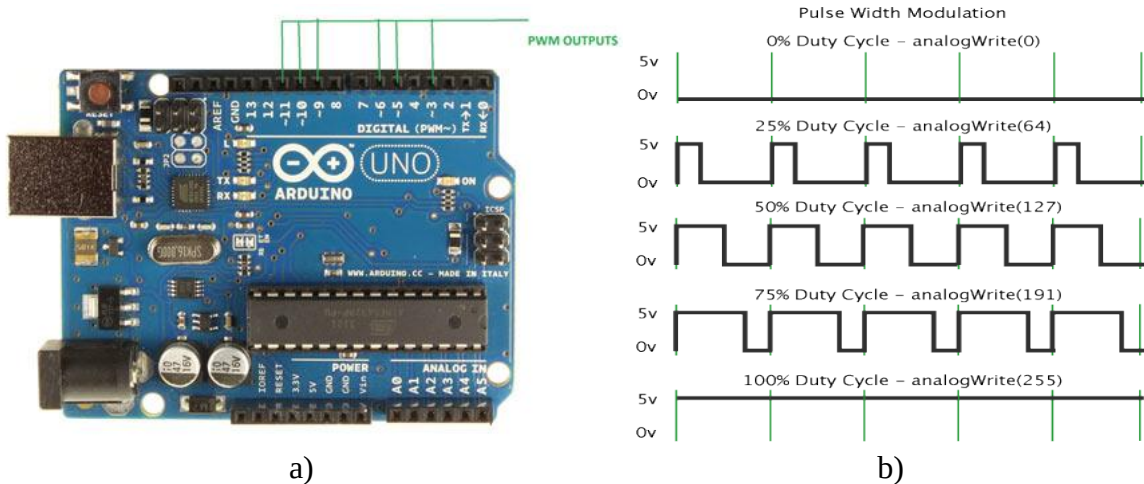


Figura 6.488. a) Pines con función PWM del Arduino. **b)** PWM para 5 anchos de dureza de la señal.

Control de luminosidad de un diodo LED usando PWM

En la Figura 6.19, se muestra un diagrama de conexiones para un LED y un potenciómetro a la salida PWM del pin 3 y la entrada analógica A0 respectivamente.

13. Conecte el circuito tal y como se muestra en la Figura 6.19. La idea con este montaje es encender controlada y gradualmente un diodo LED desde una aplicación gráfica en LabVIEW; pero al mismo tiempo, realizar la misma función manualmente con el potenciómetro.

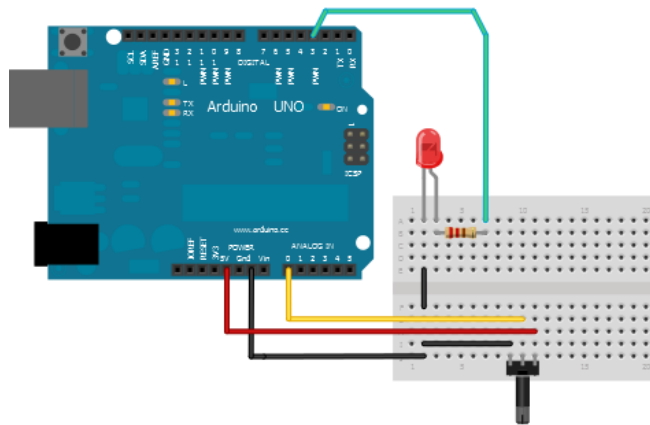


Figura 6.19. Conexiones del diodo LED y el potenciómetro a la salida PWM y a la entrada analógica A0 respectivamente

14. Diseñe una interfaz en LabVIEW que permita comunicarse con la placa *Arduino UNO* o *MEGA* y se pueda controlar y visualizar el ancho de dureza de la señal de salida PWM generado por el pin 3 de la placa desde la interfaz de usuario. Asegúrese de elegir correctamente el puerto de comunicación y la velocidad de transmisión de datos (*Baud Rate*).

La Figura 6.20, muestra un ejemplo de una interfaz desarrollada en LabVIEW para generar una señal PWM por el pin 3 de la placa Arduino.

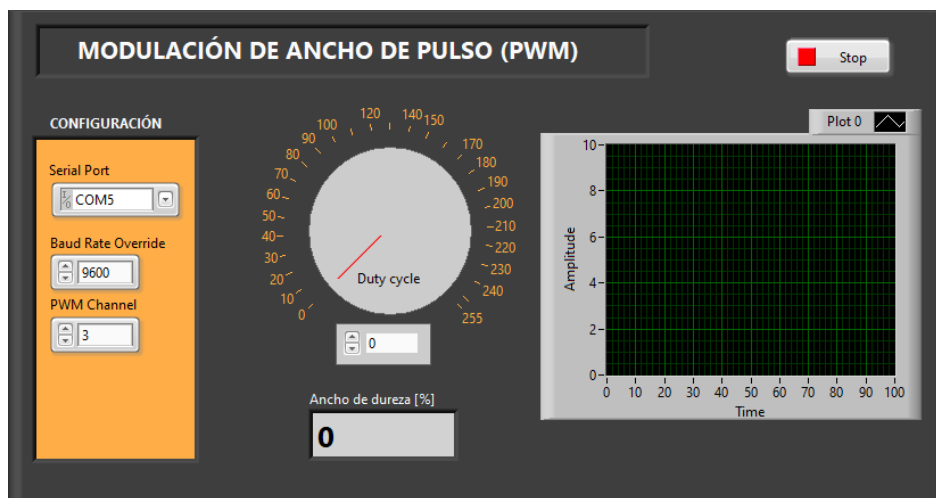


Figura 6.20. Ejemplo de interfaz en LabVIEW para generar una señal PWM

La Figura 6.21, muestra el código gráfico de la interfaz mostrada en la Figura 6.20.

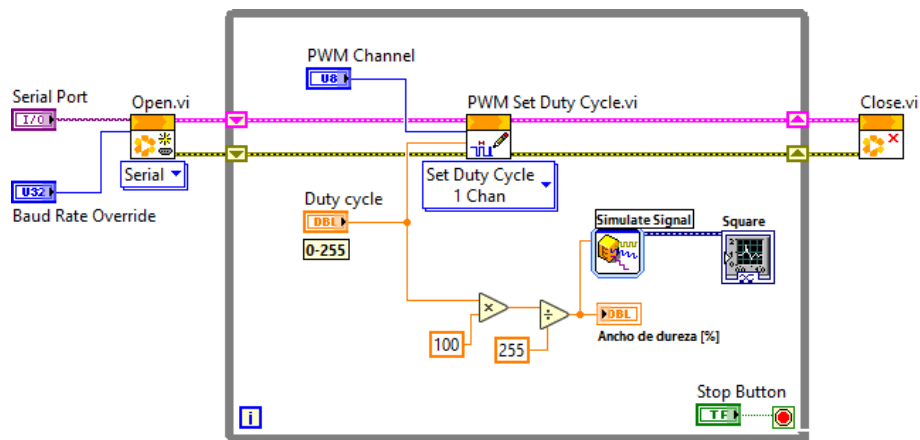


Figura 6.21. Código de la aplicación mostrada en la figura 6.20.

15. Adicione al código mostrado en la Figura 6.21, las estrategias algorítmicas necesarias para controlar manualmente el encendido del LED con el potenciómetro. Para ello tenga en cuenta que la salida del potenciómetro generará un voltaje entre 0V y 5V, el cual debe ingresar por el pin analógico A0. La idea es que el usuario pueda controlar la intensidad lumínica del LED por software o hardware (cualquiera de las dos opciones).

NOTA: Recuerde que será necesario realizar una conversión de unidades de voltaje entre 0V y 5V a datos de PWM entre 0-255.

Control de velocidad de un motor DC usando PWM

16. Una vez comprobado el correcto funcionamiento del circuito de la Figura 6.19, realice el montaje de la Figura 6.22. Reemplace el diodo LED por un motor DC y conéctelo como se indica.

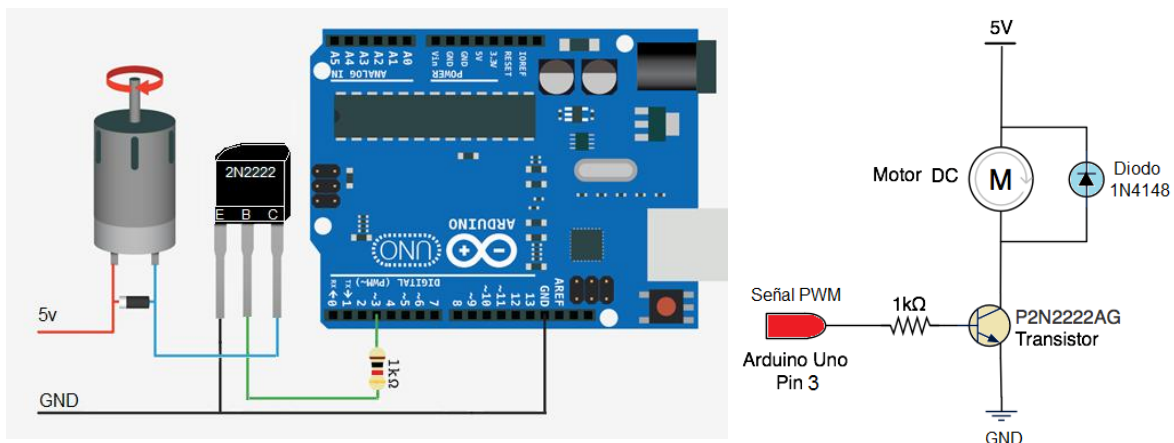


Figura 6.22. Montaje del motor DC para control de velocidad por PWM

17. Adicione al montaje anterior, la conexión del potenciómetro usado previamente tal y como se indica en la Figura 6.23.

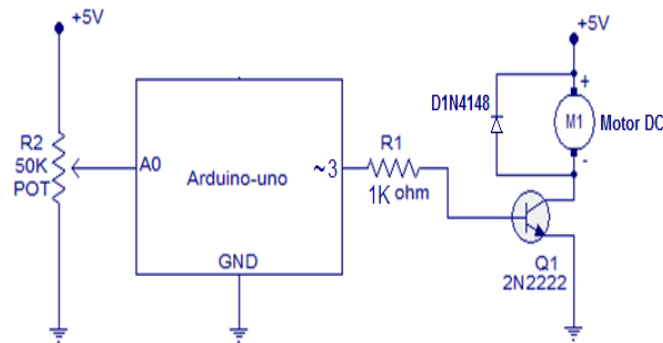


Figura 6.23. Montaje del motor DC y del potenciómetro para control de velocidad por PWM

18. Ejecute nuevamente la aplicación desarrollada en LabVIEW y verifique que la velocidad del motor varía proporcionalmente a la posición de los controles en la interfaz y a la posición del potenciómetro.

Segunda parte: Motor Brushless

Control de movimiento y sentido de giro

- Realice el montaje presentado el esquema de la Figura 6.25. Tenga en cuenta el *pinout* consultado previamente del driver del motor Brushless. En la Figura 6.24 se puede ver un ejemplo del montaje físico de los componentes destinados para esta sección como complemento para la práctica.

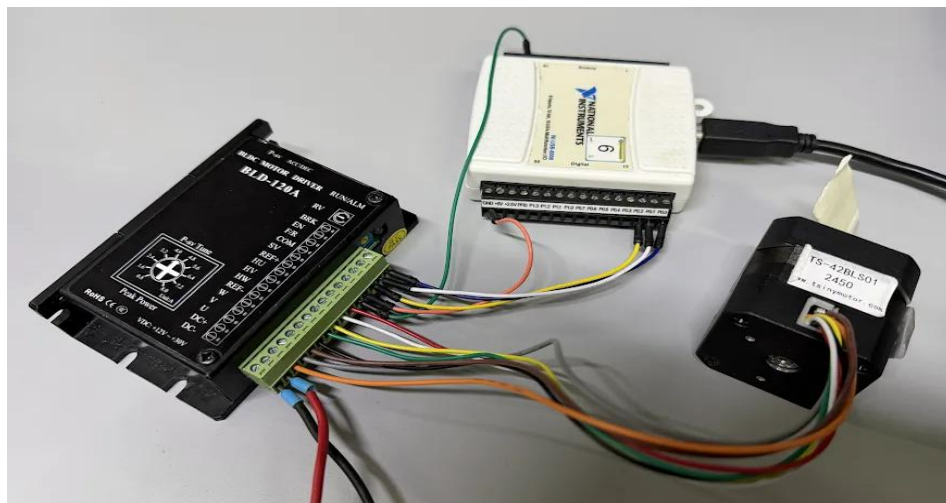


Figura 6.24. Montaje físico de los componentes para la práctica con el motor Brusless

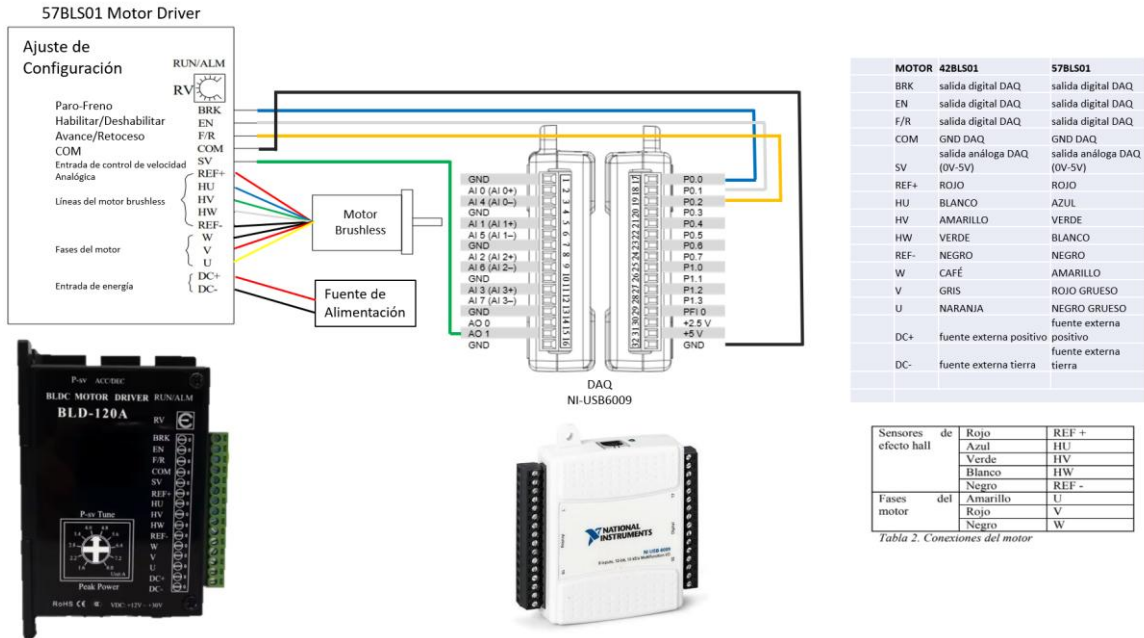


Figura 6.25. Diagrama esquemático de las conexiones con el motor *Brushless*

- Configure un bloque *DAQ Assistant* para trabajar como generador de voltajes analógicos (0-5v) por el puerto *AO1*(*Analogical Output 1*), este puerto estará conectado a la señal *SV* (*entrada de control de velocidad*).
- Configure otro bloque *DAQ Assistant2* para trabajar con sus puertos digitales P0, P01 y P02, estos estarán conectados respectivamente a las señales *BRK*, *EN* y *F/R* (*Sentido de giro*).
- Siguiendo los pasos anteriormente descritos, realice la programación del código gráfico en LabVIEW mostrado en la Figura 6.26. Este código emplea dos estructuras tipo *While loop* simultáneamente. Una estructura se ocupa de la generación de señales mediante la DAQ y la otra atiende el evento de la pulsación del botón *Stop* para terminar la ejecución del programa y desactivar el motor.

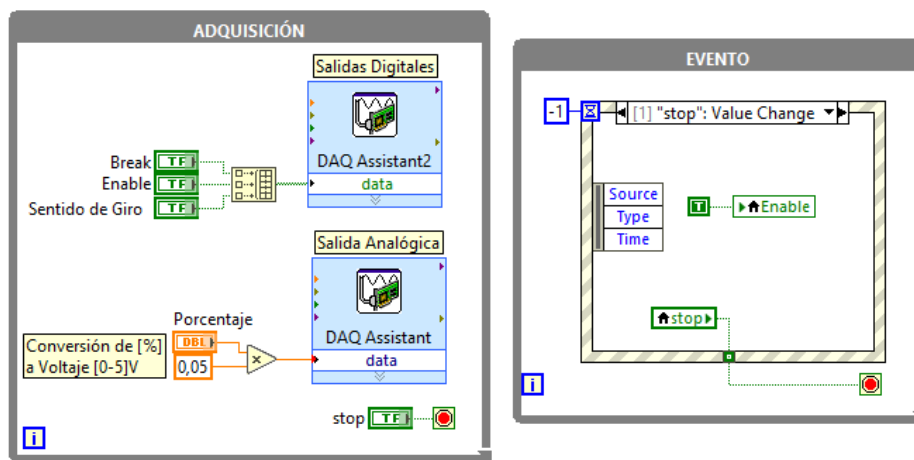


Figura 6.26. Diagrama de bloques para controlar el motor brushless

En la Figura 6.27 se muestra un ejemplo de la interfaz de usuario del código mostrado en la Figura 6.26.



Figura 6.27. Interfaz de usuario para el control del motor *Brushless*

19. Conclusiones

- Escriba las conclusiones más relevantes de la práctica de laboratorio, indicando hallazgos, dificultades y logros.

Práctica No.7

Transmisión inalámbrica de datos con módulos XBEE en modo API

Objetivo:

Implementar un sistema de comunicación inalámbrico (ZigBee), usando módulos de comunicación XBEE configurados en modo API. La aplicación debe ser monitoreada mediante un entorno gráfico desarrollado en LabVIEW.

Materiales:

- PC con LabVIEW y *driver* de comunicación serial NI-VISA instalados.
- Software X-CTU
- 2 módulos XBEE serie 2
- 1 adaptador XBee para *board*
- 1 unidad XBee *explorer* conexión mini-USB
- 2 sensores con salida analógica de voltaje (por ejemplo, LM35_(Temp) y SEN-10293_(Presión))
- 3 pulsadores
- 1 regulador de voltaje de 3.3V LD1117
- 1 diodo LED
- *Board*, Fuente DC (o batería de 9V), multímetro, cables o caimanes.
- Resistores, capacitores y dispositivos electrónicos analógicos en general

Estudio Previo

1. Consultar ¿qué es o cómo está definida la tecnología Zigbee?
2. ¿Cuál es el propósito del estándar de comunicación **IEEE 802.15.4**?
3. Describir brevemente los aspectos generales de los módulos XBee. Elabore una tabla donde identifique sus principales características técnicas.
4. Establezca la diferencia entre los modos de comunicación AT y API.
5. Describir las diferentes topologías de conexión que se pueden realizar con los módulos XBee cuando se conectan en red.
6. Consulte en qué tipo de aplicaciones industriales podría usarse la tecnología Zigbee y los módulos XBee. Liste varias de ellas.
7. Consultar en el *datasheet* de los módulos XBee el Data Frame API número **0x92** (*Zigbee IO data sample Rx Indicator*). Describir byte a byte el significado de cada dato. Consulte además el *datasheet* del regulador de voltaje LD1117 (3.3V)
8. Realice el montaje en *board* del (os) circuito(s) propuesto(s) y programe los códigos necesarios en software antes de la práctica. Recuerde que el tiempo en el laboratorio es limitado y lo debe optimizar realizando los montajes y programas en un horario extracurricular.

9. Para presentar su preinforme, use el formato (*template*) de la revista científica IEEE dispuesto en el siguiente enlace, con una extensión máxima de 3-4 páginas:
<https://www.ieee.org/conferences/publishing/templates> .

Experimento

Antes de comenzar su laboratorio, asegúrese de tener instalados todos los programas y *driver's* de comunicación y *firmware* necesarios:

- LabVIEW
- NI VISA drivers (Comunicación Serial)
- X-CTU de Digi

En la Figura 1., se muestra el módulo XBee serie 2, la base para board y la unidad explorer mini-USB los cuales serán usados en esta práctica para sensar dos variables analógicas (temperatura y presión o cualquier variable analógica de su preferencia) y dos digitales. Para cumplir con el objetivo propuesto, la práctica se realizará en tres partes:

Primera parte: Configuración de los módulos XBee usando el software X-CTU

Segunda Parte: Implementación física de hardware.

Tercera Parte: Desarrollo de interfaz gráfica en LabVIEW para comunicar los módulos XBee

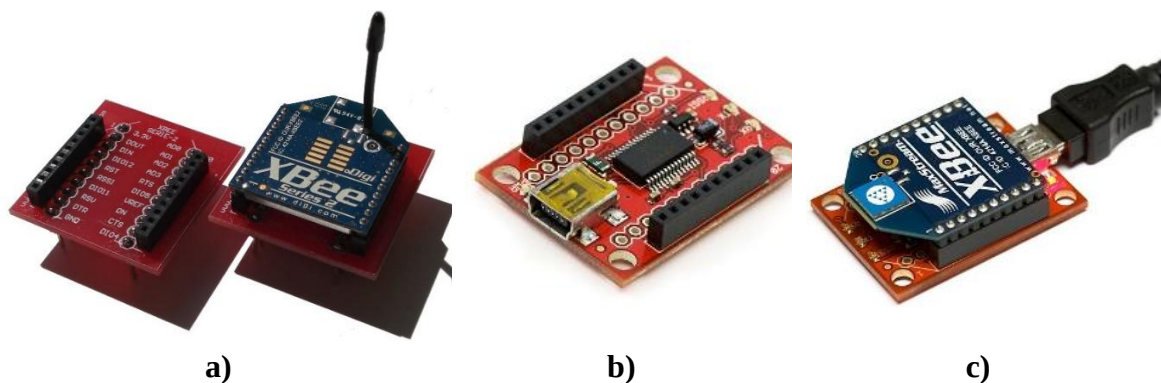


Figura 7.49. a) Módulo XBee y base para board. **b)** Unidad XBee explorer mini-USB. **c)** Módulo XBee montado sobre unidad explorer.

PRIMERA PARTE: Configuración de los módulos XBee usando el software X-CTU de Digi.

1. Descargue e instale el software X-CTU del siguiente enlace:
<https://hub.digi.com/support/products/xctu/?path=/support/asset/xctu-v-659-windows-x86x64/>

2. Monte uno de los módulos XBee en la unidad explorer mini-USB como se muestra en la Figura 7.1c y conéctelo mediante el cable USB al PC.

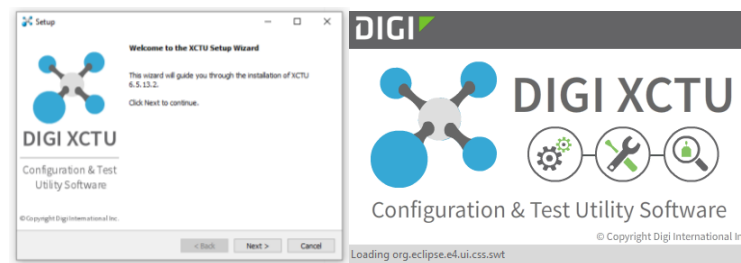
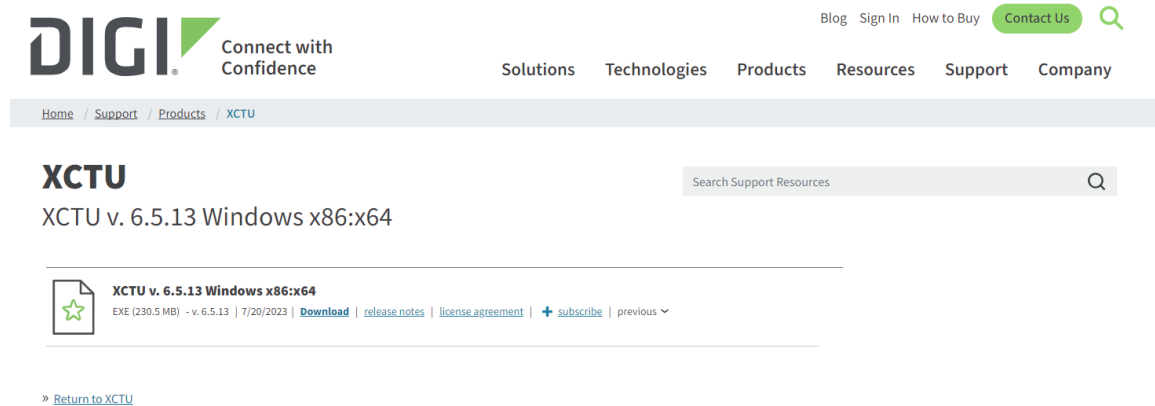


Figura 7.2. Software X-CTU de Digi

Configurando un módulo XBee como Coordinador

Para que un módulo XBee funcione como Coordinador, se debe configurar en la función de dispositivo conocida como Zigbee Coordinator API o Zigbee Coordinator AT. En nuestro caso trabajaremos en modo API.

Parámetros Clave a Configurar:

ID (PAN ID): Establece el PAN ID que utilizarán todos los dispositivos de la red. El Coordinador debe ser el primero en configurarse y este identificador debe ser igual para todos los módulos en la red.

CE (Coordinator Enable): Configura este parámetro en 1 para habilitar la función de Coordinador en el módulo.

AP (API Mode): Establece si usarás el modo API dependiendo de cómo desees comunicarte con el módulo.

CH (Channel): Define el canal que utilizará el Coordinador para la red. Es importante que este canal sea el mismo para todos los dispositivos.

JV (Channel Verification): Asegúrate de que esté configurado como *Enabled* para escanear y verificar los canales disponibles.

Una vez configurado como Coordinador, este dispositivo será el responsable de iniciar la red y gestionar las conexiones con otros dispositivos XBee configurados como *Routers* o *End Devices*.

3. Abra el software X-CTU, identifique tanto el puerto COM por donde se ha iniciado la comunicación con el módulo XBee y el tipo de módulo conectado. Después de haber identificado el módulo, lea los parámetros de configuración predeterminados haciendo *click* sobre él y cambie (re-escriba) los parámetros correspondientes, modificándolos de acuerdo con el siguiente requerimiento:

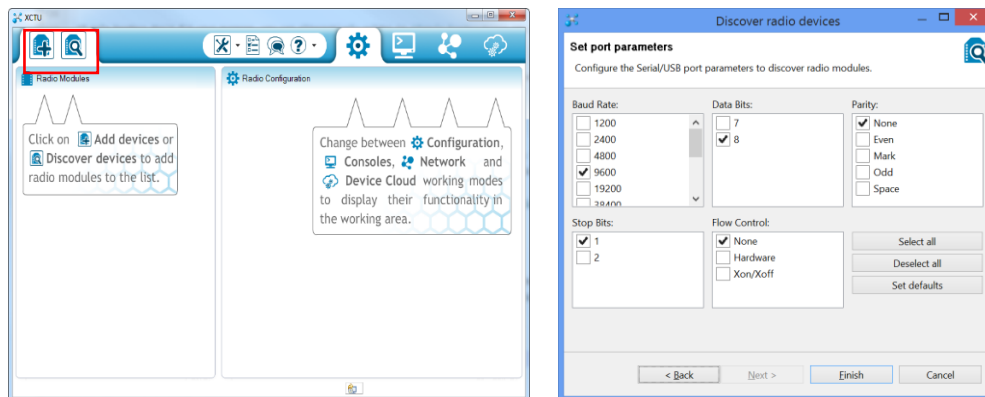


Figura 7.3. Identificación de los módulos XBee conectados en el software X-CTU

- Se requiere que este módulo sea configurado como **“Coordinator” en Modo API**. El parámetro *NI node Identifier*, hace referencia al nombre del módulo con el que se identificará en la red. Para nuestro caso será **“COORDINADOR”**. En el parámetro **“CE Coordinator Enable”** seleccione la opción **“Enabled [1]”**, tal y como se muestra en la Figura 7.4.

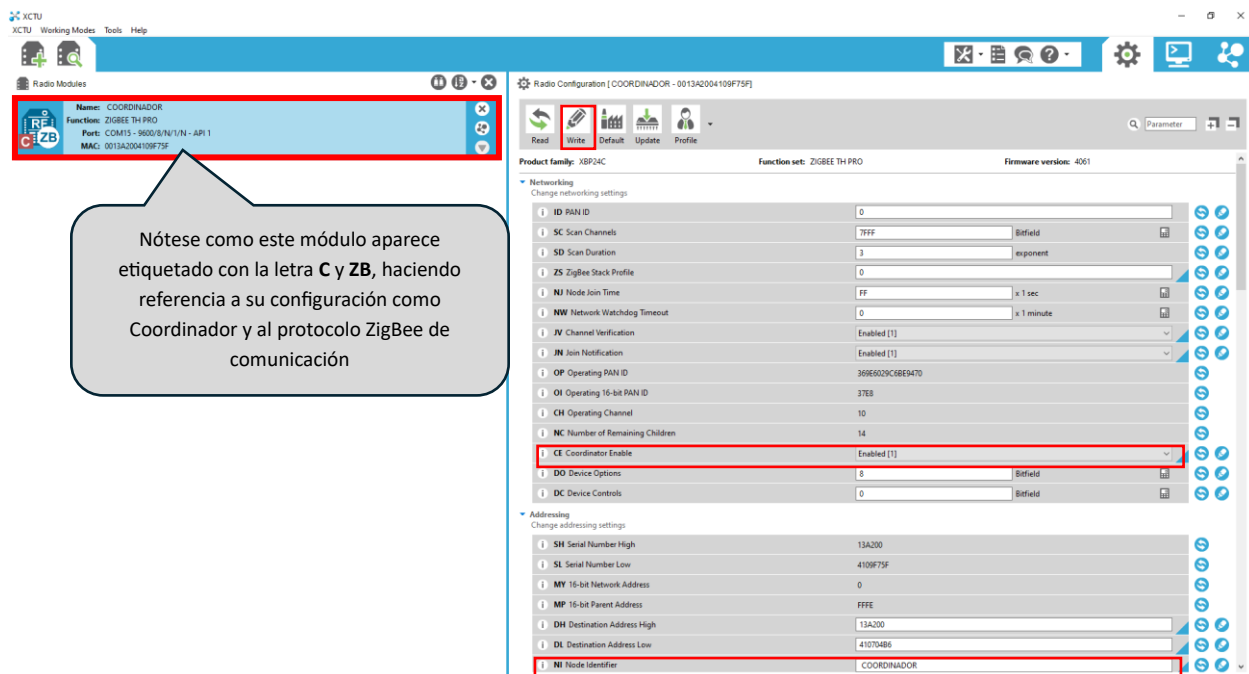


Figura 7.4. Configuración del módulo como Coordinador



NOTA: Después de haber cambiado o modificado cualquier parámetro de un módulo XBee, será necesario hacer “click” sobre el botón para escribir y reconfigurar el módulo.

- Seleccione en el parámetro “**AP API enable**”, la opción “**API Enable (1)**” como se muestra en la Figura 7.5. Este parámetro se encuentra dispuesto en la estructura “**Serial Interfacing**” del árbol jerárquico de configuración.

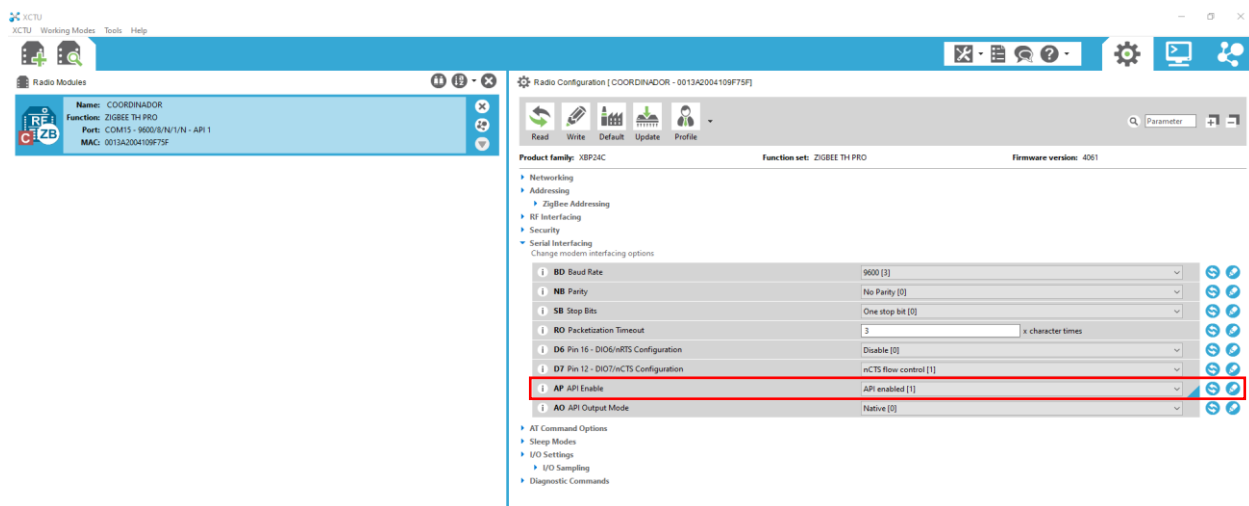


Figura 7.5. Configuración del módulo en modo API

- En los parámetros de la dirección destino (*DH Destination Address High* y *DL Destination Address Low*), digite la dirección MAC de 64 bits (*High-Low*) del módulo con el que se desea establecer la comunicación (RECEPTOR). Puede verificar esta dirección al respaldo de cada módulo. En la Figura 7.6, se muestra la conexión de un segundo módulo XBee (Receptor) con el fin de verificar la Dirección MAC de éste, y de esta manera configurar correctamente el módulo COORDINADOR.

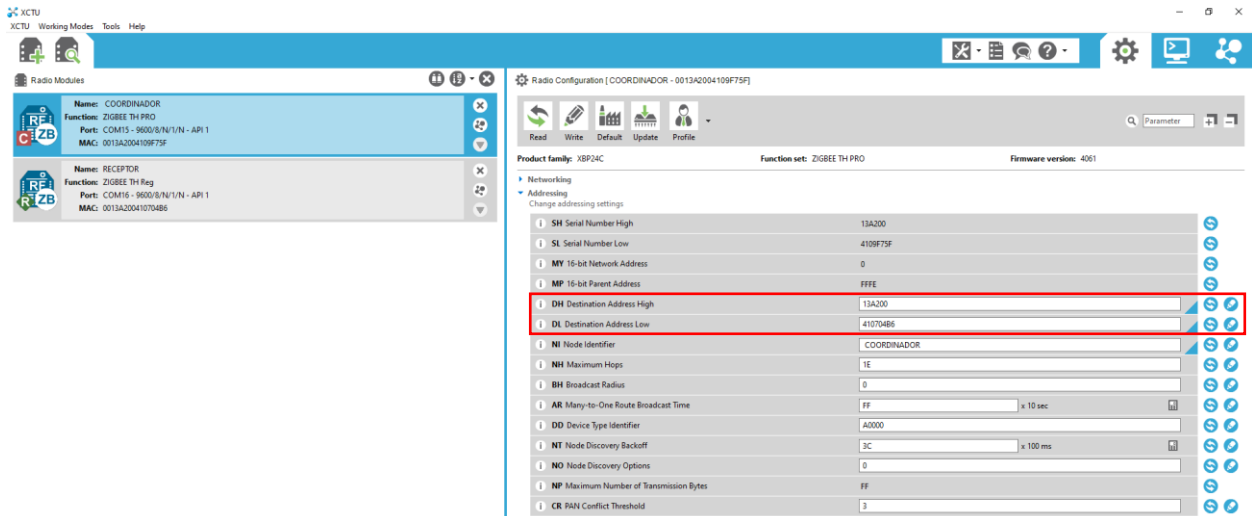


Figura 7.6. Configuración de la dirección MAC del Receptor (destino) en el módulo Coordinador

- Tenga en cuenta el canal de operación (*CH operation Channel*) que se crea para la comunicación. Necesitará este dato luego para establecer la comunicación con el módulo XBee RECEPTOR. Ver Figura 7.7.

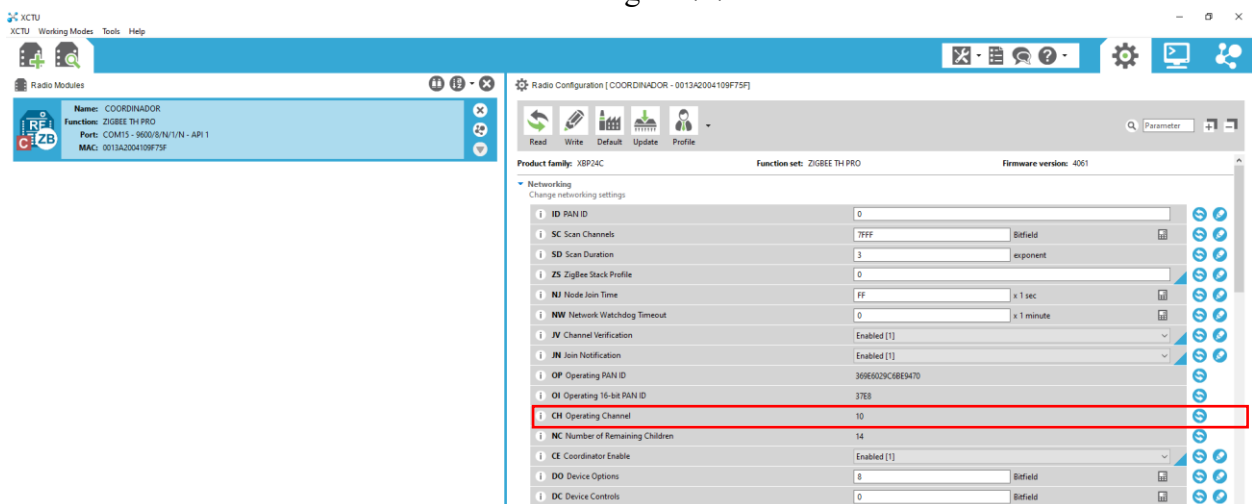


Figura 7.7. Selección del canal de comunicación

- Para establecer comunicación entre módulos, ambos deben estar configurados con el mismo tipo de *firmware*. No es posible establecer comunicación entre dos módulos XBee si uno está configurado en la función **802.15.4 TH Pro** y el otro en **Zigbee TH Reg**. Estos dos modos operan con protocolos de comunicación diferentes y no son compatibles entre sí:

Diferencias Clave entre 802.15.4 y Zigbee:

- **802.15.4** es un protocolo de capa física y de enlace de datos que proporciona una comunicación básica de punto a punto o de punto a multipunto. No incluye las capas superiores que maneja el enrutamiento o la gestión de red de forma avanzada. Un dispositivo configurado en 802.15.4 solo puede comunicarse con otros dispositivos que también estén configurados bajo este mismo protocolo y compartan el mismo PAN ID y canal.
- **Zigbee**, en cambio, utiliza 802.15.4 como base, pero añade capas adicionales (como la capa de red y la capa de aplicación) que permiten la **creación de redes de malla** y el **enrutamiento dinámico** entre dispositivos. Un dispositivo configurado en Zigbee requiere que todos los demás dispositivos en la red también estén en modo Zigbee para formar una red de malla o conectar dispositivos a través de Routers y End Devices.

En la Figura 7.8, se puede ver cómo actualizar el tipo de *firmware* o la función de cada módulo según los requerimientos de comunicación requeridos.

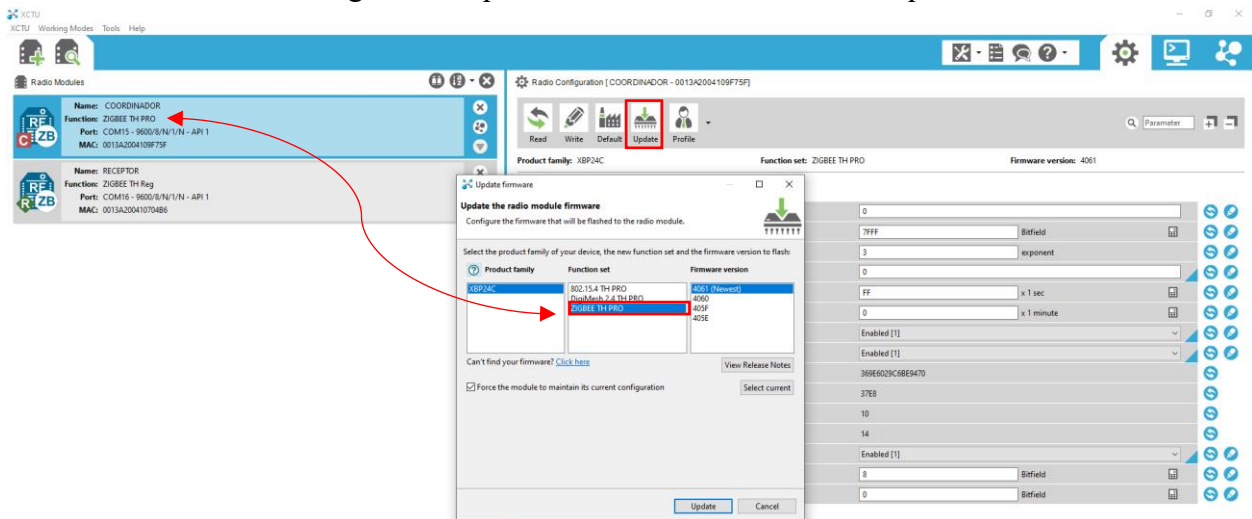


Figura 7.8. Actualización del firmware de los módulos XBee



NOTA: Para esta práctica se usará la configuración de *firmware* tipo **ZIGBEE**

Configurando un módulo XBee como Receptor

- Después de haber programado el módulo COORDINADOR, desmóntelo de la tarjeta USB y proceda a configurar el segundo XBee (RECEPTOR). Si posee dos **Unidades XBee explorer mini-USB**, podrá realizar este proceso en paralelo sin necesidad de desmontar el primer módulo.

Proceda de la misma forma que lo hizo anteriormente, pero esta vez configure el módulo Receptor de la siguiente manera:

- Se requiere que este módulo sea configurado como **END-DEVICE O ROUTER en Modo API**. Para esto, deshabilite la opción “*CE Coordinator Enable*” y elija la opción “*Disable [0]*”. Etiquete como “**RECEPTOR**” a este módulo, configurando la opción *NI Node Identifier* como se muestra en la Figura 7.9.

Nótese como este módulo aparece etiquetado con la letra R y ZB, haciendo referencia a su configuración como Receptor y al protocolo ZigBee de comunicación

Parameter	Value
ID PAN ID	0
SC Scan Channels	7FFF
SD Scan Duration	3
ZS ZigBee Stack Profile	0
NI Node Join Time	FF
NW Network Watchdog Timeout	0
JV Channel Verification	Enabled [1]
JN Join Notification	Enabled [1]
OP Operating PAN ID	3686029C0B0947D
OI Operating 16-bit PAN ID	37E8
CH Operating Channel	10
NC Number of Remaining Children	14
CE Coordinator Enable	Disabled [0]
DO Device Options	8
DC Device Controls	0
SH Serial Number High	13A200
SL Serial Number Low	410704B6
MY 16-bit Network Address	45C7
MP 16-bit Parent Address	FFFF
DH Destination Address High	13A200
DL Destination Address Low	4109F73F
NI Node Identifier	RECEPTOR

Figura 7.9. Configuración del módulo como Receptor

- En la dirección destino de este módulo, digite la dirección MAC de 64 bits (High-Low) correspondiente a la dirección propia del Coordinador. Verifique esta dirección al respaldo del módulo o desde la interfaz del X-CTU. (Ver figura 7.10).

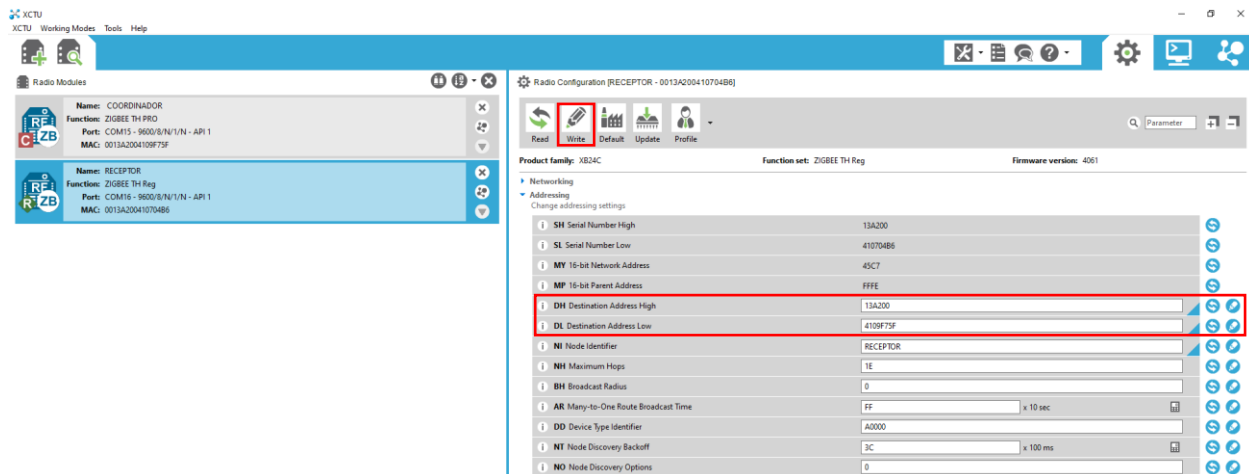


Figura 7.10. Configuración de la dirección MAC del Coordinador (destino) en el módulo Receptor

- Seleccione en el parámetro “*AP API enable*”, la opción “*API Enable (1)*” como se muestra en la Figura 7.11.

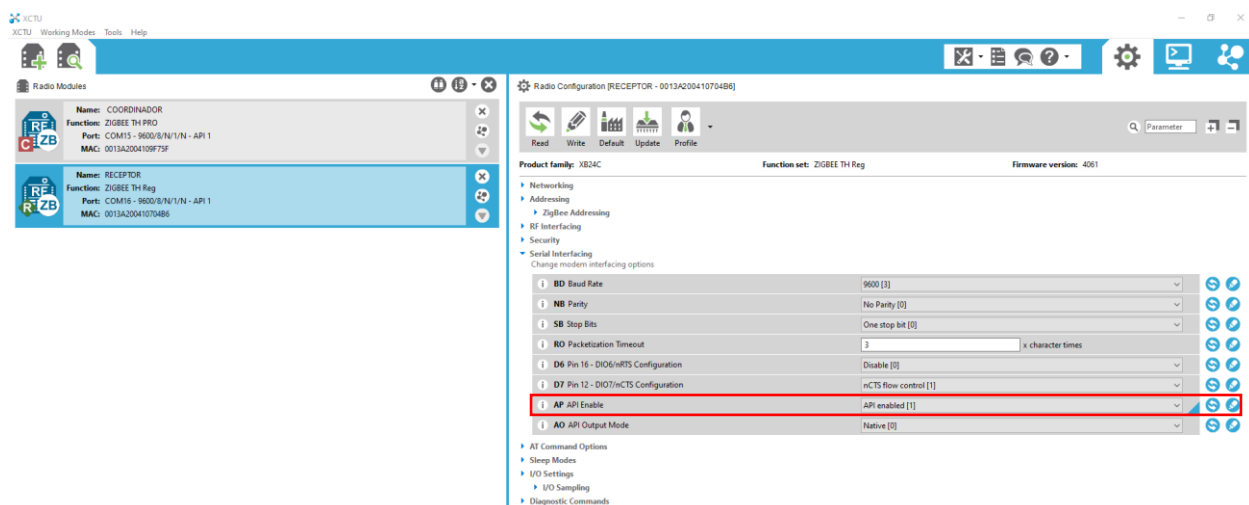


Figura 7.11. Configuración del modo API en el módulo XBee Receptor

- Verifique que este módulo tenga el mismo canal de operación que el coordinador. Los dos dispositivos deben estar en el mismo canal para establecer la comunicación entre ellos. En caso de no tener configurado el mismo canal, se debe presionar el botón de actualización para detectar el canal por donde el coordinador ha iniciado la comunicación. También se debe verificar que el parámetro *JV Channel Verification* esté habilitado (*Enabled[1]*). (Ver figura 7.12)

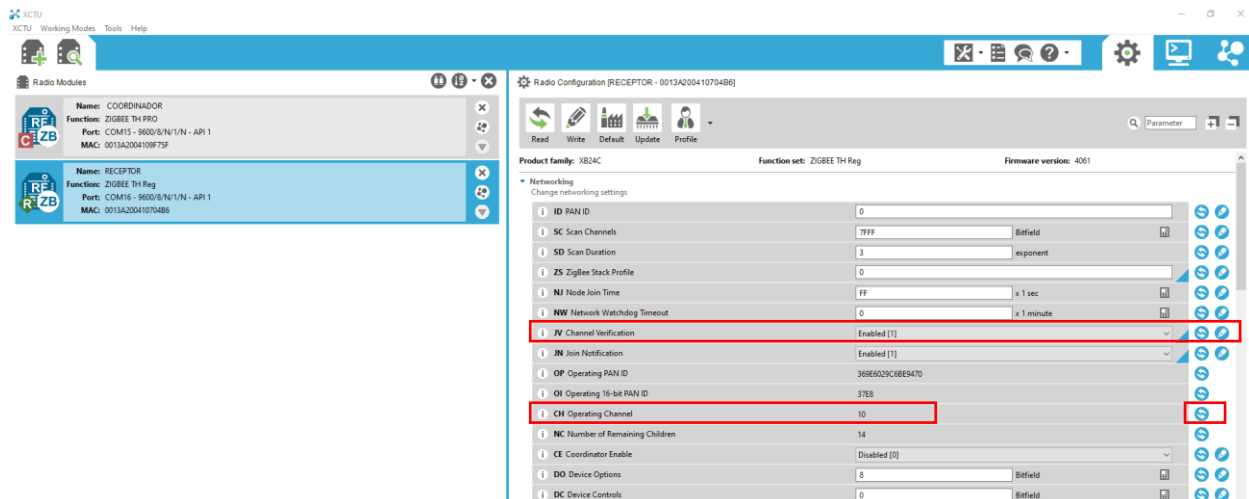


Figura 7.12. Verificación/ Configuración del canal de transmisión

- Habilite en el módulo Receptor las entradas analógicas **AD0** y **AD1** como conversores analógicos digitales ADC[2]. También habilite las entradas digitales del módulo **DIO2** y **DIO3** como *Digital Input*[3] tal y como se muestra en la Figura 7.13.

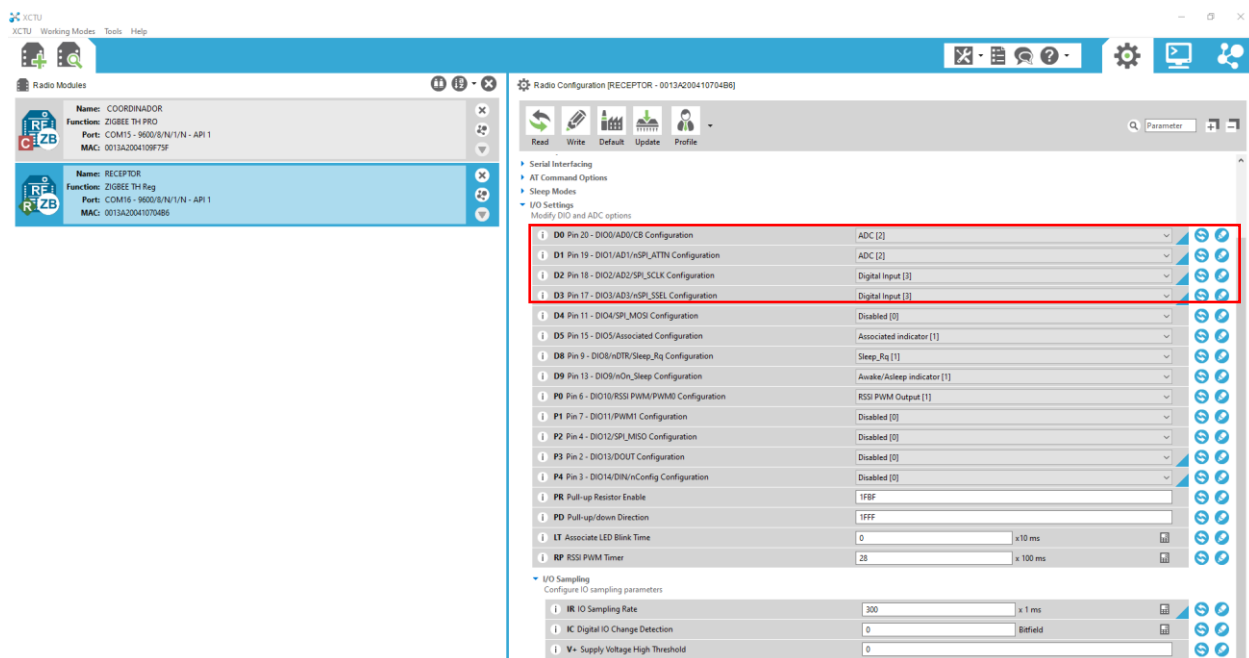


Figura 7.13. Configuración de entradas analógicas y digitales

- Configure un periodo de muestreo alrededor de 300 ms tanto en el módulo Receptor como en el módulo Coordinador. Ver Figura 7.14.

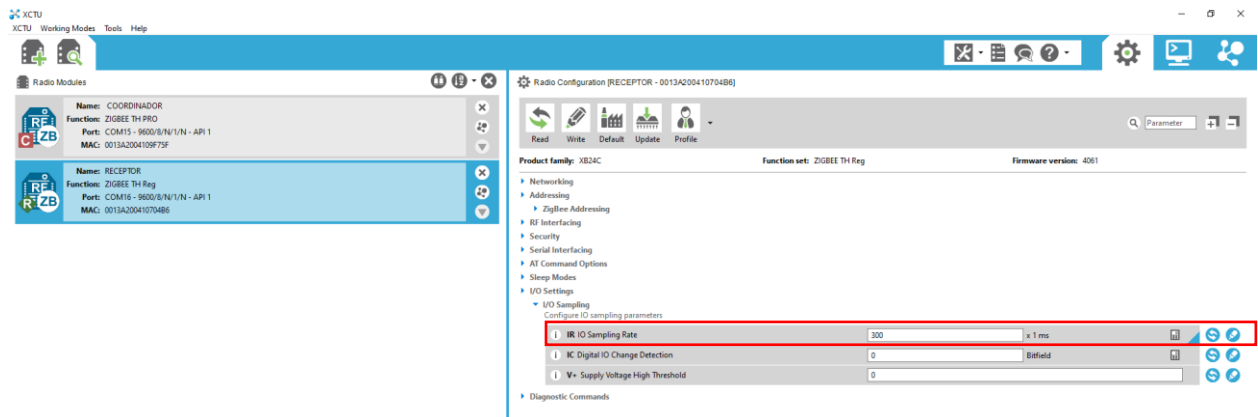


Figura 7.14. Configuración del periodo de muestreo

SEGUNDA PARTE: Implementación física de hardware.

La Figura 7.15 y la Tabla 7.1 muestran la distribución de pines de los módulos XBee.

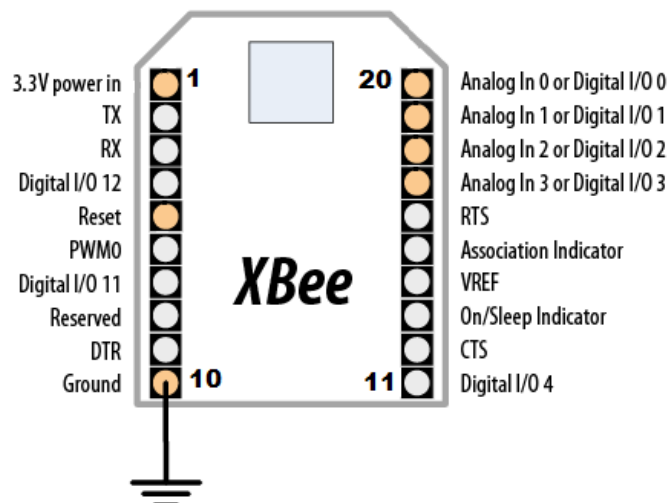


Figura 7.15. Distribución de pines del módulo XBee

Tabla 7.4. Distribución de pines del módulo XBee

Pin #	Name	Direction	Description
1	VCC	-	Power supply
2	DOUT	Output	UART Data Out
3	DIN / CONFIG	Input	UART Data In
4	DO8*	Output	Digital Output 8
5	RESET	Input	Module Reset (reset pulse must be at least 200 ns)
6	PWM0 / RSSI	Output	PWM Output 0 / RX Signal Strength Indicator
7	PWM1	Output	PWM Output 1
8	[reserved]	-	Do not connect
9	DTR / SLEEP_RQ / DI8	Input	Pin Sleep Control Line or Digital Input 8
10	GND	-	Ground
11	AD4 / DIO4	Either	Analog Input 4 or Digital I/O 4
12	CTS / DIO7	Either	Clear-to-Send Flow Control or Digital I/O 7
13	ON / SLEEP	Output	Module Status Indicator
14	VREF	Input	Voltage Reference for A/D Inputs
15	Associate / AD5 / DIO5	Either	Associated Indicator, Analog Input 5 or Digital I/O 5
16	RTS / AD6 / DIO6	Either	Request-to-Send Flow Control, Analog Input 6 or Digital I/O 6
17	AD3 / DIO3	Either	Analog Input 3 or Digital I/O 3
18	AD2 / DIO2	Either	Analog Input 2 or Digital I/O 2
19	AD1 / DIO1	Either	Analog Input 1 or Digital I/O 1
20	AD0 / DIO0	Either	Analog Input 0 or Digital I/O 0

- Realice el montaje circuital mostrado de la Figura 7.16. Tenga en cuenta que la alimentación del módulo XBee será de **3.3V**, para eso use un regulador de voltaje LD1117. Seleccione dos sensores con salida analógica de voltaje y conéctelos a los pines **20 y 19 (AD0, AD1)** del módulo XBee configurado como **End-Device o Router API** (Receptor). Asegúrese que los niveles de voltaje máximos de los sensores que use después de su etapa de acondicionamiento no superen **1.2 v**. Conecte además, dos pulsadores a las entradas digitales del módulo correspondientes a los pines 18 (DIO2) y 17 (DIO3).



NOTA: La alimentación de los módulos XBee debe ser de 3.3 V, además en sus entradas análogas el voltaje máximo debe ser de 1.2V. Esto es muy importante ya que se puede quemar el módulo.

Si está usando una Unidad XBee explorer mini-USB, no será necesario el regulador de voltaje, además podrá conectar los sensores directamente en la tarjeta sin la necesidad de la protoboard. Recuerde regular los niveles máximos de voltaje que entrega cada sensor.

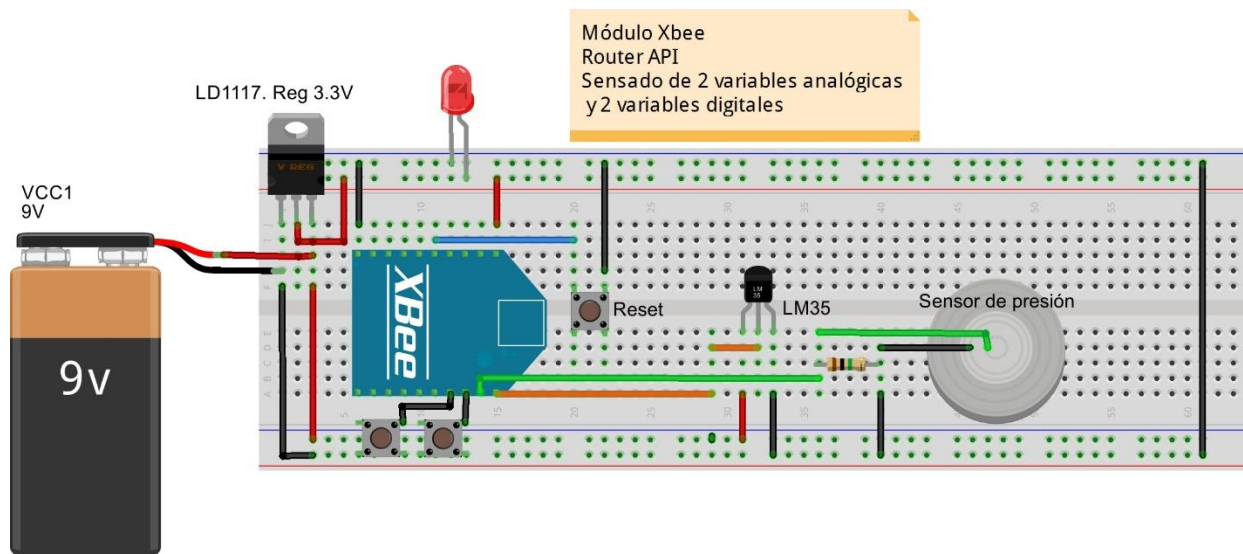


Figura 7.16. Montaje circuital del módulo XBee configurado como End-Device o Router API

La Figura 7.17, muestra el diagrama de conexiones esquemático.

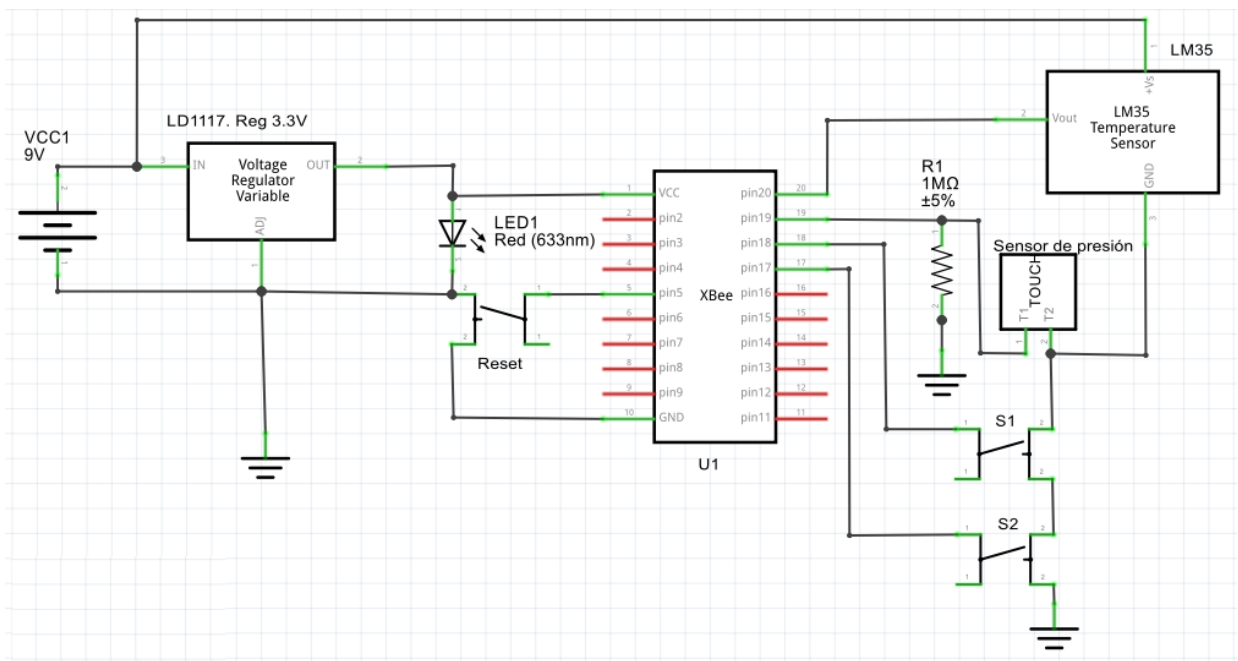


Figura 7.17. Diagrama esquemático del montaje con el módulo XBee configurado como End-Device o Router API

6. Conecte el módulo XBee configurado como **Coordinador** al PC usando la interfaz USB de la tarjeta explorer.



Figura 7.18. Conexión al PC del módulo XBee configurado como Coordinador

7. Verifique con el XCTU si se establece la conexión entre los dos módulos activando la consola del software. Al mismo tiempo, verifique que los datos entrantes pertenezcan a la trama **0x92**.

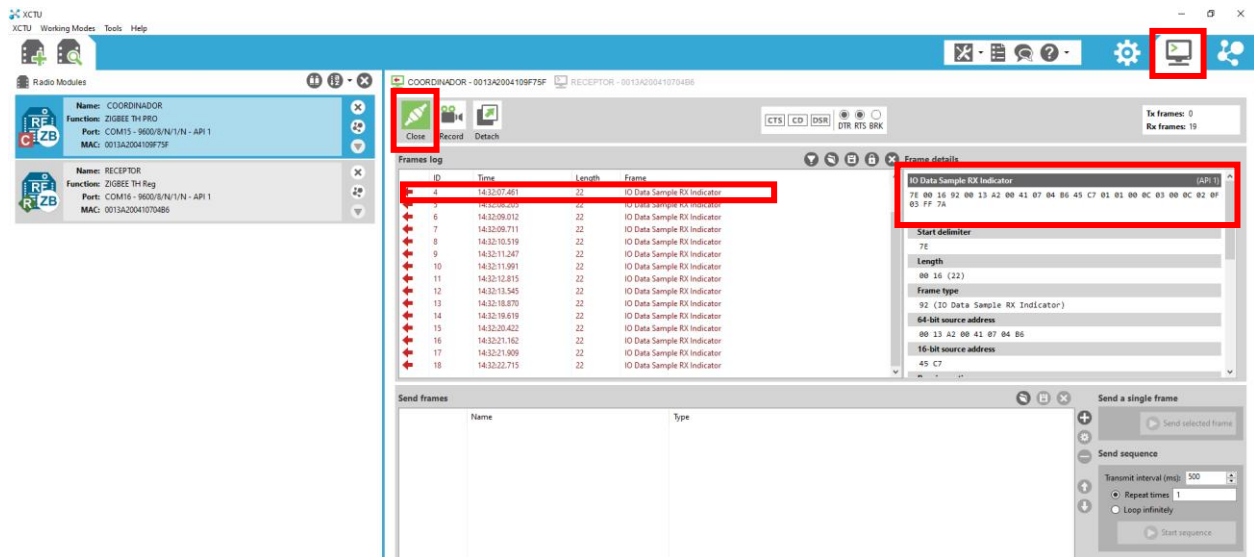
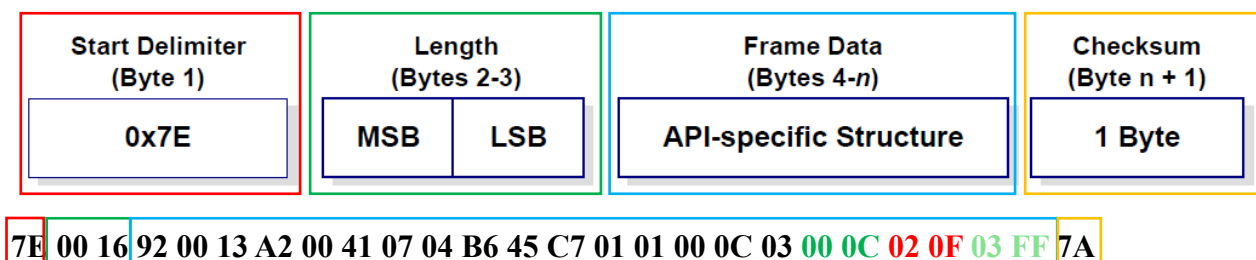


Figura 7.19. Comunicación entre los módulos XBee. Se visualiza la trama de datos enviada por el Router hacia el Coordinador

A continuación, se describe un ejemplo de una trama de datos transmitida y recibida por los módulos XBee usando un *API Frame 0x92* (ZigBee IO Data Sample Rx Indicator).



Start delimiter: 7E

Length (bytes): 00 16 (22 bytes)

Frame type: 92 (IO Data Sample RX Indicator)

64-bit source address: 00 13 A2 00 41 07 04 B6

16-bit source address: 45 C7 (*MY 16 bits Network Address of sender*)

Receive options: 01 (Paquete de datos reconocido)

Number of samples: 01

Digital channel mask: 00 0C (1100: DIO2 y DIO3 se han habilitado como entradas digitales)

Analog channel mask: 03 (0011: las entradas analógicas AD0 y AD1 se han habilitado)

DIO2/DIO3 digital value: 00 0C: (1100: DIO2 y DIO3 se encuentran activos en alto-HIGH)

AD0 analog value: 02 0F (527)

AD1 analog value: 03 FF (1023)

Checksum: 7A

TERCERA PARTE: Desarrollo de la interfaz gráfica en LabVIEW para comunicar los módulos XBee

8. Diseñe una interfaz en *LabVIEW* que permita comunicar los módulos XBee. En ella se debe visualizar el estado de las variables sensadas (2 variables analógicas y 2 variables digitales). Dependiendo del tipo de variable analógica escogida, se debe caracterizar el voltaje de entrada para representar la magnitud física de la señal en la interfaz; es decir, se debe convertir adecuadamente la señal de voltaje en unidades de °C, bar, %, etc, según sea el caso. La Figura 7.20, muestra un ejemplo de una interfaz desarrollada en LabVIEW para generar esta aplicación.

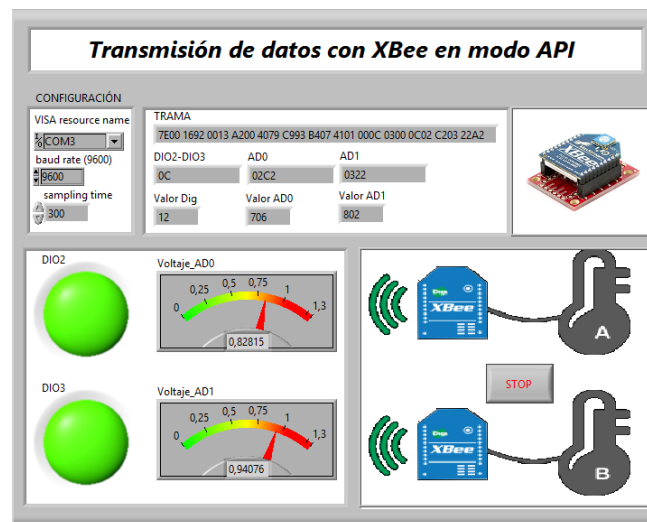


Figura 7.20. Interfaz diseñada en LabVIEW para comunicar dos módulos XBee

Para visualizar las tramas de datos correctamente en la interfaz de LabVIEW, se debe configurar los indicadores para que muestren los datos en formato Hexadecimal como lo indica la Figura 7.21.

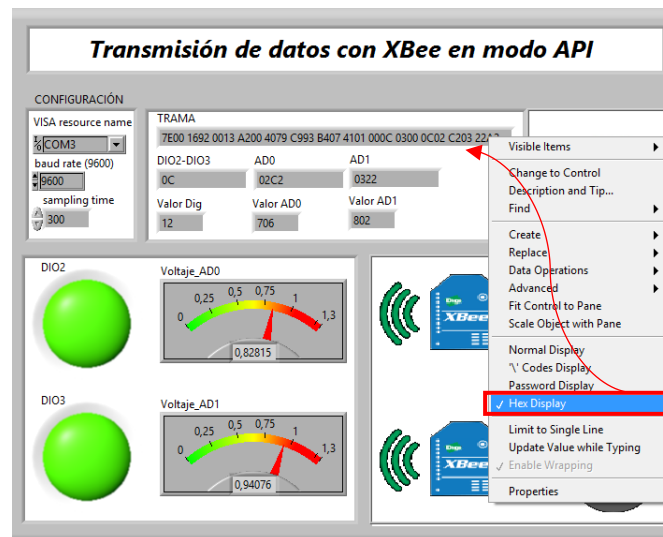


Figura 7.21. Visualización de los indicadores en formato Hexadecimal
El algoritmo gráfico que representa la interfaz de la figura 7.20 es el siguiente:

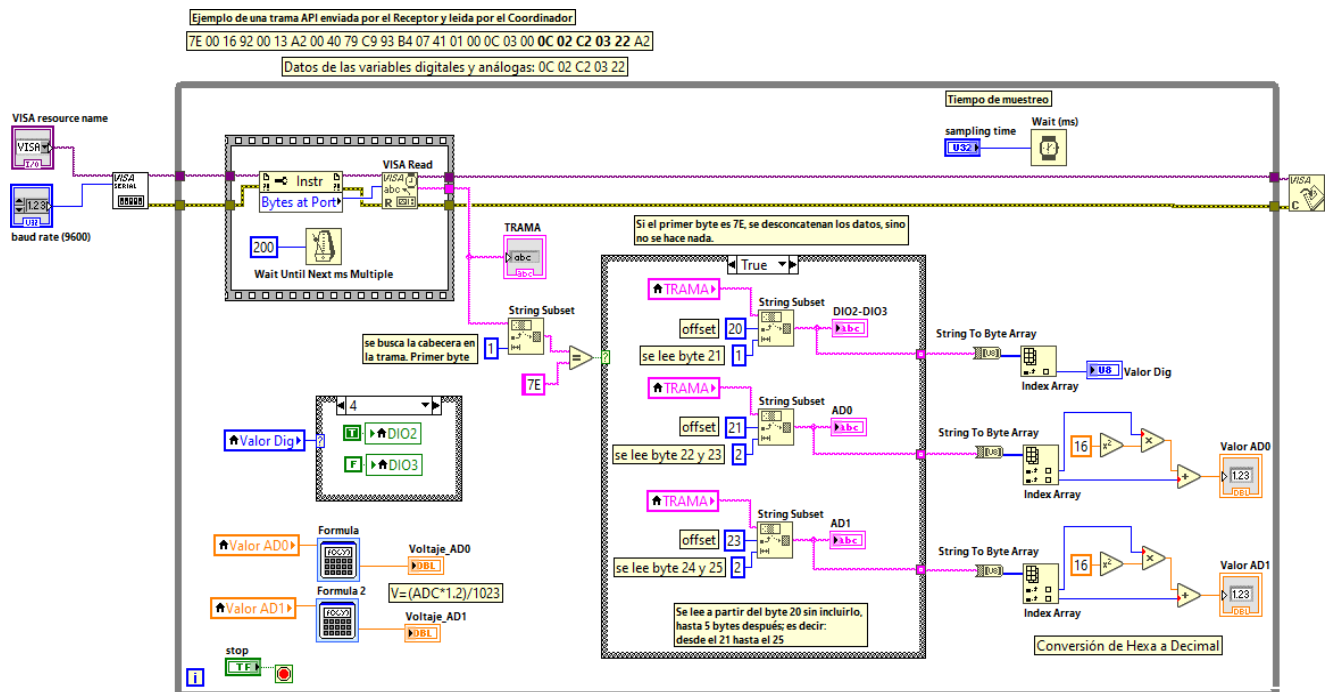


Figura 7.22. Código de la interfaz de la figura 7.20 usando NI-VISA

Como ya sabemos, el voltaje máximo en las entradas analógicas será de **1.2 V**, por lo tanto, para representar los datos del **ADC_(10 bits)** en voltaje, los convertimos de la siguiente forma:

$$V = \frac{Dato(ADC) * 1.2}{1023}$$

9. Energice el módulo Receptor y conecte el Coordinador al PC, identificando en la interfaz el puerto COM donde se ha iniciado la comunicación y el *Baud Rate*. Ejecute la aplicación desarrollada verificando su buen funcionamiento.

Conclusiones

- Escriba las conclusiones más relevantes de la práctica de laboratorio, indicando hallazgos, dificultades y logros.

Práctica No.8

Transmisión de datos usando protocolo HTTP y LabVIEW: aplicaciones de Internet of Thing- IoT

Objetivo:

Implementar un sistema de comunicación inalámbrico basado en el protocolo de Internet HTTP para aplicaciones de IoT. La aplicación debe ser monitoreada mediante un entorno gráfico desarrollado en LabVIEW.

Materiales:

- PC con LabVIEW instalado
- Conexión estable a Internet mediante Wifi o cableada
- Sensores con salida analógica o potenciómetros de 1K ohm
- *Board*, Fuente DC (o batería de 9V), multímetro, cables o caimanes.

Estudio Previo

1. Consultar ¿Qué es el protocolo HTTP y para qué se usa?
2. Explique cómo se define una trama de datos HTTP.
3. Establezca las principales diferencias técnicas entre los protocolos HTTP y MQTT. Llene una tabla con los hallazgos encontrados.
4. Consulte en qué tipo de aplicaciones industriales y de IoT podría usarse este tipo de tecnología de comunicación. Cite algunos ejemplos.
5. Consulte las siguientes aplicaciones de IoT: *Ubidots*, *ThingSpeak* y *Blynk* y explique sus principales características, ventajas y desventajas (tiempos de muestreo y reporte de datos a la nube, capacidad de número de variables, precios, etc). Lleve los hallazgos de su consulta a una tabla comparativa.
6. Se recomienda ver el siguiente video ilustrativo referente al uso del protocolo HTTP en LabVIEW y la plataforma ThingSpeak: <https://www.youtube.com/watch?v=m-P0W-Qv0D4>
7. Se recomienda ver los siguientes dos (2) videos que proporciona información adicional relativa a la creación de un proyecto IoT en la plataforma Blynk Cloud y Blynk App:
<https://www.youtube.com/watch?v=w3XL-ioddqk&t=228s>
<https://www.youtube.com/watch?v=HRGQQATYJCQ&t=1097s>
8. Programe los códigos necesarios en software antes de la práctica. Recuerde que el tiempo en el laboratorio es limitado y lo debe optimizar realizando los montajes y programas en un horario extracurricular.
9. Para presentar su preinforme, use el formato (*template*) de la revista científica IEEE dispuesto en el siguiente enlace, con una extensión máxima de 3-4 páginas:
<https://www.ieee.org/conferences/publishing/templates> .

Experimento

HTTP es el protocolo fundamental para el funcionamiento de la Web y también es ampliamente utilizado en el Internet de las Cosas (IoT) para enviar o recibir datos entre dispositivos y servicios en la nube. Su funcionamiento está basado en un servicio cliente-servidor, donde el primero inicia la comunicación enviando una solicitud (request) y el segundo procesa la solicitud y envía una respuesta (response). Algunos ejemplos de tramas de comunicación HTTP implementadas para escribir datos en plataformas web como ThingSpeak o Blynk, se pueden observar en la Tabla 1. En la Tabla 2 se ilustra la nomenclatura de colores para describir las tramas de datos enviadas como ejemplo en la Tabla 1.

Tabla 1. Ejemplos de tramas HTTP para escritura de datos en plataformas web

Método	Plataforma	String o cadena de caracteres a enviar
Escritura 1 canal	ThingSpeak	https://api.thingspeak.com/update?api_key=xxxxxxxxxx&field1=10
Escritura 2 canales		https://api.thingspeak.com/update?api_key=xxxxxxxxxx&field1=10&field2=20
Escritura 3 canales		https://api.thingspeak.com/update?api_key=xxxxxxxxxx&field1=10&field2=20&field3=5
Escritura 1 canal	Blynk	https://ny3.blynk.cloud/external/api/update?token=xxxxxxxxxx&v0=1
Lectura 1 canal		https://ny3.blynk.cloud/external/api/get?token=xxxxxxxxxx&v1*
Lectura 2 canales		https://ny3.blynk.cloud/external/api/get?token=xxxxxxxxxx&v0*&v1*

*No se necesita especificar un valor puesto que el método es de lectura

Tabla 2. Nomenclatura de colores para describir las tramas de datos de la tabla 3 y pasos para establecer conexión

	URL del servicio web	Pasos para establecer la comunicación con el servicio web (HTTP) 1. Se abre la conexión <i>HTTP</i> con el cliente 2. Se escribe la trama de datos comenzando por la dirección del servicio web (página), luego la API KEY, y después los valores de las variables con su respectivo valor. Nótese que el carácter "&" es un separador entre campos
	Método (escritura o lectura)	
	API KEY o Token	
	& "separador"	
	Variable 1+ valor	
	Variable 2+ valor	
	Variable 3+ valor	

La plataforma *Blynk* fue seleccionada entre otras aplicaciones para realizar esta práctica por su versatilidad y facilidad de integración con sistemas de monitoreo en tiempo real. En la figura 8.1, se puede observar el esquema general de la conectividad entre los equipos del sistema requeridos para la práctica.

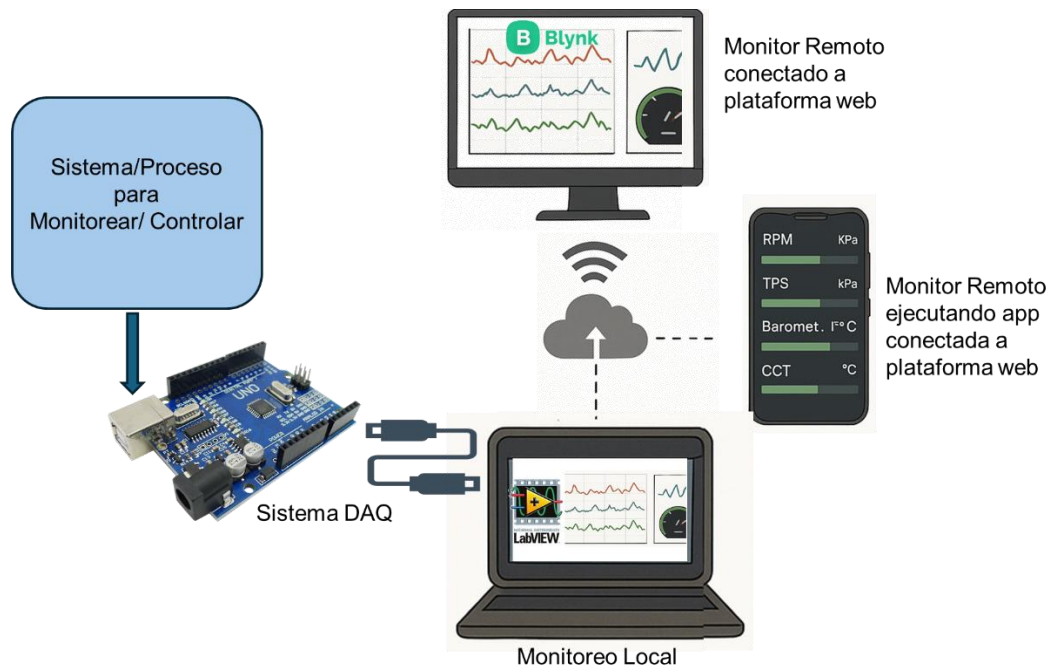


Figura 8.1. Esquema general del sistema de telemetría basado en IoT

Diseño de la interfaz de monitoreo en la plataforma Blynk Cloud para PC

Antes de comenzar su laboratorio, cree una cuenta en la plataforma Blynk IoT. Para esto visite la página <https://blynk.io/> y haga click en el botón “Sign Up Free” (ver figura 8.2 y 8.3).

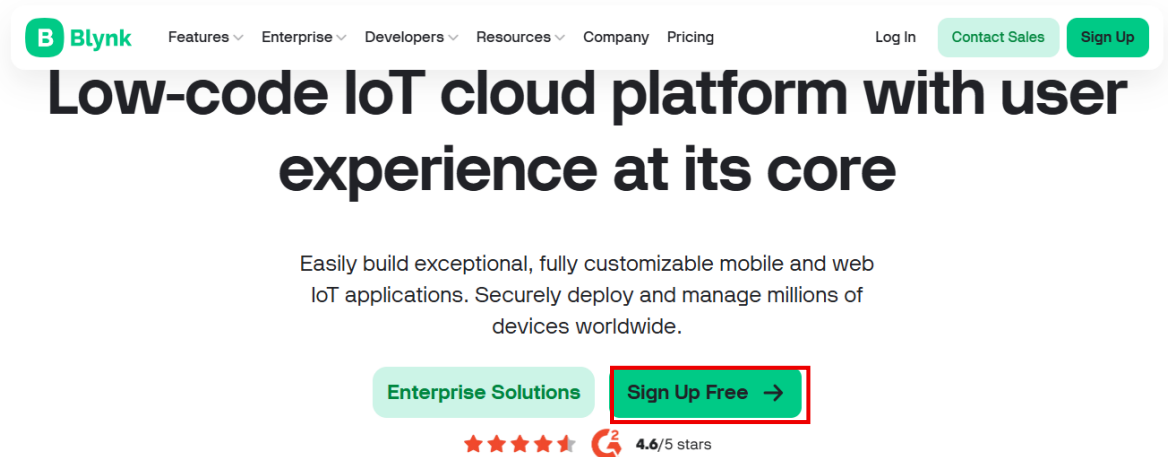


Figura 8.2. Página web de Blynk

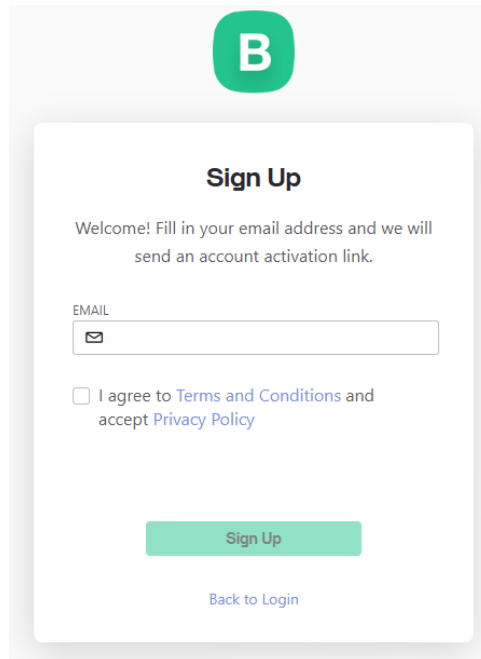


Figura 8.3. Interfaz para creación de cuenta gratuita en Blynk

Una vez creada la cuenta podrá diseñar su propia *dashboard* para el monitoreo remoto de su aplicación mediante IoT. A continuación, se ilustran los pasos necesarios que deberá seguir para crear su entorno gráfico.

1. Seleccionar la opción “*Developer Zone*”, luego en “*My Templates*” y finalmente en “*+New Template*” para crear una plantilla de trabajo. (ver figura 8.4)

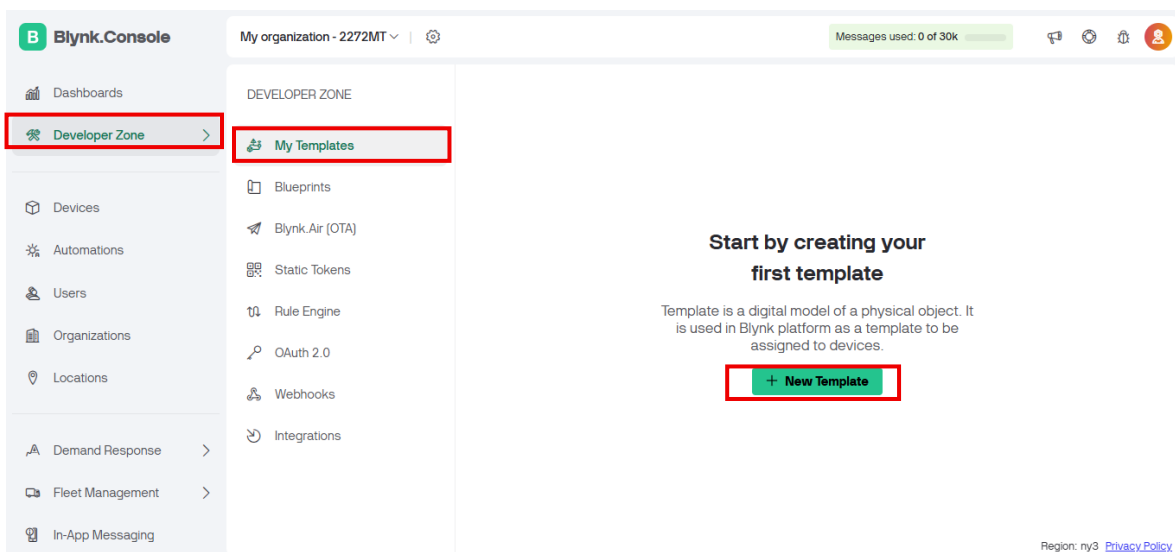


Figura 8.4. Creación de una plantilla de trabajo

2. En la plantilla (*Template*) creada, adicione el nombre del proyecto, el tipo de hardware con el que trabajará (Arduino, ESP32, etc) y el tipo de conexión (wifi, ethernet, etc),

también podrá redactar una descripción de su proyecto. Finalice con el botón “*Done*” (ver figura 8.5).

Create New Template

NAME

Proyecto IoT Acondicionamiento 30 / 50

HARDWARE CONNECTION TYPE

Arduino WiFi

DESCRIPTION

Práctica de Laboratorio de IoT

30 / 128

Cancel Done

Figura 8.5. Edición de los parámetros del proyecto sobre la plantilla de trabajo

3. Una vez creada la plantilla y definido el nombre del proyecto, haga *click* sobre la opción *Datastreams* y luego sobre el botón *Edit*. (ver figura 8.6)

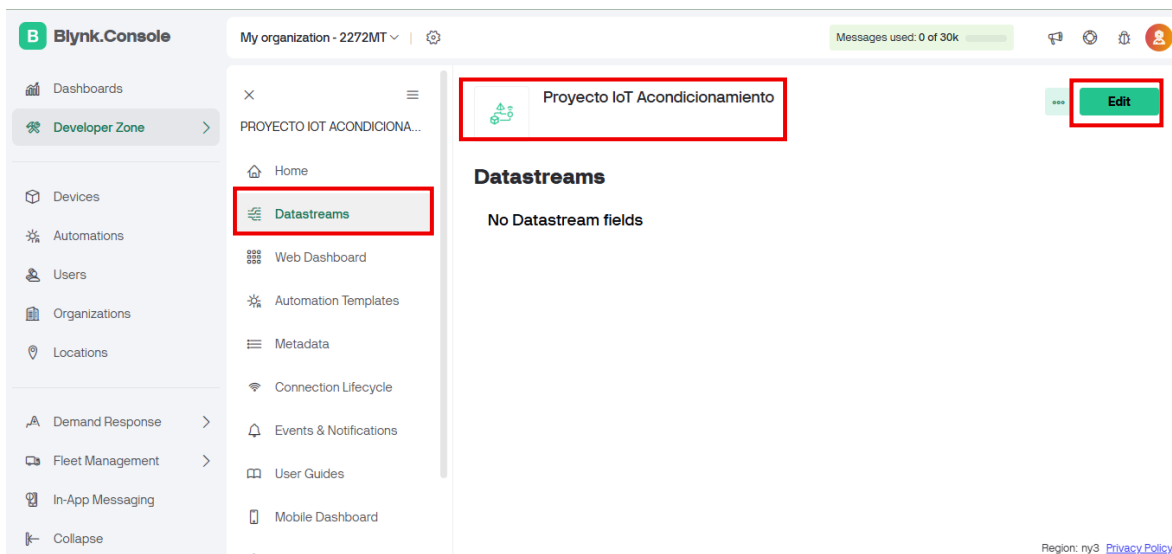


Figura 8.6. Procedimiento para la edición del flujo de datos

4. Adicione un nuevo flujo de datos haciendo *click* sobre el botón “+New Datastreams” (ver figura 8.7)

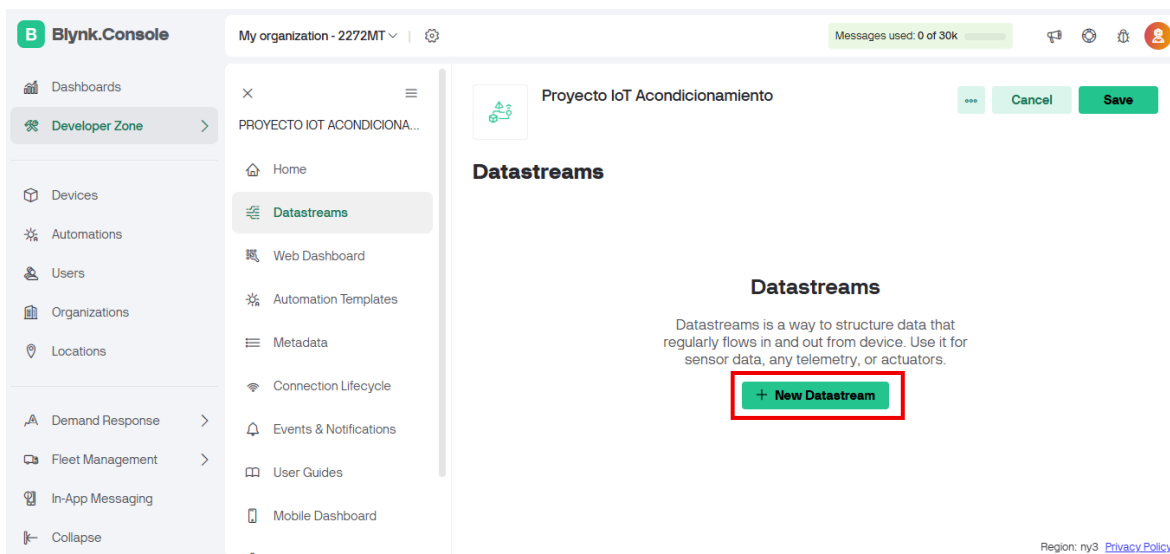


Figura 8.7. Creando un nuevo *Datastream*

5. Seleccione la opción *Virtual Pin* sobre la ventana emergente (ver figura 8.8)

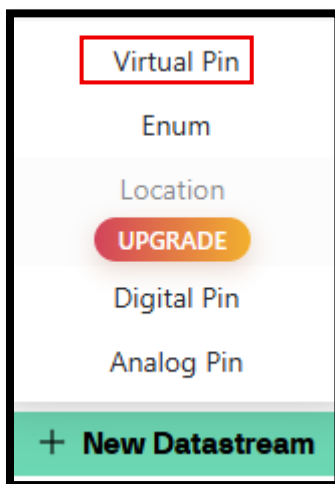


Figura 8.8. Seleccionando el tipo de pin

6. Una vez seleccionado el tipo de Pin, se debe editar las características particulares de cada uno de ellos, proporcionando datos como el nombre de la variable que estará conectada a ese Pin, número o identificador del Pin virtual (V0, V1, V2, etc), el tipo de dato (entero, flotante, booleano, etc), unidades de la variable, rango (valores mínimos y máximos de la variable), etc. Finalice con el botón “*Create*” (ver figura 8.9).

Virtual Pin Datastream

General
Expose to Automations

NAME

ALIAS

PIN

V0

DATA TYPE

Integer

UNITS

None

MIN

MAX

DEFAULT VALUE

☐ Enable history data

Figura 8.9. Configuración de parámetros del Pin Virtual

En la Figura 8.10 se puede evidenciar sobre el *Datastream* la creación de la *Variable 1* en el Pin virtual *V0* tipo Entero.

Blynk.Console

Dashboards
Developer Zone
Devices
Automations
Users
Organizations
Locations
Demand Response
Fleet Management
In-App Messaging
Collapse

My organization - 2272MT
Messages used: 0 of 30k

Proyecto IoT Acondicionamiento

Cancel

Save

Datastreams

Search datastream

+ New Datastream

1 Datastream

ID	Name	Pin	Color	Data Type	Actions
1	Variable 1	V0		Integer	

Region: ny3
Privacy Policy

Figura 8.10. Configuración de parámetros del Pin Virtual

- Realice los pasos anteriores para adicionar un nuevo *Datastream* y por lo tanto una nueva variable de medición. Este paso se realiza tantas veces como variables desee monitorear (o controlar). En la Figura 8.11 se puede evidenciar la creación de tres (3)

variables tipo entero conectadas a los pines virtuales V0, V1 y V2. Este proceso finaliza haciendo *click* sobre el botón “Save” para guardar los cambios.

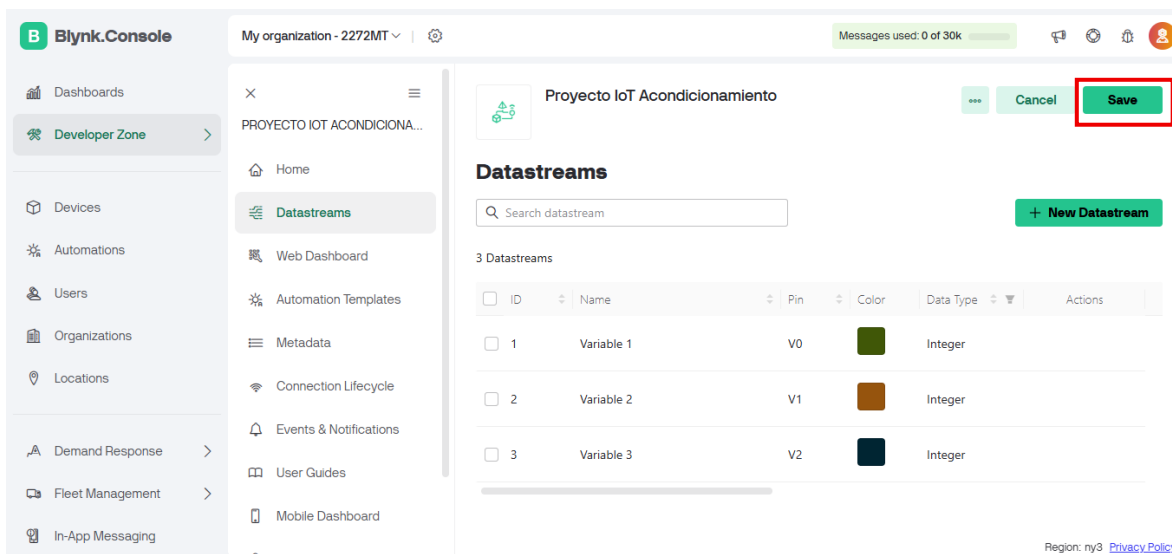


Figura 8.11. Creación de tres variables sobre la plantilla de trabajo

- Una vez configurado el *Datastream*, se debe volver a la opción “Developer Zone” y en la opción “Web Dashboard” podrá editar el tablero de monitoreo de señales. (Ver figura 8.12)

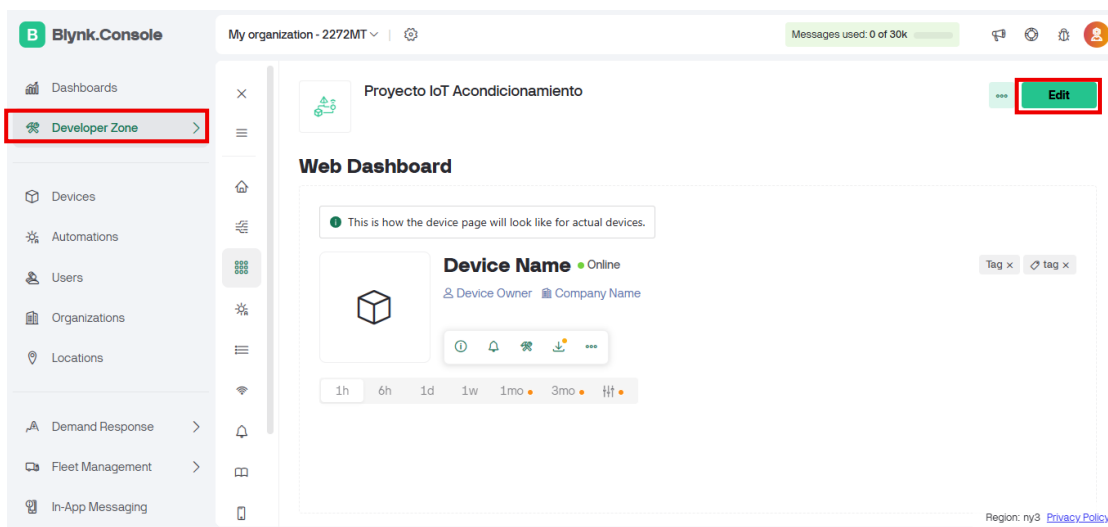


Figura 8.12. Edición del tablero o *Dashboard* para el monitoreo de señales

- Podrá seleccionar entre los diferentes instrumentos virtuales proporcionados por la interfaz algunos indicadores tipo LED, indicadores numéricos digitales, *Sliders*, indicadores tipo aguja (Gauge), etc. Seleccione los instrumentos virtuales disponibles de su preferencia y diseñe la interfaz de usuario acorde al número de variables que haya creado (ver figura 8.13).



Nota: Para el desarrollo de su aplicación o práctica de laboratorio, use al menos dos (2) variables analógicas y dos (2) digitales.

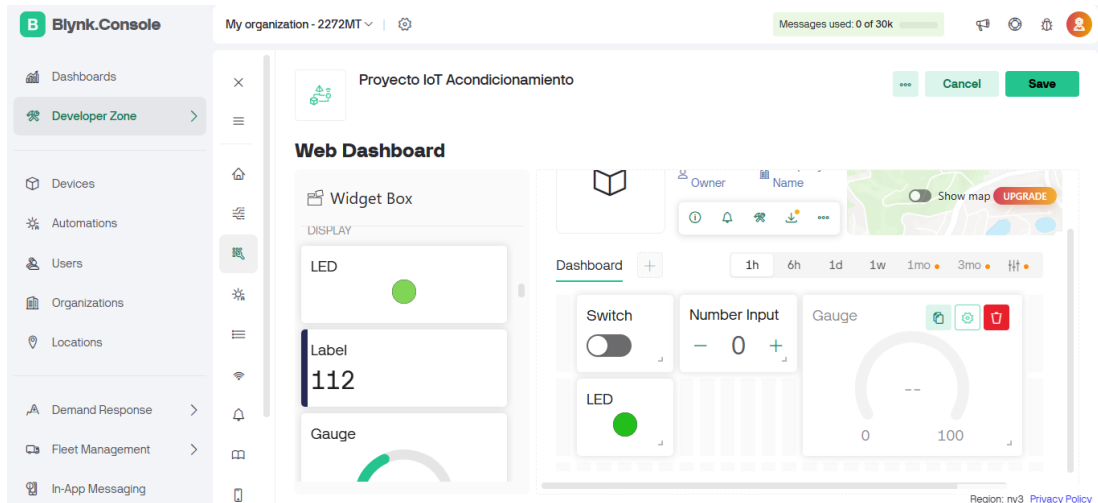


Figura 8.13. Ejemplo de diseño de una interfaz de usuario (Dashboard) en Blynk.

10. Cada *widgets* adicionado, deberá ser asociado a cada una de las variables (o pines virtuales) creados previamente en los pasos 6 y 7. Para configurar cada *widget* haga *click* sobre el ícono de configuración (piñón) como se muestra en la figura 8.14.

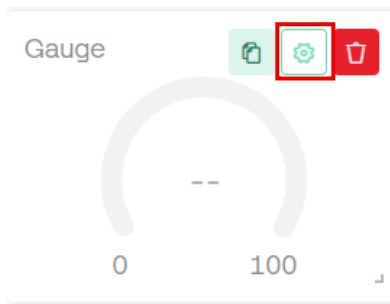


Figura 8.14. Indicador tipo Gauge y su ícono de configuración

En la figura 8.15 se muestra tanto un indicador tipo *Gauge* como un indicador numérico digital. Para cada uno de ellos se debe seleccionar un *Datastream* y un Pin Virtual de los antes creados. De esta manera se asocia la variable del flujo de datos con el indicador respectivo del *Dashboard*.

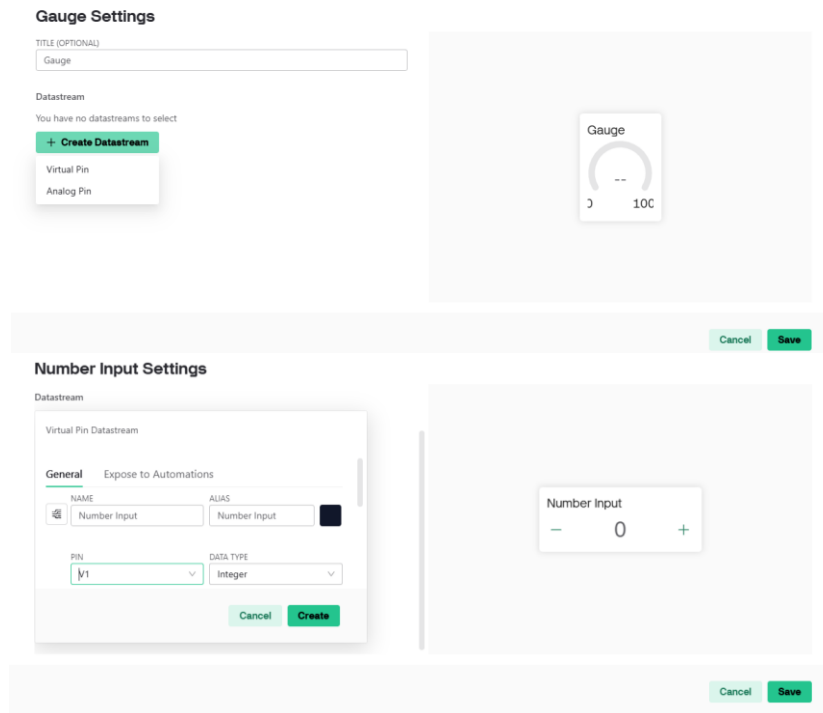


Figura 8.15. Configuración de los *widgets* del *Dashboard*

11. Una vez llegado a este paso, podrá evidenciar que haciendo *click* sobre el botón “Developer Zone” y luego en “Home” aparecerá un *check list* con los pasos realizados y los que falta por configurar. En este caso se realizó la configuración del *Template*, el *Datastream* y el *Dashboard* (Ver figura 8.16). Para terminar el proceso es necesario crear un Dispositivo (“Device”). Haga *click* sobre “Add First Device” y cree un dispositivo con el nombre de su preferencia.

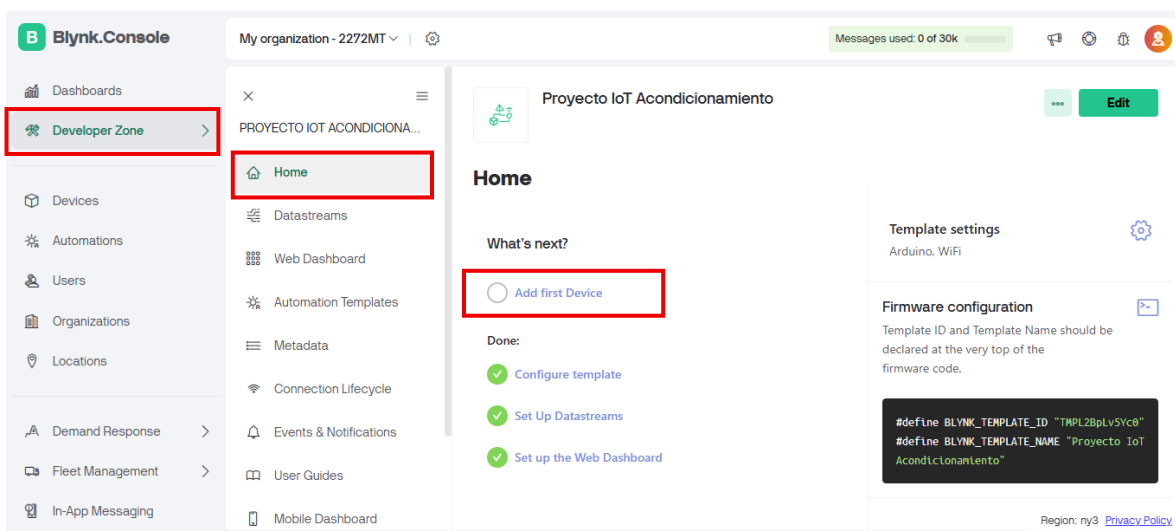


Figura 8.16. Proceso para la creación de un dispositivo

En la figura 8.17 se ilustra la ventana donde se solicita el nombre del “Device”. Finalice el proceso haciendo *click* sobre el botón “Create”.

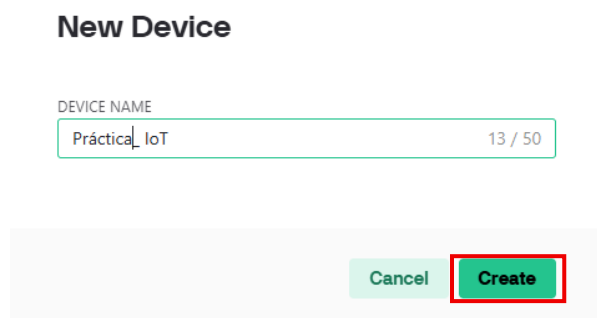


Figura 8.17. Creación de un dispositivo

12. En este momento ya habrá creado su interfaz de monitoreo para IoT usando la plataforma *Blynk*, ahora tendrá disponible su **API KEY** o **Token**, el cual podrá visualizar en la ventana emergente como se ilustra en la figura 8.18 o en la 8.19. Este código alfanumérico es clave y necesario para poder comunicar la plataforma o servicio Web con el Cliente (sistema electrónico embebido, LabVIEW, etc). *!!! No lo olvide !!!*.

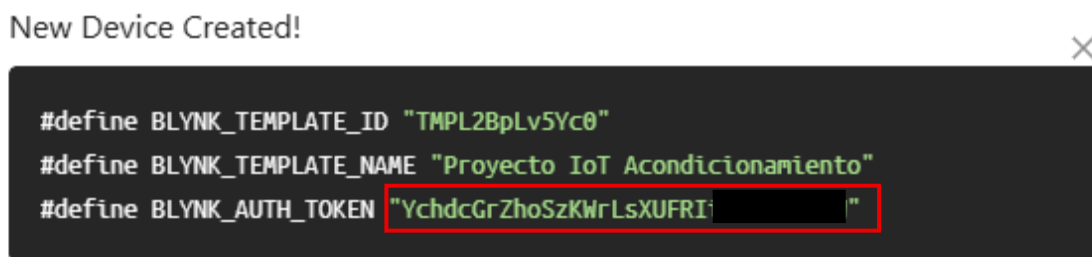


Figura 8.18. Visualización del Token o API KEY

En la figura 8.19, se puede observar el proyecto finalizado. Al hacer *click* en “Developer Zone/ My Templates”, Podrá observar la plantilla creada (Proyecto IoT Acondicionamiento), el cual incluye ya un dispositivo. Al hacer *click* en “Devices” podrá observar el nombre del Dispositivo y el Token.

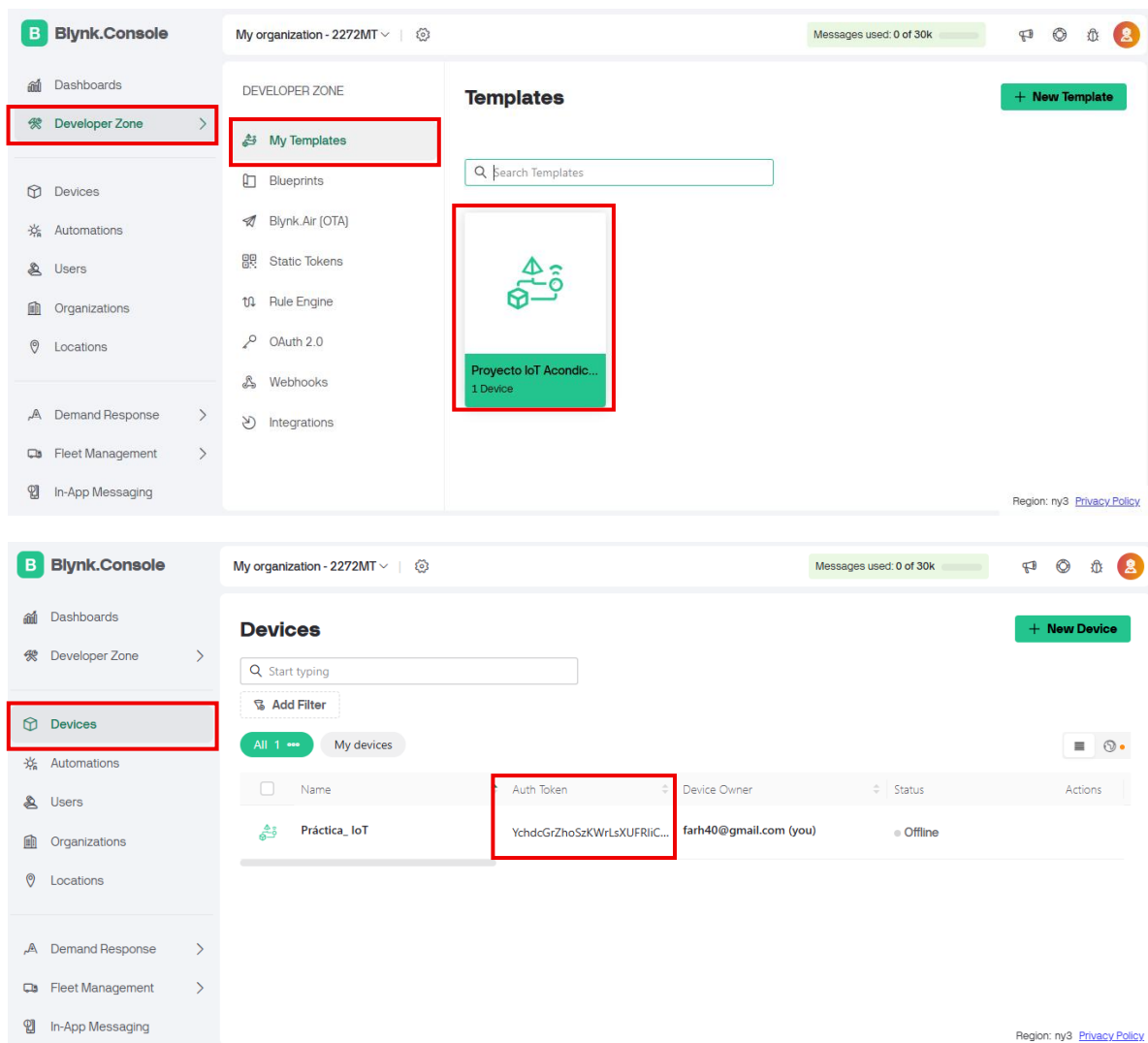


Figura 8.19. Visualización del Proyecto IoT finalizado en la plataforma web

Diseño de la interfaz de monitoreo en la plataforma Blynk App para Smartphones

A continuación, se relacionan los pasos para desarrollar el diseño de la interfaz de monitoreo de variables mediante IoT usando la aplicación celular Blynk.

1. Busque en su tienda de aplicaciones *Blynk IoT* y proceda a instalarla en su equipo.

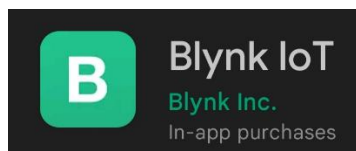
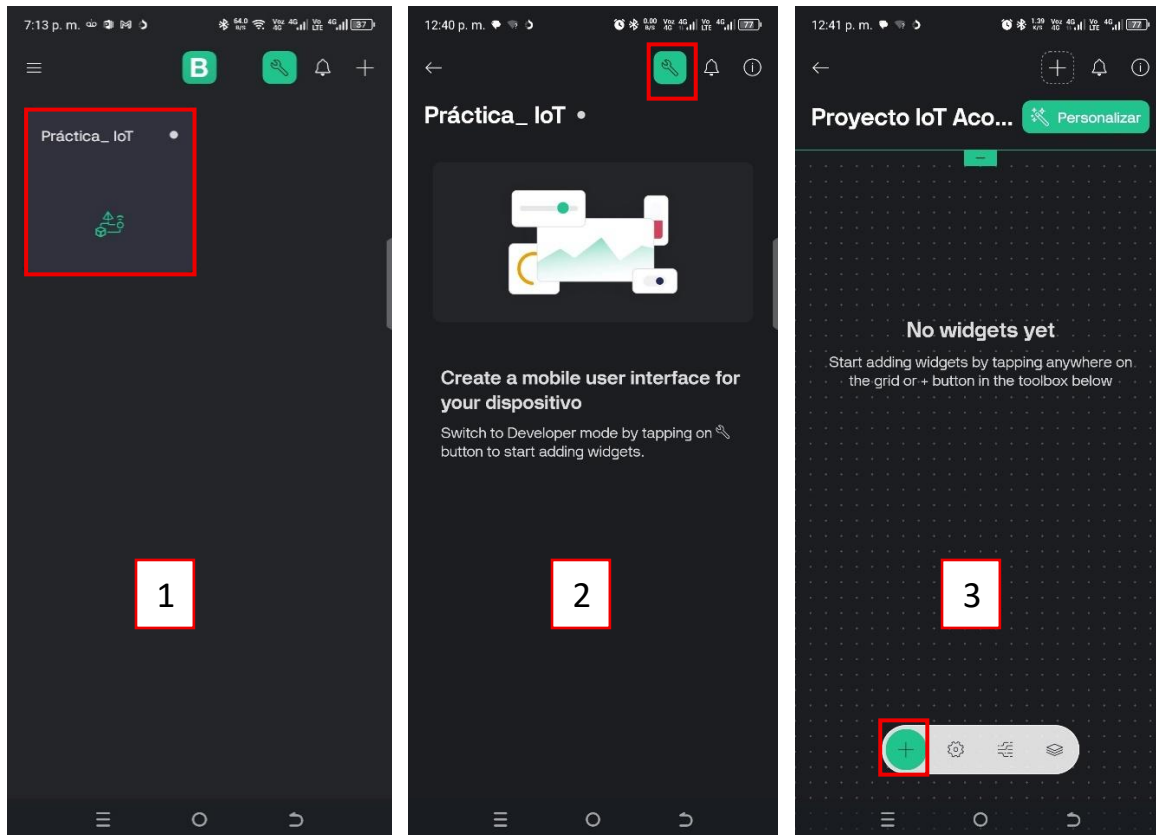


Figura 8.20. Icono de Blynk IoT

2. Una vez introducidas las credenciales de autenticación (usuario y contraseña) en la app, las cuales coinciden con las mismas credenciales de la plataforma web, aparecerá automáticamente tanto el *Template* como el *Device* anteriormente creado. En este momento solo tendrá que adicionar los *widgets* en el *dashboard* de la app y asociar cada instrumento de visualización con el pin virtual respectivo. La configuración se realiza de manera similar que en la plataforma web. Para más referencias remítase a los videos proporcionados en la sección de estudio previo y a la secuencia de figuras provista a en la figura 8.21:



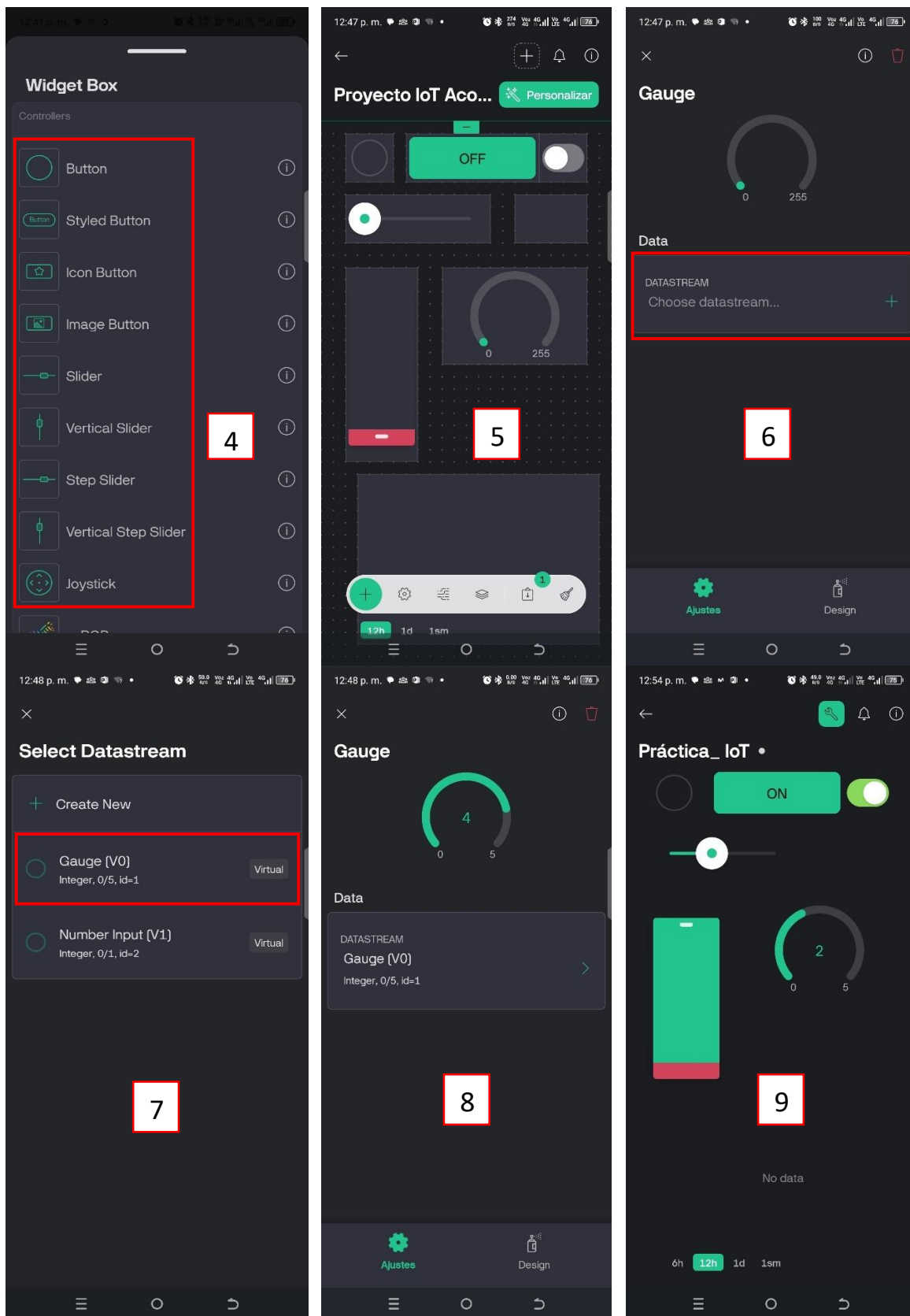


Figura 8.21. Secuencia de pasos para configurar la app IoT en Blynk

Implementación del código en LabVIEW

Para implementar la comunicación entre LabVIEW (Cliente) y el servidor web (Blynk Cloud) debe tener presente lo siguiente:

1. La dirección o URL del servicio web de *Blynk* es: <https://ny3.blynk.cloud/external/api/>
2. El método será: **update** ó **get** (dependiendo si es escritura o lectura respectivamente).

Los bloques o .vi que administran la conexión HTTP se pueden encontrar en la siguiente ruta (ver figura 8.22): *Programing/Data Communications/Protocols/HTTP Client*

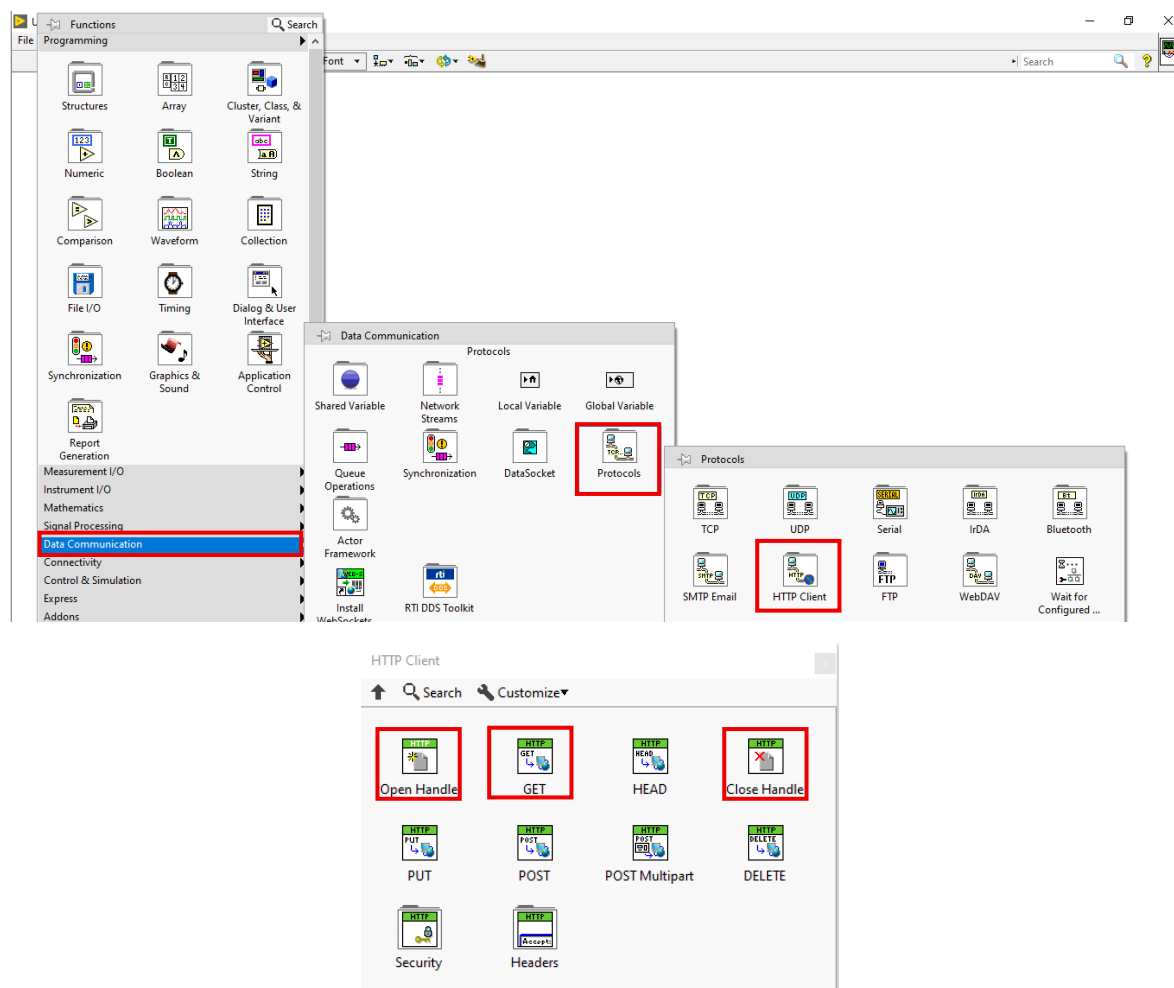


Figura 8.22. Ruta en LabVIEW para acceder a los bloques HTTP

En la figura 8.23, se muestra la forma de recibir los datos (dos variables analógicas) con LabVIEW obtenidos del proceso de adquisición de datos usando un Arduino. Nótese que el microcontrolador envía la siguiente trama para después ser desconcatenada por LabVIEW: **Volt1:valor1 Volt2:valor2&**. La aplicación se trata de un sistema que envía datos del régimen de giro de un motor [rpm] y de la posición del pedal del acelerador [%].

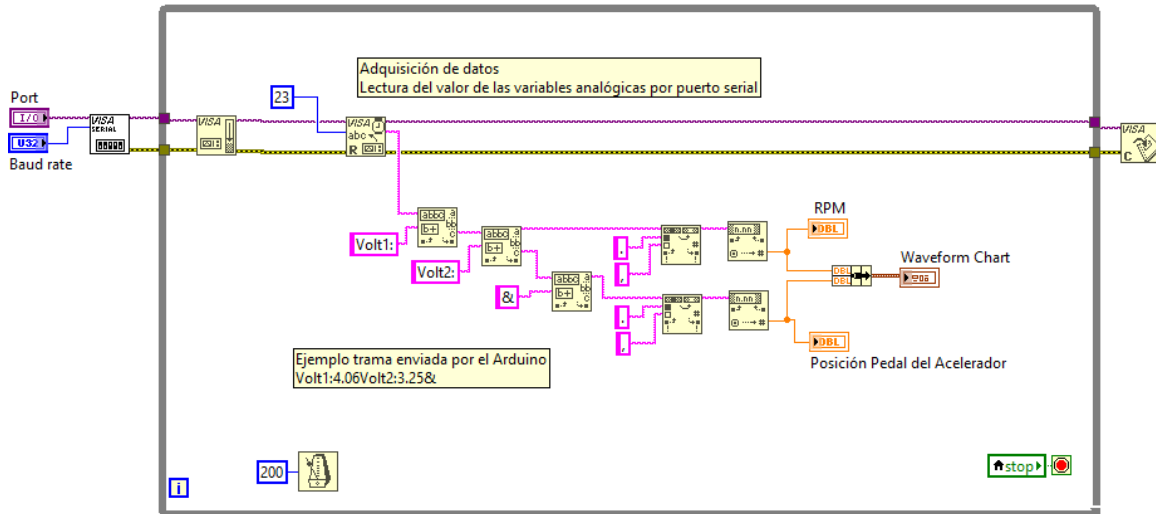


Figura 8.23. Código para desconcatenar dos variables analógicas enviadas por el puerto serial

Realice el montaje en hardware de la aplicación, conecte a su placa Arduino UNO, MEGA o Nano dos (2) sensores de su preferencia con salida analógica (pueden ser de temperatura, presión, posición, etc), caracterícelos y envíe por serial los datos de cada señal. Asegúrese de enviar la trama de datos en el formato especificado anteriormente.

A continuación, se presenta un código que puede usar como base para transmitir señales de voltaje.

El código programado en la placa Arduino es el siguiente:

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int sensorValue1 = analogRead(A0);
  int sensorValue2 = analogRead(A1);
  float voltage1 = sensorValue1 * (5.0 / 1023.0);
  float voltage2 = sensorValue2 * (5.0 / 1023.0);
  Serial.print("Volt1:");
  Serial.print(voltage1);
  Serial.print("Volt2:");
  Serial.print(voltage2);
  Serial.println("&");
  delay(200);
}
```



Figura 8.24. Código y conexión de hardware para al Arduino

En la figura 8.25 (lectura de datos) se muestra la forma de solicitar datos a la aplicación *Web* para publicarlos en LabVIEW, esto permitirá que un usuario remoto pueda modificar el estado de una variable del proceso (controlarlo) (ejemplo: Activar/desactivar un interruptor, controlar la posición de un *slider* para el control de velocidad de un motor, encender luces, etc).

Los pasos implementados son los siguientes:

1. Se abre el servicio *web HTTP* (*Open Handle.vi*)
2. Se establece la *URL* del servidor *web* que aloja la aplicación (en nuestro caso *blynk*), además se establece el método de **lectura** de datos (*get*).
3. Se concatena en un sólo *string* la solicitud o requerimiento:
URL del servicio web+método+API KEY+Nombre de la(s) variable(s). Se usa el carácter “&” como separador. Las variables que se desean leer son V0 y V1 en este caso.
4. Se adiciona el *API KEY* al *string* del paso anterior (3) en el formato especificado (en este caso %s = *string*). Para ello se usa el bloque “*Format into String.vi*”
5. Se envía la solicitud al servidor y se espera por su respectiva respuesta, la cual debe ser desconcatenada para ser visualizada en los indicadores respectivos independientemente.
6. Se cierra el servicio *web*.

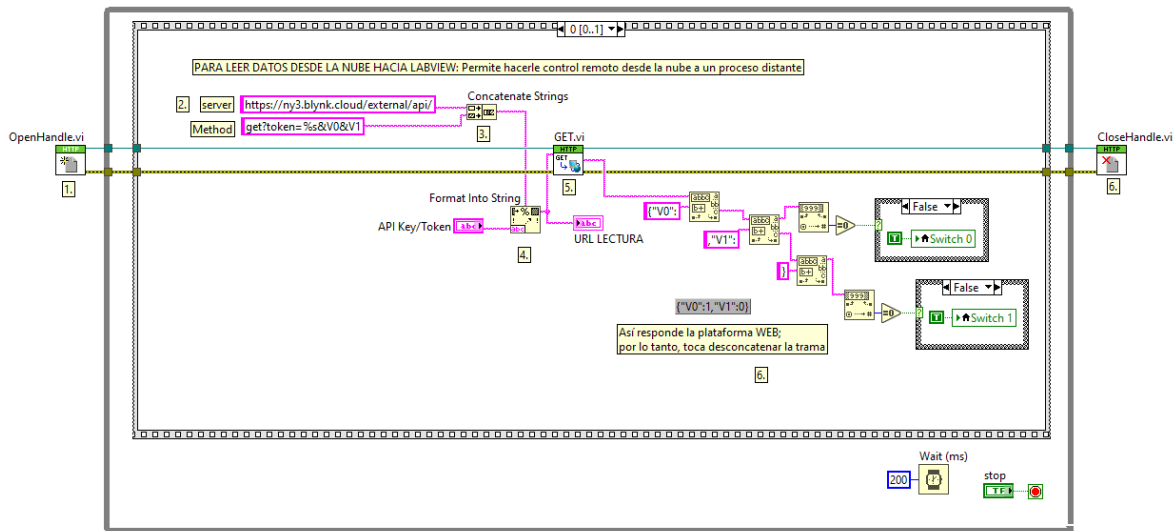


Figura 8.25. Código para lectura de variables usando HTTP

En la figura 8.26 (escritura de datos), se muestra la forma de enviar los datos obtenidos del proceso (motor) a la aplicación *web* usando protocolo *HTTP*, esto permitirá que un usuario remoto pueda monitorear el estado de las variables a distancia usando una conexión a internet. Como se observa en la gráfica, los pasos implementados son los siguientes:

1. Se abre el servicio *web HTTP* (*Open Handle.vi*)
2. Se establece la *URL* del servidor *web* que aloja la aplicación (en nuestro caso *blynk*), además se establece el método de **escritura** de datos (*update*).
3. Se concatena en un sólo *string* la solicitud o requerimiento:
URL del servicio web+método+API KEY+Nombre de la variable+valor de la variable.
4. Se adiciona tanto el *API KEY* como el valor de las variables al *string* del paso anterior (3) en el formato especificado (en este caso %s = *string*, %d= *decimal*). Para ello se usa el bloque “*Format into String.vi*”
5. Nótese que este procedimiento está enmarcado dentro de una estructura cíclica *FOR* la cual envía una solicitud cada vez por cada variable. En este caso no se pueden escribir simultáneamente varias variables al mismo tiempo, debe ser de manera multiplexada.
6. Se cierra el servicio *web*.

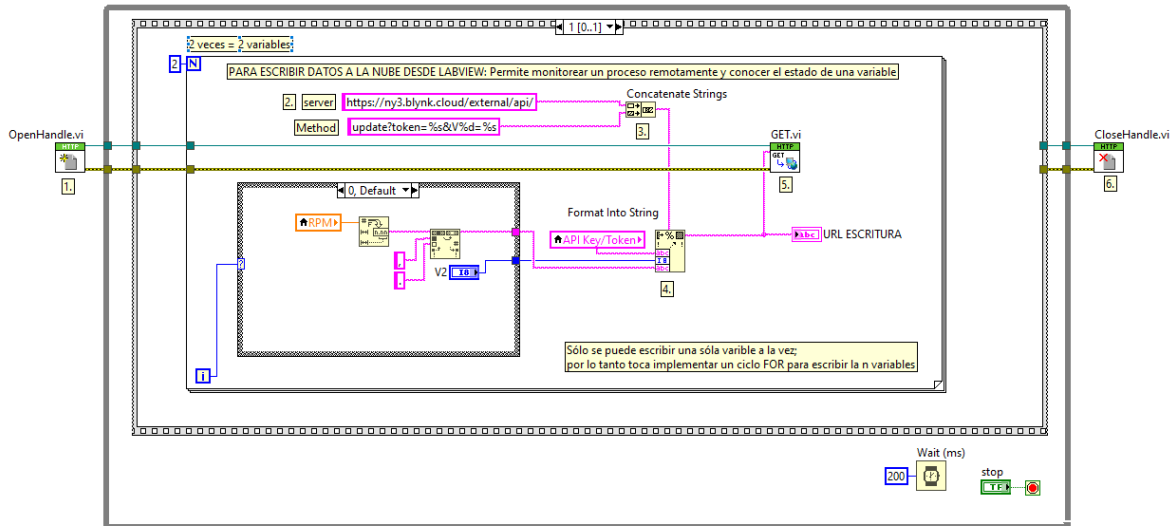


Figura 8.26. Envío de datos obtenidos del proceso a la aplicación *web* usando protocolo *HTTP*

En la Figura 8.27 se ilustra un ejemplo de la interfaz gráfica que representa los códigos mostrados en las figuras 8.25 y 8.26, también se puede ver *el Dashboard de Blynk Web y Blynk App* reportando las variables de la interfaz desarrollada en *LabVIEW*.

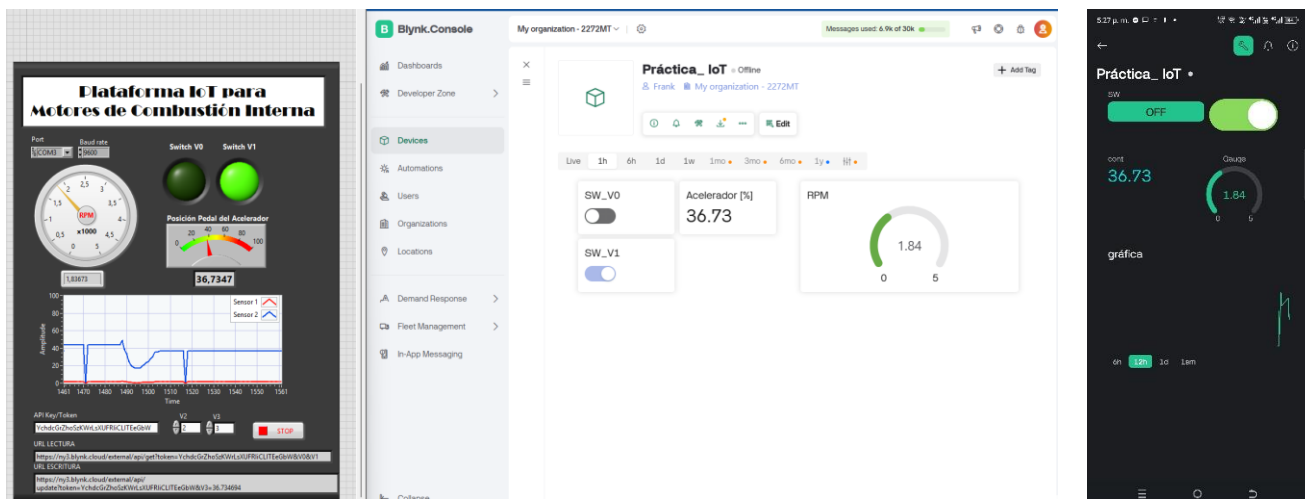


Figura 8.27. Interfaz de usuario para monitorear un proceso mediante IoT

Conclusiones

- Escriba las conclusiones más relevantes de la práctica de laboratorio, indicando hallazgos, dificultades y logros.